



PROGRAMACIÓN COMPETITIVA

Notebook

Python

Entrada/Salida • Estructuras de Datos • Grafos
Búsqueda y Optimización • Matemáticas y Simulación
Construcción y Combinatoria • Cadenas

Notebook de referencia para
ICPC • Codeforces • UVA
CSES • Gym

Compilado y mantenido por
David Santiago García Preciado
Ingeniería de Sistemas
Universidad El Bosque

Versión 1.0
Última actualización
24 de septiembre de 2026

Índice

1	Entrada/Salida Rápida en Python	2			
1.1	Lectura línea a línea (caso general)	2			
1.2	Lectura total + tokenización (máxima velocidad)	2			
1.3	Casos con bandera (condicional)	2			
1.4	Casos hasta EOF (sin contador T)	2			
1.5	Salida rápida	2			
2	Herramientas y Funciones Esenciales de Python	3			
2.1	Probar con un archivo de entrada	3			
2.2	Iteradores y funciones integradas	3			
2.3	Expresiones Regulares (módulo re)	4			
2.4	Diccionarios: dict, defaultdict y Counter	6			
2.5	Manipulación de cadenas	7			
2.6	Módulos matemáticos	9			
2.7	Librerías estándar útiles para ICPC	10			
3	Estructuras de Datos	13			
3.1	Fenwick Tree / BIT + Consultas Offline (distintos en un rango)	13			
3.2	Prefix Sums + Mapa de Primera Aparición (subarreglo balanceado más largo)	13			
3.3	Fenwick Tree con XOR + Sprague-Grundy (Nim por rangos)	14			
3.4	Segment Tree Iterativo (Actualización Puntual, Mínimo en Rango)	15			
3.5	Suma en Rango con Actualización en Rango (Dos Fenwick Trees)	16			
3.6	Sparse Table (Mínimo en Rango Estático, Consulta Constante)	17			
3.7	Rectángulo Máximo en Histograma (Pila Monótona)	18			
3.8	Sumas de Prefijos 2D (y Arreglo de Diferencias 2D)	18			
3.9	Ordered Set con Fenwick Tree (Compresión + K-ésimo por Descenso Binario)	20			
3.10	Balance de Paréntesis por Bloques de Nivel Superior (parejas incompatibles)	21			
3.11	Pila con Restricción de Capacidad (matrioshkas)	21			
3.12	Reducción con Pila (ciclos en un bang path)	22			
4	Algoritmos de Grafos	24			
4.1	Emparejamiento Bipartito Máximo, Kuhn (parejas de baile)	24			
4.2	Kuhn sobre Segmentos de Fila/Columna (torres con peones)	24			
4.3	Teorema de König: Cobertura Mínima de Vértices (quitar caballos)	25			
4.4	Componentes Fuertemente Conexas, Kosaraju (reinos de planetas)	26			
4.5	Circuito Euleriano, Hierholzer (ruta postal)	27			
4.6	Puntos de Articulación, Tarjan (islas críticas)	27			
4.7	Bron-Kerbosch: Clique Máximo (grupo compatible)	28			
4.8	Orden Topológico, Kahn + Propagación de Conteos (Digi Comp II)	29			
4.9	Union-Find / DSU con Path Halving (componentes conexas)	30			
4.10	Dijkstra con heapq (Caminos Mínimos desde un Origen)	31			
4.11	BFS en Grilla con Reconstrucción del Camino (y 0-1 BFS)	32			
4.12	Grafo Bipartito por Coloreo (y DFS Iterativo Genérico)	34			
4.13	Árbol de Expansión Mínima (Kruskal + Union-Find)	35			
4.14	Puentes y Árbol de Componentes 2-Arista-Conexas (Diámetro en una Pasada)	36			
4.15	Orden Topológico (Kahn) y DP sobre DAG con Reconstrucción	37			
4.16	Detección de Ciclo Negativo (Bellman-Ford + Ciclos en el Grafo de Padres)	38			
4.17	Distancias en Árbol con LCA por RMQ sobre el Preorden (Sparse Table)	40			
4.18	Corte Mínimo de Aristas con Flujo Máximo (Dinic)	41			
4.19	2-SAT con Componentes Fuertemente Conexas (Kosaraju)	42			
5	Búsqueda, Greedy y Optimización	44			
5.1	Búsqueda Binaria sobre la Respuesta + Two Pointers (k-ésima suma de pares)	44			
5.2	Enumeración por Bitmask (bits perdidos de un mensaje)	44			
5.3	Búsqueda Binaria sobre la Respuesta: Maximizar el Mínimo (muralla con refuerzo)	45			
5.4	Huffman: Fusión Greedy con Heap (teñir telas)	46			
6	Programación Dinámica	47			
6.1	DP de Particionamiento sobre Residuos (compra con redondeo)	47			
6.2	Distancia de Edición y LCS (DP Bit-Paralelo)	48			
6.3	Subsecuencia Creciente Más Larga (Arreglo de Colas + Búsqueda Binaria)	49			
6.4	Mochila 0/1 con DP Bit-Paralelo (SWAR)	50			
6.5	Conteo de Caminos en Grilla con Obstáculos (DP por Filas)	51			
6.6	DP en Árboles con Recorrido Iterativo (emparejamiento máximo)	52			
6.7	DP de Dígitos (Conteo en un Rango con Restricciones)	53			
6.8	DP de Intervalos (Corte Óptimo de una Barra)	53			
7	Matemáticas y Simulación	56			
7.1	Reducción Binaria tipo Josefo (fila con detector alternado)	56			
7.2	Fuerza Bruta / Simulación Directa (paradoja del cumpleaños)	56			
7.3	Simulación Lineal con Máximo de Prefijo (pares que se acercan)	56			
7.4	Simulación de Eventos con Colas (escalera doble)	57			
7.5	Simulación XOR con Detección de Ciclos (interruptores y bombillas)	58			
7.6	Invariante de Paridad por Ciclos de Permutación (rompecabezas deslizante)	58			
7.7	Descomposición en Fracciones Unitarias (Búsqueda Acotada tipo Erdős-Straus)	60			
8	Construcción y Combinatoria	61			
8.1	Constructivo por Invariante de Paridad (permutación con signos)	61			
8.2	Constructivo por Construcción Explícita (suma divisible por cada elemento)	61			
8.3	Conteo por Simetría de Binomios (subsecuencias con suma alternada 0)	62			
8.4	Greedy por Alternancia con Límite de Racha (dónde dormir cada noche)	62			
8.5	Stars and Bars con Factoriales Precalculados (días de espera entre oasis)	63			
9	Algoritmos de Cadenas (Strings)	64			
9.1	Rotación Mínima, Booth (huellas duplicadas por rotación)	64			
9.2	Normalización + Comparación con el Reverso (palíndromos)	64			
9.3	KMP sobre Codificación en Preorden (subárboles con la misma forma)	65			
9.4	Suffix Array + LCP de Kasai (subcadena repetida más larga)	66			
9.5	Firma Canónica + Hash Map (palabras con letras revueltas)	67			
9.6	Regex: findall y lookahead (conteo de patrones)	68			
9.7	Subcadenas Distintas con Restricción (Suffix Array + LCP + Dos Punteros)	70			
9.8	Z-Function (Bordes con Aparición Interior)	71			
9.9	Período Mínimo con la Función de Prefijos (KMP)	72			
9.10	Palíndromo Más Largo (Algoritmo de Manacher)	73			
9.11	Aho-Corasick (Conteo de Múltiples Patrones)	75			
10	Guía Rápida: Qué Usar y Cuándo	77			
	Referencias	83			

Este Notebook tiene una estructura simple. Cada entrada lleva el nombre de la técnica y, entre paréntesis, el problema con el que se ilustra; y siempre tiene las mismas partes: Preguntas guía para reconocer cuándo aplica, Utilidad, Complejidad, Fundamento teórico, Ejercicio aplicado, I/O (cómo se leyó la entrada y qué patrón de la sección 1 se usó) y Código solución verificado en juez.

1 Entrada/Salida Rápida en Python

En Python el I/O es el cuello de botella más común en ICPC. Estas son las 4 situaciones típicas y el patrón recomendado para cada una.

1.1 Lectura línea a línea (caso general)

Cuando cada línea tiene una cantidad de datos conocida y el número de líneas no es extremo (hasta 10^5). Es el patrón por defecto.

```
import sys
input = sys.stdin.buffer.readline
n = int(input())
arr = list(map(int, input().split()))
```

1.2 Lectura total + tokenización (máxima velocidad)

Cuando hay muchísimos números sueltos (10^5 – 10^6 +) sin importar en cuántas líneas están repartidos. Es la forma más rápida en Python, ya que es una sola llamada al sistema.

```
import sys
data = sys.stdin.buffer.read().split()
idx = 0 # el idx es lo que recorre el arreglo de tokens
n = int(data[idx]); idx += 1
arr = list(map(int, data[idx:idx + n])); idx += n
```

1.3 Casos con bandera (condicional)

El input termina cuando aparece un valor en concreto. No se sabe de antemano cuántos casos hay.

```
import sys
input = sys.stdin.buffer.readline
while True:
    a, b = map(int, input().split())
    if a == 0 and b == 0: # en este caso el valor concreto es "0 0"
        break
    # aquí se procesan los casos
```

1.4 Casos hasta EOF (sin contador T)

No hay número de casos declarado, por lo que se procesa hasta que no haya mas inputs.

```
import sys
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    # se procesa el caso
```

Una alternativa con los casos infinitos es usar el metodo de tokenización.

```
import sys
data = sys.stdin.buffer.read().split()
idx = 0
while idx < len(data):
    # se procesa el caso
```

1.5 Salida rápida

Nunca usar print() y mejor acumular resultados en una lista y escribir todo de una sola vez.

```
ans = []
# ans.append(result) dentro del loop
sys.stdout.write("\n".join(map(str, ans)) + "\n")
```

2 Herramientas y Funciones Esenciales de Python

2.1 Probar con un archivo de entrada

Copiar el caso de ejemplo a mano cada vez que se corre la solución es lento y propenso a errores. Lo práctico es dejarlo en un `input.txt` y que un script aparte se lo entregue al programa por `stdin`. Lo importante es que la solución NO se toca: lee de `stdin` igual que en el juez, así que el archivo que se prueba es exactamente el que se envía.

2.1.1 Lo mínimo: redirección de la terminal

Sin script y sin nada que instalar, la terminal ya conecta un archivo a `stdin` con <:

```
python sol.py < input.txt
pypy3 sol.py < input.txt          # PyPy, varias veces
                                  mas rapido
g++ -O2 -std=c++17 -o sol sol.cpp && ./sol < input.txt
```

2.1.2 Script de apoyo: un solo comando para Python y C++

El script siguiente recibe el nombre de la solución, le pasa `input.txt`, imprime la salida y el tiempo, y si se le da un archivo con la salida esperada la compara. Si la solución es `.cpp` la compila antes, de modo que el equipo usa el mismo comando sin importar el lenguaje.

```
python correr.py sol.py          # usa input.
                                txt
python correr.py sol.py caso.txt  # otra entrada
python correr.py sol.py caso.txt esp.txt # compara con
                                la esperada
python correr.py sol.cpp caso.txt esp.txt # compila con
                                g++ y ejecuta
```

```
import subprocess, sys, os, time

sol = sys.argv[1]                # sol.py o sol.
                                cpp
ent = sys.argv[2] if len(sys.argv) > 2 else "input.txt"

if sol.endswith(".py"):
    cmd = [sys.executable, sol]
else:                             # C++: compilar
    primero
    exe = os.path.abspath(sol.rsplit(".", 1)[0])
    if os.name == "nt": exe += ".exe" # ruta absoluta:
    Windows no busca en .
    if subprocess.run(["g++", "-O2", "-std=c++17",
                       "-o", exe, sol]).returncode:
        sys.exit("error de compilacion")
    cmd = [exe]

t = time.perf_counter()
with open(ent, "rb") as f:        # el archivo
    ENTRA por stdin
    r = subprocess.run(cmd, stdin=f, capture_output=True)
dt = time.perf_counter() - t

sal = r.stdout.decode()
print(sal, end="" if sal.endswith("\n") else "\n")
if r.stderr: print("[stderr]", r.stderr.decode())
print("-- %3fs codigo %d" % (dt, r.returncode))
```

```
if len(sys.argv) > 3:              # salida
    esperada
    esp = open(sys.argv[3]).read()
    print("OK" if sal.split() == esp.split()
          else "DIFERENTE, se esperaba:\n" + esp)
```

La comparación usa `.split()` en ambos lados, así que ignora espacios y saltos de línea sobrantes, que es como corrige la mayoría de jueces. El tiempo impreso sirve para saber si se está cerca del límite, y el código de salida delata un error de ejecución aunque el programa haya alcanzado a imprimir algo.

Solo usa `subprocess`, `sys`, `os` y `time`, los cuatro de la biblioteca estándar: no hay nada que instalar. Como la solución se ejecuta con `sys.executable`, el intérprete que mide es el mismo con el que se lanza el script; para cronometrar igual que el juez cuando se va a enviar en PyPy, hay que correrlo con `pypy3 correr.py sol.py`, no con `python`.

2.1.3 Si se ejecuta con el botón Run del editor

Cuando no hay dónde escribir argumentos ni redirección, la alternativa es redirigir `stdin` desde el propio código, con una condición que lo desactiva sola en el juez (allí no existe `input.txt`), de modo que tampoco hay que borrar nada antes de enviar:

```
import sys, os
if os.path.exists("input.txt"):    # en el juez no
    existe -> stdin real
    sys.stdin = open("input.txt")
```

Cuidado: lo que NO se debe hacer es leer el archivo directamente (`open("input.txt").read()`) en lugar de reasignar `sys.stdin`: eso obliga a editar la solución antes de enviarla, y si se olvida, el juez da error de ejecución porque ese archivo no existe.

2.2 Iteradores y funciones integradas

Estas funciones permiten recorrer estructuras de datos, transformar secuencias y escribir código más corto y legible sin sacrificar rendimiento.

2.2.1 `enumerate()`

Recorrer un iterable obteniendo simultáneamente el índice y el elemento, evitando mantener un contador manual.

```
arr = [5, 8, 2]
for i, x in enumerate(arr):
    print(i, x)
for i, x in enumerate(arr, 1):    # índices comienzan en
    print(i, x)                   1
```

Es especialmente útil cuando el problema utiliza índices 1-indexados.

2.2.2 `zip()`

Recorrer múltiples iterables en paralelo. El recorrido termina cuando el iterable más corto finaliza.

```
a = [1, 2, 3]
b = [4, 5, 6]
```

```
for x, y in zip(a, b):
    print(x, y)

pares = list(zip(a, b))
# [(1,4), (2,5), (3,6)]
```

También puede utilizarse para desempaquetar listas de tuplas.

```
pares = [(1, 4), (2, 5), (3, 6)]
a, b = zip(*pares)
print(a); print(b)
```

2.2.3 iter() y next()

iter() crea un iterador sobre cualquier iterable. next() consume el siguiente elemento del iterador. Se suelen usar cuando se procesan tokens manualmente.

```
data = ["5", "10", "20", "30"]
it = iter(data)
n = int(next(it))
arr = [int(next(it)) for _ in range(n - 1)]
```

También puede especificarse un valor por defecto.

```
x = next(it, None)
```

2.2.4 any() y all()

Evaluar rápidamente condiciones sobre un iterable.

any() devuelve True si al menos un elemento cumple la condición. all() devuelve True únicamente si todos los elementos la cumplen. Ambas realizan *short-circuit*, por lo que dejan de recorrer el iterable tan pronto conocen la respuesta.

```
if any(x % 2 == 0 for x in arr):
    print("Existe un numero par")
if all(x > 0 for x in arr):
    print("Todos son positivos")
```

2.2.5 map()

Aplica una función a todos los elementos de un iterable. Es muy utilizado durante la lectura de entrada.

```
nums = list(map(int, input().split()))
cuadrados = list(map(lambda x: x * x, nums))
```

También puede utilizarse con funciones propias.

```
def f(x):
    return x + 1
nuevo = list(map(f, nums))
```

2.2.6 sorted()

Devuelve una nueva lista ordenada sin modificar la original.

Puede ordenarse mediante una función key.

```
nums = sorted(nums)
nums = sorted(nums, reverse=True)
personas = sorted(personas, key=lambda x: x.edad)
pares = sorted(freq.items(), key=lambda kv: -kv[1])
```

Complejidad: $\mathcal{O}(n \log n)$

Python utiliza Timsort, un algoritmo estable muy eficiente para datos parcialmente ordenados.

2.2.7 reversed()

Permite recorrer una secuencia en orden inverso sin crear una copia.

```
for x in reversed(arr):
    print(x)
```

Es preferible a lo siguiente, que crea una copia de la lista:

```
arr[::-1]
```

2.2.8 encode() y decode()

Aunque rara vez aparecen en ICPC, pueden resultar útiles cuando un problema involucra archivos, comunicación o codificaciones específicas.

encode() convierte un string en bytes. decode() realiza la operación inversa.

```
s = "ICPC"
b = s.encode("utf-8")
print(b)
# b'ICPC'
texto = b.decode("utf-8")
print(texto)
# ICPC
```

Las codificaciones más comunes:

- utf-8
- ascii
- latin-1

En programación competitiva normalmente toda la entrada ya llega como texto UTF-8, por lo que estas funciones aparecen con poca frecuencia.

Resumen

Función	Uso principal
enumerate()	Recorrer con índice
zip()	Recorrer varios iterables
iter()	Crear un iterador
next()	Consumir siguiente elemento
any()	Existe algún elemento que cumpla
all()	Todos cumplen
map()	Aplicar función a todos los elementos
sorted()	Ordenar sin modificar el original
reversed()	Recorrer en orden inverso
encode()	String → bytes
decode()	Bytes → string

2.3 Expresiones Regulares (módulo re)

El módulo re permite buscar, validar, reemplazar y extraer información de cadenas mediante patrones.

En programación competitiva suele utilizarse cuando la entrada posee un formato irregular, contiene separadores complejos o es necesario validar cadenas.

```
import re
```

2.3.1 Funciones principales

Función	Descripción
re.match()	Busca únicamente al inicio del string.
re.search()	Busca la 1ra coincidencia en cualquier posición.
re.findall()	Devuelve todas las coincidencias en una lista.
re.finditer()	Devuelve un iter con todos los matches.
re.fullmatch()	Verifica que toda la cadena cumpla patrón.
re.sub()	Reemplaza coincidencias.
re.split()	Divide una cadena utilizando un patrón.
re.compile()	Compila una regex para reutilizarla.

2.3.2 Símbolos más utilizados

Patrón	Significado
.	Cualquier carácter excepto salto de línea
\d	Dígito
\D	No dígito
\w	Letra, número o _
\W	No alfanumérico
\s	Espacio en blanco
\S	No espacio
+	Una o más repeticiones
*	Cero o más repeticiones
?	Cero o una repetición
{n}	Exactamente n repeticiones
{n,m}	Entre n y m repeticiones
[]	Conjunto de caracteres
[^]	Negación del conjunto
()	Grupo de captura
(?:)	Grupo sin captura
	OR lógico
^	Inicio del string
\$	Final del string

2.3.3 Ejemplos básicos

```
import re
nums = re.findall(r'?\d+', s)
nums = list(map(int, nums))
tokens = re.split(r'\s+', s.strip())
m = re.match(r'^(\d+)-(\d+)$', s)
if m:
    a = int(m.group(1))
    b = int(m.group(2))
limpio = re.sub(r'[a-zA-Z0-9]', '', s)
```

2.3.4 Patrones útiles para ICPC

Patrón	Detecta
\d+	Enteros positivos
-?\d+	Enteros con signo
\d+\.\d+	Números decimales
[A-Za-z]+	Palabras
[A-Za-z0-9]+	Alfanuméricos
[aeiou]	Vocales
[^aeiou]	Todo excepto vocales
\s+	Uno o más espacios
\w+	Identificadores
.+	Cualquier texto

2.3.5 Combinaciones frecuentes

Extraer todos los enteros.

```
re.findall(r'?\d+', s)
```

Extraer únicamente números positivos.

```
re.findall(r'\d+', s)
```

Extraer palabras.

```
re.findall(r'[A-Za-z]+', s)
```

Extraer palabras y números.

```
re.findall(r'\w+', s)
```

Separar por múltiples espacios.

```
re.split(r'\s+', s.strip())
```

Eliminar caracteres especiales.

```
re.sub(r'^[A-Za-z0-9]', '', s)
```

Eliminar espacios repetidos.

```
re.sub(r'\s+', ' ', s)
```

Validar un correo electrónico sencillo.

```
re.fullmatch(r'\w+\@\w+\.\w+', s)
```

Validar una fecha DD/MM/AAAA.

```
re.fullmatch(r'\d{2}/\d{2}/\d{4}', s)
```

Validar un entero.

```
re.fullmatch(r'?\d+', s)
```

Buscar únicamente al inicio.

```
re.match(r'ICPC', s)
```

Buscar en cualquier posición.

```
re.search(r'ICPC', s)
```

2.3.6 Compilar una expresión regular

Cuando un mismo patrón será utilizado muchas veces, es recomendable compilarlo una única vez. Esto evita recompilar la expresión regular en cada llamada.

```
pat = re.compile(r'-?\d+')
for linea in texto:
    nums = pat.findall(linea)
```

2.3.7 Complejidad

La mayoría de operaciones tienen complejidad aproximadamente $\mathcal{O}(n)$ respecto al tamaño del texto, aunque algunos patrones muy complejos pueden provocar retroceso (*backtracking*) y aumentar considerablemente el tiempo de ejecución.

En problemas de ICPC normalmente se utilizan patrones simples, por lo que el rendimiento suele ser lineal.

Resumen

Función	Uso principal
match	Buscar al inicio
search	Buscar primera coincidencia
findall	Obtener todas las coincidencias
finditer	Iterar sobre coincidencias
fullmatch	Validar una cadena completa
split	Separar mediante un patrón
sub	Reemplazar texto
compile	Reutilizar un patrón

2.4 Diccionarios: dict, defaultdict y Counter

Los diccionarios son una de las estructuras de datos más utilizadas en programación competitiva. Permiten almacenar pares **clave** : **valor** con acceso promedio en tiempo constante.

En Python, la implementación utiliza una tabla hash, por lo que la mayoría de operaciones tienen complejidad promedio $\mathcal{O}(1)$ aunque en el peor caso pueden degradarse a $\mathcal{O}(n)$

si existen muchas colisiones.

2.4.1 Crear un diccionario

```
freq = {}
persona = {"nombre": "David", "edad": 20}
```

2.4.2 Acceso y recuperación

Acceder directamente mediante corchetes genera un **KeyError** si la clave no existe.

```
x = freq["a"]
```

Por ello normalmente es preferible utilizar `get()`.

```
freq.get("a")
freq.get("a", 0)
freq.get("a", [])
```

Las funciones principales de recuperación son:

Función	Descripción
get()	Obtiene un valor sin lanzar excepción.
keys()	Devuelve todas las claves.
values()	Devuelve todos los valores.
items()	Devuelve pares (clave, valor).

Ejemplos:

```
for k in freq.keys():
    ...
for v in freq.values():
    ...
for k, v in freq.items():
    ...
```

2.4.3 Modificar un diccionario

Insertar o modificar una clave.

```
freq["a"] = 5
```

Actualizar múltiples valores.

```
a = {"x":1}; b = {"y":2}
a.update(b)
```

Crear una clave únicamente si no existe.

```
freq.setdefault("python", [])
freq["python"].append(10)
```

2.4.4 Eliminar elementos

Eliminar una clave.

```
freq.pop("a")
```

Eliminar devolviendo un valor por defecto.

```
freq.pop("a", 0)
```

Eliminar el último elemento insertado.

```
freq.popitem()
```

Vaciar completamente el diccionario.

```
freq.clear()
```

2.4.5 Funciones integradas

Cantidad de elementos.

```
len(freq)
```

Copiar un diccionario.

```
nuevo = freq.copy()
```

Comprobar existencia.

```
if "python" in freq:
    ...
```

Eliminar una copia sin modificar el original.

```
nuevo = dict(freq)
```

2.4.6 Ordenar un diccionario

Ordenar por clave.

```
sorted(freq.items())
```

Ordenar por valor.

```
sorted(freq.items(), key=lambda kv: kv[1])
```

Orden descendente.

```
sorted(freq.items(), key=lambda kv: -kv[1])
```

2.4.7 defaultdict

Normalmente antes de incrementar una frecuencia se necesita comprobar si la clave existe.

```
freq = {}  
freq[x] = freq.get(x, 0) + 1
```

Con defaultdict esto desaparece.

```
from collections import defaultdict  
  
freq = defaultdict(int)  
  
freq[x] += 1
```

También puede utilizarse para listas.

```
grupos = defaultdict(list)  
for x in arr:  
    grupos[x % 3].append(x)
```

Con conjuntos.

```
adj = defaultdict(set)  
adj[u].add(v)
```

Y con cualquier función.

```
from collections import defaultdict  
d = defaultdict(lambda: -1)
```

2.4.8 Counter

Cuando únicamente interesa contar frecuencias, la clase Counter simplifica mucho el código.

```
from collections import Counter  
freq = Counter(arr)
```

Obtener el elemento más frecuente.

```
freq.most_common(1)
```

Los tres más frecuentes.

```
freq.most_common(3)
```

Frecuencia de un elemento.

```
freq[x]
```

2.4.9 Aplicaciones frecuentes en ICPC

Contar frecuencias.

```
freq = defaultdict(int)  
for x in arr:  
    freq[x] += 1
```

Agrupar elementos.

```
g = defaultdict(list)  
for x in arr:  
    g[x % k].append(x)
```

Construir listas de adyacencia.

```
adj = defaultdict(list)  
for u, v in edges:  
    adj[u].append(v)
```

Construir grafos sin comprobar existencia.

```
adj = defaultdict(set)  
adj[u].add(v)
```

Contar caracteres.

```
Counter(s)
```

Resumen

Función	Uso principal
get()	Obtener valor sin KeyError
keys()	Recorrer claves
values()	Recorrer valores
items()	Recorrer clave y valor
update()	Actualizar con otro diccionario
setdefault()	Crear si no existe
pop()	Eliminar una clave
popitem()	Eliminar último elemento
clear()	Vaciar el diccionario
copy()	Copia superficial
len()	Número de elementos
defaultdict	Valor por defecto automático
Counter	Conteo automático de frecuencias

2.5 Manipulación de cadenas

Las cadenas (str) son inmutables en Python, por lo que todas las operaciones que modifican su contenido devuelven una nueva cadena.

La mayoría de problemas de parsing, simulación y procesamiento de texto utilizan estas funciones.

2.5.1 replace()

Reemplaza ocurrencias de una subcadena.

```
s = "banana"  
s.replace("a", "o")  
# "bonono"  
s.replace("a", "o", 2)  
# "bonona"
```

Complejidad promedio: $\mathcal{O}(n)$

2.5.2 split()

Divide una cadena utilizando un separador.

```
s.split()
s.split(",")
s.split(":")
s.split("\n")
```

Si no se especifica un separador, utiliza cualquier cantidad de espacios consecutivos.

```
s = "a  b  c"
print(s.split())
# ['a', 'b', 'c']
```

2.5.3 join()

Une una colección de cadenas utilizando un separador.

Es mucho más eficiente que concatenar strings dentro de un ciclo.

```
nums = ["1", "2", "3"]
", ".join(nums)
# "1,2,3"
" ".join(nums)
# "1 2 3"
```

Complejidad: $\mathcal{O}(n)$

2.5.4 strip(), lstrip() y rstrip()

Eliminan caracteres al inicio y/o final.

```
s.strip()
s.lstrip()
s.rstrip()
```

También pueden eliminar caracteres específicos.

```
--ICPC--".strip("--")
# ICPC
```

2.5.5 find() y rfind()

Buscan una subcadena.

Devuelven

- la posición encontrada;
- -1 si no existe.

```
s.find("abc")
s.rfind("abc")
```

2.5.6 count()

Cuenta cuántas veces aparece una subcadena.

```
s.count("ab")
```

2.5.7 startswith() y endswith()

Permiten verificar prefijos y sufijos.

```
s.startswith("abc")
s.endswith(".cpp")
```

Muy útiles para parsing.

2.5.8 lower(), upper() y capitalize()

Cambian el formato de las letras.

```
s.lower()
s.upper()
s.capitalize()
```

2.5.9 isalpha(), isdigit(), isalnum()

Verifican el contenido de una cadena.

```
s.isalpha()
s.isdigit()
s.isalnum()
s.isspace()
```

2.5.10 partition()

Divide únicamente en la primera aparición del separador.

```
s.partition("=")
# ('x', '=', '15')
```

A diferencia de split(), siempre devuelve exactamente tres elementos.

2.5.11 Concatenación

```
a + b
a += b
```

Cuando se concatenan muchas cadenas es preferible utilizar

```
"".join(lista)
```

ya que evita crear múltiples copias intermedias.

2.5.12 Formateo de cadenas

Python permite insertar variables directamente mediante f-strings.

```
x = 3.14159265
f"{x:.2f}"
# 3.14
f"{x:.10f}"
# 3.1415926500
nombre = "ICPC"
f"Hola {nombre}"
```

Este mecanismo es el recomendado para imprimir números reales.

2.5.13 Aplicaciones frecuentes en ICPC

Eliminar espacios al leer una línea.

```
s = input().strip()
```

Separar una línea.

```
tokens = input().split()
```

Unir la respuesta.

```
print(" ".join(ans))
```

Comprobar un prefijo.

```
if s.startswith("YES"):
    ...
```

Eliminar caracteres.

```
s = s.replace(" ", "")
```

Buscar una palabra.

```
if s.find("abc") != -1:
    ...
```

Contar caracteres.

```
vocales = s.count("a")
```

Resumen

Función	Uso principal
replace()	Reemplazar texto
split()	Dividir una cadena
join()	Unir cadenas
strip()	Eliminar espacios
find()	Buscar subcadena
rfind()	Buscar desde el final
count()	Contar ocurrencias
startswith()	Verificar prefijo
endswith()	Verificar sufijo
lower()	Minúsculas
upper()	Mayúsculas
capitalize()	Primera letra mayúscula
isalpha()	Solo letras
isdigit()	Solo dígitos
isalnum()	Alfanumérico
isspace()	Solo espacios
partition()	Dividir una vez

2.6 Módulos matemáticos

Python incluye varios módulos que simplifican operaciones matemáticas frecuentes en programación competitiva.

El más utilizado es `math`.

```
import math
```

2.6.1 Redondeo

Función	Descripción
round(x,n)	Redondea a n decimales
math.floor(x)	Entero menor o igual
math.ceil(x)	Entero mayor o igual
math.trunc(x)	Elimina la parte decimal

```
import math
math.floor(3.8) # 3
math.ceil(3.2) # 4
round(3.141592,2) # 3.14
math.trunc(7.98) # 7
factor = 10 ** N
floor_n = math.floor(x * factor) / factor
ceil_n = math.ceil(x * factor) / factor
```

2.6.2 Valor absoluto

```
abs(x)
math.fabs(x)
```

Normalmente basta utilizar

```
abs(x)
```

2.6.3 Raíces

Función	Uso
math.sqrt(x)	Raíz cuadrada
math.isqrt(x)	Raíz entera exacta

```
math.sqrt(25) # 5.0
math.isqrt(30) # 5
```

En teoría de números suele preferirse `isqrt()`.

2.6.4 Máximo común divisor y mínimo común múltiplo

```
math.gcd(a,b)
math.lcm(a,b)
```

También aceptan más de dos argumentos.

```
math.gcd(a,b,c)
math.lcm(a,b,c)
```

Complejidad: $\mathcal{O}(\log(\min(a,b)))$

2.6.5 Potencias

```
pow(a,b)
pow(a,b,m)
a ** b
math.pow(a,b)
```

Observaciones.

- `pow(a,b,m)` calcula la potencia módulo m de forma eficiente.
- `**` devuelve enteros cuando es posible.
- `math.pow()` siempre devuelve float.

2.6.6 Factoriales

```
math.factorial(n)
```

Complejidad: $\mathcal{O}(n)$

2.6.7 Combinatoria

Desde Python 3.8 existen funciones incorporadas.

```
math.comb(n,k)
math.perm(n,k)
```

Ejemplos.

```
math.comb(5,2) # 10
math.perm(5,2) # 20
```

2.6.8 Logaritmos

```
math.log(x)
math.log2(x)
math.log10(x)
```

2.6.9 Trigonometría

```
math.sin(x)
math.cos(x)
math.tan(x)
math.asin(x)
math.acos(x)
math.atan(x)
```

Los ángulos siempre se expresan en radianes.

2.6.10 Distancias

```
math.dist(p1,p2)
math.hypot(x,y)
```

```
math.hypot(3,4) # 5
```

2.6.11 Producto

```
math.prod(lista)
```

Ejemplo.

```
math.prod([2,3,4]) # 24
```

2.6.12 Constantes

```
math.pi
math.e
math.inf
math.nan
```

2.6.13 Fracciones exactas

Cuando los números en punto flotante producen errores de precisión puede utilizarse el módulo

```
from fractions import Fraction
```

```
Fraction(1,3)
Fraction(1,6)
Fraction(1,3)+Fraction(1,6)
```

Resultado.

```
Fraction(1,2)
```

No existe error de redondeo.

2.6.14 División entera

En Python la división entera siempre redondea hacia menos infinito.

```
7 // 2
# 3
```

```
-7 // 2
# -4
```

Este comportamiento es diferente al de C++ y Java.

2.6.15 Aplicaciones frecuentes en ICPC

Máximo común divisor.

```
math.gcd(a,b)
```

Potencia modular.

```
pow(a,b,MOD)
```

Raíz entera.

```
math.isqrt(n)
```

Binomiales.

```
math.comb(n,k)
```

Permutaciones.

```
math.perm(n,k)
```

Fracciones exactas.

```
Fraction(a,b)
```

Resumen

Función	Uso principal
<code>floor()</code>	Redondear hacia abajo
<code>ceil()</code>	Redondear hacia arriba
<code>trunc()</code>	Eliminar parte decimal
<code>gcd()</code>	Máximo común divisor
<code>lcm()</code>	Mínimo común múltiplo
<code>sqrt()</code>	Raíz cuadrada
<code>isqrt()</code>	Raíz entera
<code>factorial()</code>	Factorial
<code>comb()</code>	Combinaciones
<code>perm()</code>	Permutaciones
<code>pow()</code>	Potencias
<code>log2()</code>	Logaritmo base 2
<code>hypot()</code>	Distancia euclidiana
<code>prod()</code>	Producto de un iterable
<code>Fraction</code>	Fracciones exactas

2.7 Librerías estándar útiles para ICPC

Python posee una biblioteca estándar muy completa. Conocer estos módulos permite resolver muchos problemas sin implementar estructuras o algoritmos desde cero.

2.7.1 collections

Contiene estructuras de datos optimizadas.

```
from collections import deque
from collections import defaultdict
from collections import Counter
from collections import OrderedDict
from collections import namedtuple
```

Clase	Uso
deque	Cola doble, BFS, Sliding Window
defaultdict	Dict con valor por defecto
Counter	Conteo automático de frecuencias
OrderedDict	Dict que conserva operaciones de orden
namedtuple	Tuplas con nombres para los campos

a) deque

Permite insertar y eliminar elementos por ambos extremos en $\mathcal{O}(1)$

```
from collections import deque
q = deque()
q.append(5)
q.appendleft(2)
q.pop()
q.popleft()
q.rotate(1)
q.rotate(-1)
```

Es la estructura estándar para implementar BFS.

b) defaultdict

Ya estudiado en la sección anterior.

Se utiliza principalmente para:

- frecuencias;
- listas de adyacencia;
- agrupaciones.

c) Counter

Muy útil para conteo de frecuencias.

```
freq = Counter(arr)
freq.most_common(3)
```

2.7.2 heapq

Implementa un montículo mínimo (Min Heap).

```
import heapq
```

Funciones principales.

Función	Uso
heappush()	Insertar
heappop()	Extraer mínimo
heapify()	Construir heap
heapreplace()	Reemplazar mínimo
nlargest()	Mayores elementos
nsmallest()	Menores elementos

Ejemplo.

```
heap=[]
heapq.heappush(heap,4)
heapq.heappush(heap,2)
heapq.heappush(heap,8)
heapq.heappop(heap)
```

Todas las operaciones de inserción y extracción tienen complejidad $\mathcal{O}(\log n)$

Se utiliza principalmente en:

- Dijkstra;

- Prim;
- Huffman;
- Top-K.

2.7.3 bisect

Permite realizar búsqueda binaria sobre listas ordenadas.

```
from bisect import bisect_left
from bisect import bisect_right
from bisect import insort
```

```
bisect_left(arr,x)
bisect_right(arr,x)
insort(arr,x)
```

Todas las búsquedas tienen complejidad $\mathcal{O}(\log n)$

2.7.4 functools

Funciones de orden superior.

```
from functools import lru_cache
from functools import cache
from functools import cmp_to_key
from functools import reduce
```

a) lru_cache

Muy utilizado para memoización.

```
@lru_cache(None)
def dp(i,j):
    ...
```

b) cmp_to_key

Permite utilizar comparadores personalizados.

```
sorted(arr,key=cmp_to_key(cmp))
```

2.7.5 itertools

Herramientas para trabajar con iteradores.

```
from itertools import permutations
from itertools import combinations
from itertools import combinations_with_replacement
from itertools import product
from itertools import accumulate
from itertools import cycle
from itertools import repeat
from itertools import pairwise
```

Función	Uso
permutations	Permutaciones
combinations	Combinaciones
product	Producto cartesiano
accumulate	Prefijos acumulados
cycle	Ciclo infinito
repeat	Repetición
pairwise	Pares consecutivos

Ejemplos.

```
permutations(arr)
combinations(arr,2)
```

```
product(a,b)
accumulate(arr)
```

2.7.6 operator

Contiene operadores como funciones.

```
from operator import itemgetter
from operator import attrgetter
```

Ejemplo.

```
sorted(arr, key=itemgetter(1))
```

2.7.7 decimal

Permite realizar aritmética decimal exacta.

```
from decimal import Decimal
```

Normalmente no se utiliza en ICPC debido a su menor rendimiento.

2.7.8 fractions

Permite trabajar con fracciones exactas.

```
from fractions import Fraction
```

Muy útil cuando se desea evitar errores de punto flotante.

2.7.9 array

Implementa arreglos homogéneos.

```
from array import array
```

Consumen menos memoria que una lista convencional.

2.7.10 queue

```
from queue import PriorityQueue
```

Aunque implementa colas de prioridad, en programación competitiva normalmente se prefiere utilizar

```
heapq
```

por ser considerablemente más rápido.

2.7.11 sys

El módulo más utilizado para entrada y salida rápida.

```
import sys
input = sys.stdin.readline
sys.setrecursionlimit(200000)
```

2.7.12 Aplicaciones frecuentes en ICPC

Problema	Librería recomendada
BFS	deque
Dijkstra	heapq
Memoización	lru_cache
Conteo	Counter
Frecuencias	defaultdict
Binary Search	bisect
Permutaciones	itertools
Combinaciones	itertools
Ordenamientos personalizados	cmp_to_key
Prefijos	accumulate

Resumen

Módulo	Uso principal
collections	Estructuras de datos
heapq	Heap mínimo
bisect	Búsqueda binaria
functools	Memoización y comparadores
itertools	Iteradores avanzados
operator	Funciones auxiliares para ordenar
fractions	Fracciones exactas
decimal	Decimales exactos
array	Arreglos homogéneos
queue	Colas sincronizadas
sys	Entrada rápida y recursión

3 Estructuras de Datos

3.1 Fenwick Tree / BIT + Consultas Offline (distintos en un rango)

Preguntas guía:

- ¿Necesito sumas o conteos acumulados sobre un rango que cambia con actualizaciones puntuales?
- ¿Los updates son de un solo índice y las queries son sobre prefijos o rangos $[l, r]$?
- ¿Si no hay updates "online", puedo resolver offline ordenando las queries (p. ej. por su extremo derecho)?

Utilidad: Mantener sumas de prefijo (o cualquier operación invertible) sobre un arreglo con actualizaciones puntuales, respondiendo consultas de rango en tiempo logarítmico. Es la base de técnicas como conteo de elementos distintos por rango, inversiones en un arreglo y frecuencias acumuladas.

Complejidad: Construcción $\mathcal{O}(n)$. Update puntual $\mathcal{O}(\log n)$. Query de prefijo $\mathcal{O}(\log n)$. Espacio $\mathcal{O}(n)$.

Fundamento teórico: Cada posición i del arreglo interno almacena la suma de un bloque de tamaño $(i \& -i)$, el bit menos significativo de i . Al actualizar se "sube" sumando $(i \& -i)$; al consultar un prefijo se "baja" restando $(i \& -i)$. Esto aprovecha la representación binaria de los índices para cubrir cualquier prefijo con solo $\mathcal{O}(\log n)$ bloques. [1]

Ejercicio aplicado

(SPOJ DQUERY — D-query) Dado un arreglo de n números y q consultas (i, j) , responder para cada una cuántos valores distintos hay en el subarreglo $[i, j]$. Técnica: procesar las consultas offline ordenadas por extremo derecho r , recorriendo el arreglo de izquierda a derecha y manteniendo en el BIT un "1" solo en la última aparición de cada valor.

I/O: $n \leq 30\,000$ y $q \leq 200\,000$ hasta $\sim 430\,000$ tokens de entrada: se usa lectura total + tokenización (patrón 2) en vez de readline por línea.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.readline
    n = int(data())
    nums = list(map(int, data().split()))
    q = int(data())

    queries = []
    for i in range(q):
        l, r = map(int, data().split())
        queries.append((r, l, i))  # se ordena por r
    (offline)
    queries.sort()

    bit = [0] * (n + 1)

    def update(i, val):
        while i <= n:
            bit[i] += val
            i += i & (-i)  # ultimo bit encendido

    def query(i):
        res = 0
        while i > 0:
            res += bit[i]
            i -= i & (-i)
        return res

    last = {}  # ultima posicion vista de cada valor
    ans = [0] * q
    cur = 0
    for r, l, qi in queries:
        while cur < r:
            cur += 1
            x = nums[cur - 1]
            prev = last.get(x)
            if prev:
                update(prev, -1)  # desactiva la aparicion anterior
            update(cur, 1)  # activa la aparicion actual
            last[x] = cur
            ans[qi] = query(r) - query(l - 1)

    sys.stdout.write("\n".join(map(str, ans)))

solve()
```

```
def query(i):
    res = 0
    while i > 0:
        res += bit[i]
        i -= i & (-i)
    return res

last = {}  # ultima posicion vista de cada valor
ans = [0] * q
cur = 0
for r, l, qi in queries:
    while cur < r:
        cur += 1
        x = nums[cur - 1]
        prev = last.get(x)
        if prev:
            update(prev, -1)  # desactiva la aparicion anterior
        update(cur, 1)  # activa la aparicion actual
        last[x] = cur
        ans[qi] = query(r) - query(l - 1)

sys.stdout.write("\n".join(map(str, ans)))

solve()
```

3.2 Prefix Sums + Mapa de Primera Aparición (subarreglo balanceado más largo)

Preguntas guía:

- ¿Piden la subsecuencia contigua más larga que cumple una condición de "balance" (mismo número de dos tipos de elementos, o suma total igual a un valor fijo)?
- ¿Puedo transformar cada elemento en un delta $(+1/-1/0)$ tal que la condición se traduzca en "suma de un subarreglo == 0"?
- ¿Me alcanza con guardar, para cada valor de suma acumulada, solo la PRIMERA posición donde apareció?

Utilidad: Encontrar en $\mathcal{O}(n)$ el subarreglo contiguo más largo cuya suma (o balance entre dos tipos de elementos) es igual a un valor fijo, sin revisar los $\mathcal{O}(n^2)$ subarreglos posibles.

Complejidad: $\mathcal{O}(n)$ tiempo, $\mathcal{O}(n)$ espacio.

Fundamento teórico: Si $P[i]$ es la suma acumulada de los primeros i elementos (con la transformación adecuada), la suma del subarreglo $(i + 1..j)$ es $P[j] - P[i]$. Esa suma es 0 exactamente cuando $P[j] = P[i]$. Para maximizar la longitud $j - i$, basta con recordar, para cada valor de prefijo visto, la PRIMERA posición en que apareció (nunca la última), y comparar contra ella cada vez que el valor se repite.

Ejercicio aplicado

Dada una secuencia de N transacciones (positiva = depósito, negativa = retiro, cero = consulta), calcular el depósito máximo, el retiro más negativo, y la longitud del subperíodo contiguo más largo con igual cantidad de depósitos y retiros.

I/O: Múltiples casos con centinela "0" (patrón 3). $N \leq 10\,000 \rightarrow$ readline por línea (patrón 1); se probó también con lectura total y resultó más lenta para este patrón de "un entero, luego un arreglo, repetido" ($\sim 2.5s$ vs $\sim 3.9s$ en 500 casos de $N=10\,000$).

Código solución

```
import sys
input = sys.stdin.buffer.readline

def solve():
    out = []
    while True:
        n = int(input())
        if n == 0:
            break
        arr = list(map(int, input().split()))

        d = 0
        w = 0
        for x in arr:
            if x > 0:
                d = max(d, x)
            elif x < 0:
                w = min(w, x) # el mas negativo, no
                               # el de mayor magnitud

        # prefijo: +1 por deposito, -1 por retiro, 0 no
        # cambia nada
        first_seen = {0: 0} # valor de prefijo ->
        PRIMERA posicion vista
        prefix = 0
        r = 0
        for i, x in enumerate(arr, start=1):
            if x > 0:
                prefix += 1
            elif x < 0:
                prefix -= 1
            if prefix in first_seen:
                r = max(r, i - first_seen[prefix]) # se
            repite: hay balance
            else:
                first_seen[prefix] = i #
                primera vez: se guarda

        out.append(f"{d} {w} {r}")
        sys.stdout.write("\n".join(out) + "\n")
    solve()
```

3.3 Fenwick Tree con XOR + Sprague-Grundy (Nim por rangos)

Preguntas guía:

- ¿El juego es una suma de sub-juegos independientes (aquí: columnas de fragmentos) donde en cada turno se modifica UNA sola pila?
- ¿Necesito responder "quién gana" para muchos rangos distintos, con actualizaciones puntuales intercaladas?
- ¿La operación de combinación relevante es XOR en vez de suma?

Utilidad: Generalizar el Fenwick Tree (visto para sumas) a CUALQUIER operación invertible para mantener el valor de Grundy combinado de un rango de pilas de Nim, con actualizaciones puntuales en tiempo logarítmico. Combinado con el teorema de Sprague-Grundy/Bouton,

responde instantáneamente quién gana un juego de Nim sobre cualquier subrango.

Complejidad: Construcción $\mathcal{O}(n)$. Update puntual $\mathcal{O}(\log n)$. Query de rango $\mathcal{O}(\log n)$.

Fundamento teórico: En Nim con pilas independientes donde un jugador retira cualquier cantidad positiva de UNA pila, el valor de Grundy de una pila de tamaño v es exactamente v . Por el teorema de Bouton, el jugador que mueve PRIMERO gana si y solo si el XOR de todas las pilas en juego es distinto de 0. Como XOR es su propio inverso ($v \oplus v = 0$), el mismo esquema del Fenwick Tree que acumula sumas de prefijo funciona igual acumulando XOR de prefijo: para modificar el valor de una posición de v_{viejo} a v_{nuevo} , basta aplicar un update con $\Delta = v_{viejo} \oplus v_{nuevo}$ (equivalente a "quitar" el valor viejo y "poner" el nuevo en una sola operación). [2], [3]

Ejercicio aplicado

Juan y Frank juegan Nim sobre columnas de fragmentos: en cada turno, un jugador retira cualquier cantidad positiva de fragmentos de UNA columna; pierde quien no puede mover. Dadas Q consultas, cada una una "profecía" (determinar el ganador en un rango $[l, r]$, sabiendo que Frank siempre empieza) o un "ritual" (sumar x fragmentos a una columna), resolver cada profecía.

I/O: $N, Q \leq 10^6 \rightarrow$ lectura total con tokenización (patrón 2), y las funciones de update/query del Fenwick se escriben SIN llamadas a función (inline) para reducir el overhead de Python en este volumen de operaciones. Aun así, en el peor caso ($N=Q=10^6$) la solución pura en Python ronda $\sim 7s$ si el juez es muy estricto con el límite de tiempo, este es el techo práctico de Python puro para este volumen sin cambiar de lenguaje.

Código solución

```
import sys

def main():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); q = int(data[idx + 1]); idx += 2
    arr = [0] * (n + 1)
    tree = [0] * (n + 1)

    # construccion en O(n): cada nodo se combina UNA vez
    # con su padre directo,
    # en vez de hacer n updates completos de O(log n)
    # cada uno
    for i in range(1, n + 1):
        v = int(data[idx]); idx += 1
        arr[i] = v
        tree[i] ^= v
        j = i + (i & (-i))
        if j <= n:
            tree[j] ^= tree[i]

    out = []
    out_append = out.append
    ANS = ("JUAN", "FRANK")
    P_BYTE = 80 # ord('P'): comparar el primer byte
    # evita decodificar el token

    i2 = idx
    for _ in range(q):
        tok = data[i2]
```

```

if tok[0] == P_BYTE:
    l = int(data[i2 + 1]); r = int(data[i2 + 2]);
    i2 += 3
    # xor(prefijo r) ^ xor(prefijo l-1) = xor del
    rango [l, r]
    res = 0
    i = r
    while i:
        res ^= tree[i]
        i &= i - 1          # equivalente a i -=
        i & (-i), un poco mas rapido
    i = l - 1
    while i:
        res ^= tree[i]
        i &= i - 1
    out_append(ANS[res != 0]) # XOR != 0 ->
    gana quien mueve primero (Frank)
else:
    k = int(data[i2 + 1]); x = int(data[i2 + 2]);
    i2 += 3
    old = arr[k]
    new = old + x
    delta = old ^ new          # "sacar" old y "
    poner" new en un solo update
    arr[k] = new
    i = k
    while i <= n:
        tree[i] ^= delta
        i += i & (-i)

sys.stdout.write("\n".join(out) + "\n")

main()

```

3.4 Segment Tree Iterativo (Actualización Puntual, Mínimo en Rango)

Preguntas guía:

- ¿Debo responder muchas consultas sobre rangos (mínimo, máximo, suma, gcd, xor) de un arreglo que ADEMÁS cambia entre consulta y consulta?
- ¿La operación es asociativa (y en este caso también conmutativa), de modo que el resultado de un rango se puede armar combinando resultados de subrangos?
- ¿Las actualizaciones son puntuales (un elemento a la vez) y no sobre rangos completos?
- ¿ n y q son del orden de 10^5 o más, y en Python necesito evitar recursión y llamadas a función en el bucle principal?

Utilidad: Mantiene un arreglo dinámico permitiendo, en tiempo logarítmico, cambiar un elemento y consultar la combinación de cualquier rango contiguo bajo una operación asociativa. Es la estructura por defecto para consultas de rango con modificaciones: donde una tabla de prefijos o una sparse table dejan de servir porque el arreglo cambia. La versión iterativa (de abajo hacia arriba) guarda el árbol completo en un arreglo de tamaño $2n$ sin punteros ni recursión, lo que la hace mucho más rápida en Python que la versión recursiva clásica.

Complejidad: $\mathcal{O}(n)$ para construir, $\mathcal{O}(\log n)$ por actualización y por consulta; $\mathcal{O}(n)$ de memoria.

Fundamento teórico: Las hojas se guardan en las posiciones $n..2n - 1$ del arreglo t y cada nodo interno i

guarda la combinación de sus hijos $2i$ y $2i + 1$; la raíz es $t[1]$. Actualizar un elemento solo invalida los nodos en el camino desde su hoja hasta la raíz, que son $\lceil \log_2 n \rceil$: se recorre subiendo con $p \gg 1$ y el hermano de un nodo es $p \oplus 1$ (cambia solo el último bit, que distingue hijo izquierdo de derecho). Para consultar el rango semiabierto $[l, r)$ se suben ambos extremos a la vez: si l es un hijo DERECHO (impar), su padre cubriría también elementos a la izquierda del rango, así que se toma $t[l]$ tal cual y se avanza l ; simétricamente, si r es impar, el nodo $r - 1$ es un hijo izquierdo que cae entero dentro del rango y se toma antes de subir. En cada nivel se toman a lo sumo dos nodos, así que la consulta usa $\mathcal{O}(\log n)$ nodos que cubren el rango exactamente, sin solapamiento. Esta disposición funciona para cualquier n , no solo potencias de 2, cuando la operación es CONMUTATIVA como el mínimo: los nodos tomados por el lado izquierdo y el derecho se combinan fuera de orden, y además algunos nodos internos mezclan hojas no contiguas en el arreglo original, lo cual es inofensivo porque ninguna consulta los usa enteros salvo cuando todo su contenido está dentro del rango. Para operaciones no conmutativas (concatenación, multiplicación de matrices) hay que acumular por separado un resultado izquierdo y uno derecho. En la actualización se agrega un corte temprano: si el nuevo mínimo de un nodo coincide con el que ya tenía, sus ancestros tampoco cambian y se puede parar.

Ejercicio aplicado

(CSES 1649 — Dynamic Range Minimum Queries) Dado un arreglo de n enteros, procesar q operaciones que pueden ser cambiar el valor de una posición, o consultar el mínimo en un rango $[a, b]$.

I/O: $n, q \leq 2 \cdot 10^5$ con tres enteros por consulta $\Rightarrow$ lectura total + tokenización con índice `idx` (patrón 2), comparando el tipo de consulta directamente como `bytes` para no convertirlo. Sobre el rendimiento conviene ser explícito, porque aquí la forma de escribir el código importa tanto como el algoritmo: todo el árbol está escrito en línea dentro del bucle, sin funciones `update/query` y sin llamadas a `min`, comparando con `if`. Con $n = q = 2 \cdot 10^5$ esta versión tarda entre $\sim 0.26s$ y $\sim 0.67s$ según la mezcla de operaciones (el peor caso es 100% consultas aleatorias; el update sin corte temprano posible, con valores siempre decrecientes, da $\sim 0.59s$), mientras que la misma lógica escrita con funciones y `min` tarda entre $\sim 0.94s$ y $\sim 1.09s$, en el borde del límite de 1 segundo. Verificado contra el caso de ejemplo y contrastado con una fuerza bruta (actualizar la lista y tomar `min` del slice) en 400 tandas aleatorias con n que no es potencia de 2, $n = 1$ y valores pequeños con muchos empates para ejercitar el corte temprano, sin discrepancias.

Código solución

```

import sys

def solve():
    data = sys.stdin.buffer.read().split()

```

```

n = int(data[0]); q = int(data[1])
# hojas en t[n..2n-1]; nodo i combina a sus hijos 2i
# y 2i+1; raiz en t[1]
t = [0]*n + list(map(int, data[2:2+n]))
for i in range(n - 1, 0, -1):
    a = t[2*i]; b = t[2*i+1]
    t[i] = a if a < b else b    # comparar con if es
    # mas rapido que min()
out = []
idx = 2 + n
for _ in range(q):
    tipo = data[idx]; x = int(data[idx+1]); y = int(
data[idx+2]); idx += 3
    if tipo == "1":
        # ---- update: se sube de la hoja a la raiz
        ----
        p = x - 1 + n
        t[p] = y
        while p > 1:
            a = t[p]; b = t[p ^ 1]    # p ^ 1 =
hermano de p
            m = a if a < b else b
            p >>= 1
            if t[p] == m:              # el padre no
cambia: sus ancestros tampoco
                break
            t[p] = m
        else:
            # ---- query sobre [l, r) subiendo ambos
            # extremos ----
            l = x - 1 + n; r = y + n
            res = 10**18
            while l < r:
                if l & 1:                # l es hijo
derecho: su padre se sale del rango
                    if t[l] < res: res = t[l]
                    l += 1
                if r & 1:                # r-1 es hijo
izquierdo y cae dentro del rango
                    r -= 1
                    if t[r] < res: res = t[r]
                    l >>= 1; r >>= 1
            out.append(res)
sys.stdout.write("\n".join(map(str, out)) + "\n")
solve()

```

3.5 Suma en Rango con Actualización en Rango (Dos Fenwick Trees)

Preguntas guía:

- ¿Debo sumar un valor a TODOS los elementos de un rango, e intercalar consultas de SUMA sobre rangos?
- ¿La operación de consulta es la suma (invertible), y no mínimo o máximo, que no se pueden "deshacer"?
- ¿Con $n, q \sim 10^5$ un segment tree con lazy propagation recursivo sería demasiado lento en Python?
- ¿Puedo expresar la suma de prefijos del arreglo como combinación lineal de dos sumas de prefijos más simples?

Utilidad: Resuelve la combinación "sumar v en $[l, r]$ " y "consultar la suma de $[l, r]$ " en tiempo logarítmico usando solo dos árboles de Fenwick (BIT), sin lazy propagation. Es la alternativa estándar al segment tree con lazy cuando tanto la actualización como la consulta son de suma: el código es más corto, no usa recursión y en Python resulta varias veces más rápido. Para mínimo o máximo

en rango con actualización en rango no aplica, porque depende de que la suma sea invertible; ahí sí hace falta lazy propagation.

Complejidad: $\mathcal{O}(\log n)$ por actualización y por consulta (dos recorridos de BIT en cada una), $\mathcal{O}(n)$ de memoria.

Fundamento teórico: Un BIT guarda en la posición i la suma de un bloque de longitud i & $(-i)$ (el bit menos significativo de i) que termina en i : sumar un prefijo recorre $i -= i \& (-i)$ y actualizar un punto recorre $i += i \& (-i)$, ambos en $\mathcal{O}(\log n)$ pasos. Por sí solo, un BIT hace actualización PUNTUAL y consulta de PREFIJO. El truco para rangos parte del arreglo de diferencias d : sumar v a $[l, r]$ equivale a $d_l += v$ y $d_{r+1} -= v$, y el valor de cada elemento es $a_i = \sum_{j \leq i} d_j$. La suma de prefijos que se quiere consultar es entonces una doble suma que se puede reordenar:

$$\begin{aligned}
 S(x) &= \sum_{i=1}^x a_i = \sum_{i=1}^x \sum_{j=1}^i d_j = \sum_{j=1}^x d_j (x - j + 1) \\
 &= x \sum_{j=1}^x d_j - \sum_{j=1}^x d_j (j - 1).
 \end{aligned}$$

Ambas sumas son sumas de prefijo de arreglos que solo cambian en DOS puntos por actualización: el primer BIT guarda d_j y el segundo guarda $d_j (j - 1)$. Así, una actualización de rango son dos actualizaciones puntuales en cada BIT (en l con v y $v(l - 1)$, en $r + 1$ con $-v$ y $-vr$), y la suma de $[l, r]$ es $S(r) - S(l - 1)$, con cada S calculado recorriendo ambos BIT a la vez. El factor x se aplica al final, fuera del recorrido, porque multiplica a toda la suma del primer BIT. [1]

Ejercicio aplicado

(SPOJ HORRIBLE — Horrible Queries) Dados T casos, cada uno con un arreglo de N ceros y C comandos que pueden ser sumar v a todas las posiciones de $[p, q]$ o consultar la suma de $[p, q]$, responder todas las consultas.

I/O: $N, C \leq 10^5$ por caso, con comandos de 3 o 4 números según el tipo $\Rightarrow$ lectura total + tokenización con índice `idx` (patrón 2), leyendo primero el tipo del comando para saber cuántos tokens consumir. El enunciado no garantiza $p \leq q$, así que se ordenan en cada comando. Las sumas pueden llegar a $\sim 10^{17}$, sin problema para los enteros de Python. Verificado contra el caso de ejemplo y contrastado con una fuerza bruta (sumar y consultar sobre la lista directamente) en 250 tandas con varios casos cada una e índices con $p > q$, sin discrepancias. Con $N = C = 10^5$ tarda $\sim 0.2s$ a $\sim 0.46s$ por caso según la mezcla de comandos, mientras que un segment tree con lazy propagation recursivo, verificado igual de correcto, tarda $\sim 0.75s$ a $\sim 2.86s$: unas 6 veces más lento. Como T no está acotado, con varios casos máximos el tiempo crece linealmente ($\sim 2.4s$ con $T = 5$).

Código solución

```

import sys

def solve():

```

```

data = sys.stdin.buffer.read().split()
idx = 0
t = int(data[idx]); idx += 1
out = []
for _ in range(t):
    n = int(data[idx]); c = int(data[idx+1]); idx += 2
    b1 = [0]*(n + 1)      # BIT de d[j]
    b2 = [0]*(n + 1)      # BIT de d[j]*(j-1)
    for _ in range(c):
        if data[idx] == b"0":
            p = int(data[idx+1]); q = int(data[idx+2]); v = int(data[idx+3]); idx += 4
            if p > q: p, q = q, p          # el
            enunciado no garantiza p <= q
            # d[p] += v y d[q+1] -= v, reflejado en
            ambos BIT
            i = p; w = v*(p - 1)
            while i <= n:
                b1[i] += v; b2[i] += w
                i += i & -1
            i = q + 1; w = v*q
            while i <= n:
                b1[i] -= v; b2[i] -= w
                i += i & -1
        else:
            p = int(data[idx+1]); q = int(data[idx+2]); idx += 3
            if p > q: p, q = q, p
            # S(x) = x * sum(d[1..x]) - sum(d[j]*(j-1))
            -1)) -> S(q) - S(p-1)
            s1 = s2 = 0; i = q
            while i > 0:
                s1 += b1[i]; s2 += b2[i]
                i -= i & -1
            res = s1*q - s2
            s1 = s2 = 0; i = p - 1
            while i > 0:
                s1 += b1[i]; s2 += b2[i]
                i -= i & -1
            res -= s1*(p - 1) - s2
            out.append(res)
sys.stdout.write("\n".join(map(str, out)) + "\n")
solve()

```

3.6 Sparse Table (Mínimo en Rango Estático, Consulta Constante)

Preguntas guía:

- ¿Debo responder muchas consultas de mínimo (o máximo, o gcd) sobre rangos de un arreglo que NUNCA cambia?
- ¿La operación es IDEMPOTENTE, es decir, combinar un elemento consigo mismo no altera el resultado ($\min(x, x) = x$), de modo que puedo cubrir el rango con dos bloques que se solapan?
- ¿El número de consultas es grande y quiero que cada una cueste $\mathcal{O}(1)$ en vez de $\mathcal{O}(\log n)$?
- ¿Me alcanza la memoria para $\mathcal{O}(n \log n)$ valores?

Utilidad: Precalcula el mínimo de todos los bloques cuya longitud es potencia de 2, y con eso responde el mínimo de cualquier rango en tiempo constante combinando solo dos bloques. Es la estructura ideal para consultas de rango sobre arreglos estáticos con operaciones idempotentes (mínimo, máximo, gcd, AND, OR). También es la pieza que convierte el LCA en un árbol, y el LCP

entre sufijos arbitrarios de un suffix array, en consultas $\mathcal{O}(1)$. Si el arreglo cambia, hay que usar un segment tree; si la operación no es idempotente (suma), la sparse table todavía funciona pero la consulta pasa a $\mathcal{O}(\log n)$, y conviene más una tabla de prefijos.

Complejidad: $\mathcal{O}(n \log n)$ en tiempo y memoria para construir, $\mathcal{O}(1)$ por consulta.

Fundamento teórico: La tabla guarda en $st[j][i]$ el mínimo del bloque de longitud 2^j que empieza en i . El nivel 0 es el propio arreglo, y cada nivel se obtiene del anterior en tiempo lineal porque un bloque de largo 2^j es la unión de dos bloques consecutivos de largo 2^{j-1} : $st[j][i] = \min(st[j-1][i], st[j-1][i+2^{j-1}])$. Hay $\lfloor \log_2 n \rfloor + 1$ niveles y el nivel j tiene $n - 2^j + 1$ entradas. Para consultar un rango $[a, b]$ de longitud $L = b - a$, se toma $k = \lfloor \log_2 L \rfloor$, el mayor exponente con $2^k \leq L$, y se usan los dos bloques de largo 2^k que empiezan en a y que terminan en b : el primero es $[a, a + 2^k]$ y el segundo $[b - 2^k, b]$. Como $2^k \leq L < 2^{k+1}$, cada bloque cabe dentro del rango y juntos lo cubren entero, aunque se solapan en el medio. Ese solapamiento es inofensivo SOLO porque el mínimo es idempotente: un elemento contado dos veces no cambia el mínimo, mientras que en una suma se contaría doble. El exponente se obtiene como $L.bit_length() - 1$, que es la posición del bit más alto de L y corre en C , sin necesidad de precalcular una tabla de logaritmos. [4]

Ejercicio aplicado

(CSES 1647 — Static Range Minimum Queries) Dado un arreglo estático de n enteros, responder q consultas del mínimo en el rango $[a, b]$.

I/O: $n, q \leq 2 \cdot 10^5$ con dos enteros por consulta $\Rightarrow$ lectura total + tokenización con índice `idx` (patrón 2). Cada nivel de la tabla se construye con una comprensión sobre `zip` de dos slices del nivel anterior, comparando con `if`; una alternativa más compacta, `list(map(min, A, B))`, se midió correcta pero entre un 40% y un 60% más lenta, porque `min` es una función genérica pensada para iterables y su llamada con dos argumentos cuesta más que una comparación. Verificado contra el caso de ejemplo y contrastado con una fuerza bruta (`min` sobre el slice) en 400 tandas aleatorias que incluyen $n = 1$, n igual a potencias de 2 y sus vecinos, rangos de un solo elemento y el rango completo, sin discrepancias. Con $n = q = 2 \cdot 10^5$ tarda entre $\sim 0.29s$ y $\sim 0.36s$ según el patrón de consultas.

Código solución

```

import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); q = int(data[1])
    fila = list(map(int, data[2:2+n]))
    # st[j][i] = minimo del bloque de largo 2^j que
    # empieza en i
    st = [fila]
    j = 1
    while (1 << j) <= n:
        mitad = 1 << (j - 1)
        prev = st[-1]

```

```

# bloque de 2^j = dos bloques consecutivos de 2^(j-1)
st.append([x if x < y else y
          for x, y in zip(prev[:n - (1 << j) + 1], prev[mitad:])])
j += 1
out = []
idx = 2 + n
for _ in range(q):
    a = int(data[idx]) - 1; b = int(data[idx+1]); idx += 2
    # rango [a, b)
    k = (b - a).bit_length() - 1 # mayor k con 2^k <= largo del rango
    t = st[k]
    # dos bloques de 2^k que cubren [a, b)
    # solapandose: sirve porque min es idempotente
    x = t[a]; y = t[b - (1 << k)]
    out.append(x if x < y else y)
sys.stdout.write("\n".join(map(str, out)) + "\n")

```

solve()

3.7 Rectángulo Máximo en Histograma (Pila Monótona)

Preguntas guía:

- ¿Me piden el rectángulo de mayor área bajo un histograma, o el mayor valor de (mínimo de un subarreglo) $\times$ (su longitud)?
- ¿Para cada elemento necesito saber hasta dónde se puede extender a izquierda y derecha sin encontrar uno más bajo (el "next smaller element" a ambos lados)?
- ¿Probar todos los subarreglos sería $\mathcal{O}(n^2)$ y n llega a 10^5 o más?
- ¿El problema se puede reformular como "cada elemento es el mínimo de un intervalo maximal", de modo que basta evaluar n candidatos?

Utilidad: Una pila monótona (que mantiene sus elementos ordenados crecientemente desde el fondo) encuentra en tiempo lineal, para cada posición, el elemento menor más cercano a su izquierda y a su derecha. Con eso se resuelven todos los problemas de "intervalo maximal donde este elemento es el mínimo (o máximo)": rectángulo máximo en histograma, submatriz de unos más grande (aplicándolo fila por fila), suma de los mínimos de todos los subarreglos, y los clásicos "next greater element" y "stock span".

Complejidad: $\mathcal{O}(n)$: cada índice entra y sale de la pila exactamente una vez.

Fundamento teórico: En un rectángulo óptimo, la altura está limitada por la tabla más baja que cubre; llamemos i a esa tabla. Entonces el rectángulo óptimo es, para algún i , el de altura h_i extendido lo más posible a ambos lados, hasta justo antes de la primera tabla estrictamente más baja por la izquierda y por la derecha. Así basta evaluar n candidatos, uno por tabla. La pila guarda índices con alturas crecientes desde el fondo. Cuando llega la tabla i y la cima de la pila tiene altura $\geq h_i$, esa cima ya no puede extenderse más a la derecha: i es su límite derecho. Y su límite izquierdo es el elemento que queda debajo en

la pila, porque todo lo que estaba entre ambos fue sacado antes por ser más alto, así que el de debajo es el primero más bajo hacia la izquierda. Por eso, al sacar el índice con altura h , su rectángulo es $h \times (i - \text{debajo} - 1)$, y ambos límites se conocen en el momento de sacarlo: no hacen falta dos pasadas. Dos centinelas simplifican los bordes: un -1 en el fondo de la pila hace de límite izquierdo de quien quede más abajo, y una altura 0 agregada al final saca todo lo que quedó en la pila, porque ninguna altura real es menor. Con alturas repetidas, sacar también las iguales ($\geq$) es correcto: la primera de un grupo de iguales calcula un ancho menor, pero la última del grupo se extiende sobre todas y obtiene el área correcta. Cada índice entra y sale una sola vez, así que el total es lineal aunque un solo paso del `while` pueda sacar muchos.

Ejercicio aplicado

(CSES 1142 — Advertisement) Dada una cerca de n tablas de ancho 1 y alturas k_i , hallar el área máxima de un anuncio rectangular que se pueda pegar sobre ella.

I/O: $n \leq 2 \cdot 10^5$ alturas hasta $10^9 \Rightarrow$ lectura total + tokenización (patrón 2). El área puede llegar a $2 \cdot 10^{14}$, sin problema para los enteros de Python. Verificado contra el caso de ejemplo y contrastado con una fuerza bruta $\mathcal{O}(n^2)$ (mínimo acumulado desde cada inicio) en 800 casos con alturas crecientes, decrecientes, todas iguales y aleatorias, sin discrepancias. Con $n = 2 \cdot 10^5$ tarda ~ 0.06 s a ~ 0.08 s en todos los perfiles medidos: aleatorio, creciente (la pila crece hasta n), decreciente (cada tabla saca a la anterior), todas iguales y dientes de sierra.

Código solución

```

import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    h = list(map(int, data[1:1+n]))
    h.append(0) # centinela: altura 0
    # saca todo al final
    pila = [-1] # centinela: limite izquierdo por defecto
    mejor = 0
    for i in range(n + 1):
        hi = h[i]
        # la cima ya no puede extenderse a la derecha: i es su limite derecho
        # y el que queda debajo en la pila es su limite izquierdo
        while len(pila) > 1 and h[pila[-1]] >= hi:
            alto = h[pila.pop()]
            area = alto * (i - pila[-1] - 1)
            if area > mejor: mejor = area
        pila.append(i)
    print(mejor)

solve()

```

3.8 Sumas de Prefijos 2D (y Arreglo de Diferencias 2D)

Preguntas guía:

- ¿Debo responder muchas consultas de suma (o conteo) sobre rectángulos de una matriz que NO cambia?
- ¿Recorrer cada rectángulo en cada consulta sería $\mathcal{O}(n^2)$ por consulta y hay cientos de miles de consultas?
- ¿Al revés: me piden SUMAR un valor a muchos rectángulos y solo al final leer la matriz resultante (arreglo de diferencias 2D)?
- ¿La matriz cabe en memoria, del orden de 10^6 celdas?

Utilidad: La tabla de prefijos 2D precalcula en $\mathcal{O}(n^2)$ la suma de cada rectángulo que parte de la esquina superior izquierda, y con eso responde la suma de CUALQUIER rectángulo en $\mathcal{O}(1)$ combinando cuatro valores. Es la versión bidimensional de la suma de prefijos: sirve para contar celdas marcadas, sumar valores en regiones, y como base de técnicas como buscar la submatriz de suma máxima o verificar si un rectángulo está vacío. Su operación inversa, el arreglo de diferencias 2D, resuelve el problema dual: aplicar muchas sumas sobre rectángulos en $\mathcal{O}(1)$ cada una y reconstruir la matriz completa al final.

Complejidad: $\mathcal{O}(n^2)$ para construir la tabla, $\mathcal{O}(1)$ por consulta. Para el arreglo de diferencias, $\mathcal{O}(1)$ por actualización y $\mathcal{O}(n^2)$ para reconstruir.

Fundamento teórico: Sea $P[i][j]$ la suma del rectángulo con esquinas $(1, 1)$ y (i, j) , con $P[0][\cdot] = P[\cdot][0] = 0$. La suma del rectángulo $[y_1..y_2] \times [x_1..x_2]$ se obtiene por inclusión-exclusión: $P[y_2][x_2]$ incluye todo lo que está arriba y a la izquierda de (y_2, x_2) ; se le resta la franja superior $P[y_1 - 1][x_2]$ y la franja izquierda $P[y_2][x_1 - 1]$, pero la esquina $P[y_1 - 1][x_1 - 1]$ quedó restada dos veces, así que se suma una vez. La construcción usa que la tabla de la fila i es la de la fila $i - 1$ más las sumas de prefijo de la fila i sola: $P[i][j] = P[i - 1][j] + \sum_{t \leq j} g[i][t]$. Escrito así, cada fila se arma con dos operaciones que Python ejecuta en C: `accumulate` sobre la fila y `map(add, ...)` con la fila anterior, sin un bucle interpretado por celda; convertir cada fila de texto a ceros y unos con `bytes.translate` también ocurre en C. El arreglo de diferencias 2D es la operación inversa: si D es tal que su tabla de prefijos es la matriz buscada, sumar v a un rectángulo equivale a modificar solo sus cuatro esquinas de D ($+v$ en (y_1, x_1) , $-v$ en $(y_1, x_2 + 1)$ y $(y_2 + 1, x_1)$, $+v$ en $(y_2 + 1, x_2 + 1)$), porque al acumular, el $+v$ se propaga hacia abajo y a la derecha, y los tres ajustes lo cancelan exactamente fuera del rectángulo. Al final, una sola pasada de prefijos 2D sobre D reconstruye la matriz.

Ejercicio aplicado

(CSES 1652 — Forest Queries) Dado un bosque de $n \times n$ celdas (árbol * o vacío .), responder q consultas de cuántos árboles hay dentro de un rectángulo dado.

I/O: $n \leq 1000$ filas de texto sin espacios y $q \leq 2 \cdot 10^5$ consultas de cuatro enteros $\Rightarrow$ lectura total + tokenización (patrón 2): cada fila del bosque es un token y se procesa como `bytes`. Verificado contra el caso de ejemplo y contrastado con una fuerza bruta (contar celda por celda) en 400 tandas con bosques vacíos, llenos, densidades

intermedias y $n = 1$, incluyendo el rectángulo completo y rectángulos de una celda, sin discrepancias. Con $n = 1000$ y $q = 2 \cdot 10^5$ tarda ~ 0.30 s. El arreglo de diferencias de los extras se verificó por separado contra fuerza bruta en 400 tandas (incluyendo valores negativos), y aplicar $2 \cdot 10^5$ sumas de rectángulo sobre una matriz de 1000×1000 y reconstruirla tarda ~ 0.27 s.

Código solución

```
import sys
from itertools import accumulate
from operator import add

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); q = int(data[1])
    tabla = bytes.maketrans(b" .", b"\x00\x01") # ' .'
    # -> 0, '*' -> 1
    # P[i][j] = arboles en el rectangulo (1,1)-(i,j);
    #      fila y columna 0 en cero
    P = [[0]*(n + 1)]
    for i in range(n):
        fila = data[2 + i].translate(tabla) #
        # conversion en C
        # fila de P = fila anterior de P + prefijos de
        # esta fila del bosque
        P.append(list(map(add, P[-1], accumulate(fila,
            initial=0))))
    out = []
    idx = 2 + n
    for _ in range(q):
        y1 = int(data[idx]); x1 = int(data[idx+1]); y2 =
        int(data[idx+2]); x2 = int(data[idx+3]); idx += 4
        A = P[y2]; B = P[y1 - 1]
        # inclusion-exclusion: la esquina se resto dos
        # veces, se suma una
        out.append(A[x2] - B[x2] - A[x1 - 1] + B[x1 - 1])
    sys.stdout.write("\n".join(map(str, out)) + "\n")

solve()
```

Extras: arreglo de diferencias 2D — para el problema inverso: aplicar muchas sumas de un valor sobre rectángulos y leer la matriz al final. Cada actualización toca solo cuatro celdas, y la reconstrucción es la misma construcción de prefijos 2D de arriba.

```
from itertools import accumulate
from operator import add

def aplicar(n, updates):
    # updates: lista de (y1, x1, y2, x2, v), 1-indexado e
    # inclusivo
    D = [[0]*(n + 2) for _ in range(n + 2)]
    for y1, x1, y2, x2, v in updates:
        D[y1][x1] += v # empieza a sumar
        # desde la esquina
        D[y1][x2 + 1] -= v # corta a la derecha
        # del rectangulo
        D[y2 + 1][x1] -= v # corta debajo del
        # rectangulo
        D[y2 + 1][x2 + 1] += v # la esquina de abajo
        # a la derecha se corto dos veces
    M = []
    prev = [0]*(n + 2)
    for i in range(1, n + 1):
        prev = list(map(add, prev, accumulate(D[i]))) #
        # prefijos 2D sobre D
        M.append(prev[1:n + 1])
    return M # M[y-1][x-1] = valor
```

3.9 Ordered Set con Fenwick Tree (Compresión + K-ésimo por Descenso Binario)

Preguntas guía:

- ¿Debo mantener un conjunto dinámico con inserciones y borrados, y consultar el k -ésimo menor o cuántos elementos son menores que x ?
- ¿Conozco TODAS las operaciones de antemano (procesamiento offline), de modo que puedo comprimir los valores posibles a índices $1..m$?
- ¿Los valores son enormes (hasta 10^9) pero la cantidad de valores DISTINTOS está acotada por el número de operaciones?
- ¿En Python no tengo un `ordered_set` como el de C++ y necesito algo $\mathcal{O}(\log n)$ por operación?

Utilidad: Reemplaza en Python al `ordered_set` de C++ (`find_by_order` y `order_of_key`) cuando las operaciones se pueden leer todas antes de procesarlas. Un Fenwick tree sobre los valores comprimidos guarda un 1 en cada valor presente; así, contar menores que x es una suma de prefijo, e insertar o borrar es sumar ± 1 en un punto. El k -ésimo menor se obtiene con un descenso binario sobre el propio Fenwick, sin búsqueda binaria externa. Sirve también para mediana dinámica, contar inversiones, y cualquier problema de "rango de un elemento" en un multiconjunto que cambia (guardando conteos en vez de 0/1).

Complejidad: $\mathcal{O}(Q \log Q)$ para comprimir, $\mathcal{O}(\log m)$ por operación, con $m \leq Q$ la cantidad de valores distintos.

Fundamento teórico: Como solo importan las comparaciones entre valores, se reemplaza cada valor por su posición en la lista ordenada de valores distintos que aparecen en inserciones y borrados; esto preserva el orden y reduce el universo de $2 \cdot 10^9$ a a lo sumo Q . El arreglo de presencia $e[1..m]$ (1 si el valor está en el conjunto) se mantiene en un Fenwick, cuya suma de prefijo hasta p es exactamente la cantidad de elementos presentes con índice $\leq p$. Para `COUNT(x)`, el valor x puede no haber sido comprimido; `bisect_left` sobre la lista ordenada da cuántos valores distintos son estrictamente menores que x , y la suma de prefijo hasta ese índice cuenta los presentes. Para el k -ésimo se busca la menor posición p con `prefijo(p)  $\geq k$` , sin repetir sumas de prefijo: el Fenwick guarda en la posición i la suma del bloque $(i - \text{lowbit}(i), i]$, y si p es múltiplo de 2^{j+1} , la posición $p + 2^j$ guarda justamente la suma del bloque $(p, p + 2^j]$. Por lo tanto se puede construir p bit a bit, de la potencia de 2 más alta hacia abajo: si el bloque siguiente contiene menos de lo que falta (`bit[p + 2j] < resto`), se lo salta entero, se avanza p y se descuenta; si no, el k -ésimo está dentro y se prueba un paso más chico. Al terminar, p es la última posición con `prefijo < k`, así que la respuesta es la posición $p + 1$, es decir

`vals[p]` en la lista 0-indexada. Las inserciones repetidas y los borrados de valores ausentes se ignoran consultando un arreglo auxiliar de presencia, que además evita que el Fenwick guarde valores distintos de 0 o 1. [1]

Ejercicio aplicado

(SPOJ ORDERSET — Order statistic set) Procesar $Q \leq 2 \cdot 10^5$ operaciones sobre un conjunto de enteros: insertar x (si no está), borrar x (si está), imprimir el k -ésimo menor (o `invalid` si hay menos de k elementos), e imprimir cuántos elementos son menores que x .

I/O: Q pares (letra, entero) $\Rightarrow$ lectura total + tokenización (patrón 2), separando letras y parámetros con slices de paso 2 (`data[1::2]` y `data[2::2]`), que corren en C. La lectura completa por adelantado es la que permite la compresión offline. Verificado contra el caso de ejemplo y contrastado con una lista ordenada mantenida con `bisect` en 500 tandas aleatorias, incluyendo inserciones repetidas, borrados de valores ausentes, `COUNT` con valores que nunca se insertaron, k mayores que el tamaño del conjunto y rangos de valores pequeños y grandes, sin discrepancias. Con $Q = 2 \cdot 10^5$ tarda entre $\sim 0.25s$ y $\sim 0.35s$ según la mezcla de operaciones, frente a un límite de 1.5 a 2 segundos.

Código solución

```
import sys
from bisect import bisect_left

def solve():
    data = sys.stdin.buffer.read().split()
    q = int(data[0])
    ops = data[1:1 + 2*q:2]  # letras I, D, K, C
    args = list(map(int, data[2:2 + 2*q:2]))  # parametros
    # compresion offline: solo los valores que se insertan o borran
    vals = sorted({args[i] for i in range(q) if ops[i] in ('I', 'D')})
    m = len(vals)
    pos = {v: i + 1 for i, v in enumerate(vals)}  # valor -> indice 1..m
    bit = [0]*(m + 1)
    esta = bytearray(m + 1)  # presencia, para ignorar repetidos
    total = 0
    paso0 = 1  # mayor potencia de 2 <= m
    while paso0*2 <= m:
        paso0 *= 2
    out = []
    for i in range(q):
        op = ops[i]; x = args[i]
        if op == 'I':
            p = pos[x]
            if not esta[p]:
                esta[p] = 1; total += 1
                while p <= m:
                    bit[p] += 1; p += p & -p
            elif op == 'D':
                p = pos[x]
                if esta[p]:
                    esta[p] = 0; total -= 1
                    while p <= m:
                        bit[p] -= 1; p += p & -p
            elif op == 'K':
                if x > total:
```

```

        out.append("invalid")
    else:
        # descenso binario: salta bloques enteros
        con menos de 'resto'
        p = 0; paso = paso0; resto = x
        while paso:
            nxt = p + paso
            if nxt <= m and bit[nxt] < resto:
                p = nxt; resto -= bit[nxt]
                paso >>= 1
            out.append(str(vals[p])) # posicion p
        +1 (1-idx) = vals[p]
    else:
        # cuantos valores distintos < x, y de esos
        cuantos estan presentes
        p = bisect_left(vals, x)
        s = 0
        while p > 0:
            s += bit[p]; p -= p & -p
        out.append(str(s))
    sys.stdout.write("\n".join(out) + "\n")

solve()

```

3.10 Balance de Paréntesis por Bloques de Nivel Superior (parejas incompatibles)

Preguntas guía:

- ¿Trabajo con una secuencia balanceada de paréntesis y necesito razonar sobre qué emparejamientos específicos son válidos?
- ¿La pregunta involucra qué pasa con el RESTO de la secuencia al fijar/quitar un par específico?
- ¿Puedo identificar los "bloques de nivel superior" (tramos donde el balance nunca vuelve a cero hasta el final del bloque)?

Utilidad: Analizar, para una secuencia balanceada de paréntesis, la validez de emparejamientos NO necesariamente anidados de la forma estándar, descomponiendo la secuencia en bloques de nivel superior (tramos maximales donde el balance se mantiene estrictamente positivo hasta el cierre). La clave es que un '(' y un ')' se pueden emparejar sin romper la validez del resto si y solo si están en el MISMO bloque.

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico: Sea $bal(p)$ el balance (número de '(' menos número de ')') tras procesar los primeros p caracteres. Un bloque de nivel superior termina exactamente donde $bal(p) = 0$. Al emparejar una posición i '(' con una posición j ')', $i < j$, el balance de las posiciones ENTRE i y $j - 1$ baja en 1 respecto al original (porque ya se "usó" el '(' pero no el ')'); esto rompe la validez del resto si y solo si el balance ORIGINAL en algún punto de ese rango ya era 0 (pues bajaría a -1). Como los únicos puntos con balance 0 son los cierres de bloque, el par es INCOMPATIBLE si y solo si i y j pertenecen a bloques distintos. El total de parejas incompatibles se calcula entonces como (total de parejas (i, j) válidas en toda la secuencia) menos (parejas válidas contando solo dentro de cada bloque por separado). Ambas cantidades

se obtienen en un único recorrido llevando un contador de '(' abiertos, global y reiniciado por bloque.

Ejercicio aplicado

Dada una secuencia de N bailarines Yang '(' y Yin ') (garantizada balanceada), donde cada Yang debe emparejarse con un Yin posterior, contar cuántas parejas (i, j) son "incompatibles": aquellas que, de formarse, impiden que el resto de bailarines encuentre pareja.

I/O: $N \leq 10^6 \rightarrow$ un solo recorrido lineal con lectura total (patrón 2); no hace falta tokenizar por espacios ya que la secuencia es un único string contiguo. Verificado: $\sim 0.1s$ en el peor caso.

Código solución

```

import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    s = data[1].decode()

    total_pairs = 0 # total de parejas (i,j) con i<j,
    s[i]='(', s[j]=')'
    within_block = 0 # de esas, cuantas quedan DENTRO
    del mismo bloque de nivel superior
    open_global = 0
    open_block = 0
    bal = 0

    for ch in s:
        if ch == '(':
            open_global += 1
            open_block += 1
            bal += 1
        else:
            total_pairs += open_global
            within_block += open_block
            bal -= 1
            if bal == 0: # cierre de un bloque
                de nivel superior y reinicia el contador de bloque
                open_block = 0

    print(total_pairs - within_block)

solve()

```

3.11 Pila con Restricción de Capacidad (matrioshkas)

Preguntas guía:

- ¿La entrada representa una estructura jerárquica donde los elementos se abren y posteriormente deben cerrarse respetando el orden (LIFO)?
- ¿Cada elemento abierto posee una restricción que depende de todos los elementos contenidos directamente en su interior?
- ¿Necesito validar simultáneamente el correcto anidamiento y una condición adicional sobre el contenido de cada nivel?

Utilidad: Validar estructuras anidadas donde cada objeto debe cerrarse exactamente en el orden inverso en que fue abierto y, además, debe cumplir una restricción sobre los elementos contenidos directamente en su interior. Este patrón aparece frecuentemente en problemas de parsing,

validación de expresiones jerárquicas y simulación de estructuras recursivas.

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico: Cada juguete comienza cuando aparece un entero negativo $-x$ y termina cuando aparece el correspondiente entero positivo x . Debido a que un juguete debe cerrarse antes de continuar cerrando el que lo contiene, el orden de cierre siempre sigue la política **Last In, First Out (LIFO)**, por lo que una pila modela naturalmente la estructura.

Ejercicio aplicado

(UVA 11111 — Generalized Matrioshkas) Dada una secuencia de enteros que representa la apertura y cierre de juguetes generalizados, determinar si la estructura corresponde a una matrioshka válida.

Cada juguete de tamaño m puede contener varios juguetes, pero la suma de los tamaños de todos sus hijos directos debe ser estrictamente menor que m .

I/O: Cada línea representa un caso de prueba independiente. Los valores aparecen separados por espacios y pueden alcanzar magnitudes de hasta 10^7 . Como únicamente se realiza un recorrido secuencial sobre cada línea utilizando una pila, el algoritmo procesa cada caso en tiempo lineal.

Código solución

```
import sys

for line in sys.stdin:
    nums = list(map(int, line.split()))
    stack = []
    valid = True

    for x in nums:
        if x < 0:
            stack.append([-x, 0])
        else:
            if not stack:
                valid = False
                break
            size, total = stack.pop() # numero del
            # cierre y cuantos tiene adentro
            if size != x:
                valid = False
                break
            if total >= size:
                valid = False
                break
            if stack:
                stack[-1][1] += size # del ultimo en
                # en la cola le suma numero

    if stack:
        valid = False
    if valid:
        print(":-) Matrioshka!")
    else:
        print(":-( Try again.")
```

3.12 Reducción con Pila (ciclos en un bang path)

Preguntas guía:

- ¿Debo procesar una secuencia donde ciertos elementos pueden cancelarse al aparecer nuevamente?
- ¿Eliminar un ciclo puede generar inmediatamente otro ciclo más grande?
- ¿Necesito conservar el orden original mientras simplifico la secuencia al máximo?

Utilidad: Simplificar recorridos o secuencias eliminando ciclos innecesarios. Cada vez que un elemento vuelve a aparecer, todo el recorrido intermedio deja de ser útil y puede eliminarse. Después de cada eliminación es necesario volver a comprobar si ahora se formó un nuevo ciclo con los elementos restantes. Este patrón aparece en problemas de rutas, navegación, directorios, historial de movimientos y simplificación de secuencias.

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico: La solución utiliza una pila que representa la ruta simplificada construida hasta el momento; cada máquina se procesa exactamente una vez y se inserta al final de la pila. Después de insertar una máquina pueden ocurrir dos situaciones. La primera es si las dos últimas máquinas tienen el mismo nombre donde la última visita es redundante y puede eliminarse. La segunda donde la última máquina coincide con la antepenúltima, por lo que tanto la máquina intermedia como la repetida pueden eliminarse.

Ejercicio aplicado

Dado un bang path de UUCP (UNIX-to-UNIX Copy Program), eliminar todos los saltos innecesarios producidos por ciclos entre máquinas, conservando exactamente el mismo destino final y respetando las mayúsculas y minúsculas de las máquinas que permanecen.

Las comparaciones entre nombres de máquinas son **case-insensitive**, aunque la salida debe conservar la capitalización original.

I/O: La entrada consiste en una única línea cuya longitud no supera los 256 caracteres. Cada componente está separado por el carácter '!' y el último elemento siempre corresponde al usuario destino, por lo que nunca participa en las simplificaciones. Debido al pequeño tamaño de la entrada, basta dividir el string utilizando `split("!!")` y recorrer los componentes una única vez.

Código solución

```
import sys
input = sys.stdin.buffer.readline
data = input().decode("utf-8").split("!!") # Separa cada
# componente del bang path

if len(data) <= 2: # Si no existen maquinas intermedias
    # no hay nada que simplificar
    sys.stdout.write("!!".join(data))
    exit()

machines = data[:-1] # todas las maquinas
user = data[-1] # usuario final (no se modifica)
stack = []

for machine in machines:
    stack.append(machine) # agrega la maquina actual
    # a la ruta
```

```

while True:      # mientras aparezcan nuevos ciclos
    despues de eliminar otros
    if (len(stack) >= 2 and stack[-1].lower() ==
        stack[-2].lower()): #CASO AA
        stack.pop()
        continue

    if (len(stack) >= 3 and stack[-1].lower() ==
        stack[-3].lower()): #CASO ABA
        stack.pop()      # elimina la ultima A
        stack.pop()      # elimina B
        continue

    break # ya no existen reducciones posibles
sys.stdout.write("!".join(stack + [user])) # reconstruye
el bang path conservando el usuario final

```

4 Algoritmos de Grafos

4.1 Emparejamiento Bipartito Máximo, Kuhn (parejas de baile)

Preguntas guía:

- ¿Tengo dos conjuntos disjuntos (personas / tareas) y aristas que solo conectan un conjunto con el otro?
- ¿Me piden el número máximo de "parejas" o asignaciones simultáneas sin repetir elementos de ningún lado?
- ¿El tamaño de ambos lados permite $\mathcal{O}(V \cdot E)$ (hasta unos pocos miles)?

Utilidad: Encontrar el máximo número de aristas seleccionables en un grafo bipartito sin que dos aristas compartan vértice (asignación óptima uno a uno). Ejemplos: asignar trabajadores a tareas, parejas de baile, cobertura de tareas.

Complejidad: $\mathcal{O}(V \cdot E)$ con la versión simple de caminos aumentantes, donde V es el lado izquierdo y E el número de aristas.

Fundamento teórico: Se buscan repetidamente "caminos aumentantes": para cada vértice izquierdo se intenta, vía DFS, encontrar una pareja libre o "desalojar" a otro vértice ya emparejado para liberar espacio y aumentar el emparejamiento en 1. Es un caso particular del teorema de König / flujo máximo en redes. [5], [6]

Ejercicio aplicado

Hay n chicos y m chicas y k parejas de baile posibles. Hallar el número máximo de parejas simultáneas y mostrar una asignación válida.

I/O: $n, m \leq 500$ y $k \leq 1000 \rightarrow$ entrada pequeña, se usa readline por línea (patrón 1), suficiente y más simple de leer.

Código solución

```
import sys
input = sys.stdin.buffer.readline

def try_kuhn(u, adj, matched, visited):
    # intenta emparejar el vertice u del lado izquierdo
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
            # v esta libre, o su pareja actual puede
            reubicarse...
            if matched[v] == -1 or try_kuhn(matched[v],
            adj, matched, visited):
                matched[v] = u
                return True
    return False

def solve():
    n, m, k = map(int, input().split())
    adj = [[] for _ in range(n)]
    for _ in range(k):
        a, b = map(int, input().split())
        adj[a - 1].append(b - 1)

    matched = [-1] * m                # pareja actual
    # de cada vertice derecho
    total = 0
    for u in range(n):
        visited = [False] * m        # se reinicia en
        # cada intento
        if try_kuhn(u, adj, matched, visited):
            total += 1
```

```
def try_kuhn(u, adj, matched, visited):
    total += 1

    out = [str(total)]
    for v in range(m):
        if matched[v] != -1:
            out.append(f"{matched[v] + 1} {v + 1}")
    sys.stdout.write("\n".join(out))

solve()
```

4.2 Kuhn sobre Segmentos de Fila/Columna (torres con peones)

Preguntas guía:

- ¿La cuadrícula tiene obstáculos que "cortan" el alcance de piezas o rangos dentro de cada fila y columna?
- ¿Cada fila y cada columna se puede dividir en segmentos independientes (separados por obstáculos) donde a lo más un elemento puede colocarse por segmento?
- ¿Piden el máximo número de elementos colocables sin conflicto entre sí?

Utilidad: Resolver problemas de colocación de piezas sobre una cuadrícula con obstáculos, donde el conflicto es "compartir fila o columna sin un obstáculo de por medio". En vez de razonar celda por celda, se etiquetan los segmentos libres de cada fila y columna, y el problema se convierte en un emparejamiento bipartito clásico.

Complejidad: $\mathcal{O}(N^2)$ para etiquetar segmentos, más $\mathcal{O}(V \cdot E)$ para el emparejamiento (V, E acotados por N^2).

Fundamento teórico: Cada casilla vacía pertenece a exactamente un segmento de fila (corrida maximal de casillas vacías, delimitada por obstáculos o bordes) y a exactamente un segmento de columna. Como a lo más una torre puede ocupar cada segmento (todas sus casillas se atacan entre sí sin obstáculo de por medio), colocar torres sin conflicto equivale a elegir aristas entre segmentos de fila y segmentos de columna sin repetir segmento de ningún lado: exactamente un emparejamiento en el grafo bipartito (segmentos de fila) $\times$ (segmentos de columna), donde cada casilla vacía es una arista posible.

Ejercicio aplicado

Dado un tablero $N \times N$ con algunas casillas ocupadas por peones, hallar el máximo número de torres que se pueden colocar en casillas vacías sin que se ataquen entre sí (dos torres se atacan si comparten fila o columna sin un peón entre ellas). La entrada trae varios tableros seguidos.

I/O: $N \leq 100$, múltiples casos hasta EOF (patrón 4 de la sección de E/S) $\rightarrow$ lectura total con `sys.stdin.read().split()` y procesamiento por índice.

Código solución

```
import sys

def try_kuhn(u, adj, matched, visited):
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
            if matched[v] == -1 or try_kuhn(matched[v],
            adj, matched, visited):
                matched[v] = u
                return True
    return False
```

```

        matched[v] = u
        return True
    return False

def solve_board(n, grid):
    # etiqueta cada corrida maximal de celdas vacias en
    # cada FILA con un id
    row_id = [[-1] * n for _ in range(n)]
    row_count = 0
    for r in range(n):
        c = 0
        while c < n:
            if grid[r][c] == '.':
                while c < n and grid[r][c] == '.':
                    row_id[r][c] = row_count
                    c += 1
                row_count += 1
            else:
                c += 1

    # lo mismo por COLUMNAS
    col_id = [[-1] * n for _ in range(n)]
    col_count = 0
    for c in range(n):
        r = 0
        while r < n:
            if grid[r][c] == '.':
                while r < n and grid[r][c] == '.':
                    col_id[r][c] = col_count
                    r += 1
                col_count += 1
            else:
                r += 1

    # grafo bipartito: segmentos de fila (izq) --
    # segmentos de columna (der);
    # cada celda vacia es una arista posible entre ambos
    # segmentos
    adj = [[] for _ in range(row_count)]
    for r in range(n):
        for c in range(n):
            if grid[r][c] == '.':
                adj[row_id[r][c]].append(col_id[r][c])

    matched = [-1] * col_count
    total = 0
    for u in range(row_count):
        visited = [False] * col_count
        if try_kuhn(u, adj, matched, visited):
            total += 1
    return total

def main():
    data = sys.stdin.read().split()
    idx = 0
    out = []
    while idx < len(data):
        n = int(data[idx]); idx += 1
        grid = data[idx:idx + n]; idx += n
        out.append(str(solve_board(n, grid)))
    sys.stdout.write("\n".join(out) + "\n")

main()

```

4.3 Teorema de König: Cobertura Mínima de Vértices (quitar caballos)

Preguntas guía:

- ¿Piden el mínimo número de elementos a eliminar para que ningún conflicto (arista) quede activo?

- ¿El grafo de conflictos es bipartito (se puede partir en 2 grupos donde todo conflicto va de un grupo al otro)?
- ¿Ya sé calcular el emparejamiento máximo de ese grafo (entrada anterior, Kuhn)?

Utilidad: Calcular la cobertura mínima de vértices (mínimo conjunto de nodos que toca todas las aristas) en un grafo bipartito. En grafos generales este problema es NP-difícil, pero en grafos bipartitos se vuelve polinomial gracias al teorema de König.

Complejidad: $\mathcal{O}(V \cdot E)$, la misma del emparejamiento bipartito máximo del que depende.

Fundamento teórico: El teorema de König establece que, en cualquier grafo bipartito, el tamaño de la cobertura mínima de vértices es EXACTAMENTE igual al tamaño del emparejamiento máximo. Basta con correr Kuhn (entrada anterior) y esa cifra ya es la respuesta. Importante: heurísticas locales (como quitar iterativamente puntos de articulación, o dividir el número de vértices con grado > 0 entre 2) NO son válidas para este problema en general - solo coinciden por casualidad en casos muy simples (aristas aisladas, caminos, estrellas) y fallan en cuanto el grafo tiene una estructura más rica (por ejemplo, un grafo bipartito 2-conexo sin puntos de articulación). [7]

Ejercicio aplicado

Dado un tablero $N \times N$ con K caballos de ajedrez colocados, hallar el mínimo número de caballos que hay que quitar para que ninguno ataque a otro. El grafo de ataques es bipartito por construcción: cada movimiento de caballo cambia la paridad de (fila + columna), así que alcanza con particionar por esa paridad y correr Kuhn.

I/O: $N \leq 25$, $K \leq N^2 \leq 625 \rightarrow$ entrada pequeña, readline por línea (patrón 1).

Código solución

```

import sys
input = sys.stdin.buffer.readline

def try_kuhn(u, adj, matched, visited):
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
            if matched[v] == -1 or try_kuhn(matched[v],
                adj, matched, visited):
                matched[v] = u
                return True
    return False

def max_bipartite_matching(left_nodes, n_right, adj):
    matched = [-1] * n_right
    total = 0
    for u in left_nodes:
        visited = [False] * n_right
        if try_kuhn(u, adj, matched, visited):
            total += 1
    return total

def solve():
    n, k = map(int, input().split())
    pos = []
    for _ in range(k):
        r, c = map(int, input().split())
        pos.append((r - 1, c - 1))

```

```

index = {p: i for i, p in enumerate(pos)}
moves = [(1,2),(2,1),(-1,2),(-2,1),(1,-2),(2,-1),
        ,(-1,-2),(-2,-1)]

# el ataque de caballo SIEMPRE cambia el color (fila+
col) par/impar:
# el grafo de ataques es bipartito por construccion,
sin necesidad
# de demostrarlo -- solo hay que particionar por ese
color
left_nodes = [i for i, (r, c) in enumerate(pos) if (r
+ c) % 2 == 0]
adj = [[] for _ in range(k)]
for i in left_nodes:
    r, c = pos[i]
    for dr, dc in moves:
        j = index.get((r + dr, c + dc))
        if j is not None:
            adj[i].append(j)

# Konig: cobertura minima de vertices ==
emparejamiento maximo
print(max_bipartite_matching(left_nodes, k, adj))

solve()

```

4.4 Componentes Fuertemente Conexas, Kosaraju (reinos de planetas)

Preguntas guía:

- ¿Es un grafo dirigido y preguntan por "grupos" donde todo par de nodos es mutuamente alcanzable?
- ¿Necesito "condensar" el grafo (contraer cada componente en un nodo) para trabajar luego sobre un DAG?
- ¿n puede ser grande ($\sim 10^5$), por lo que hay que evitar recursión profunda en Python?

Utilidad: Particionar un grafo dirigido en componentes fuertemente conexas (SCC): subconjuntos maximales donde cada nodo puede llegar a cualquier otro del mismo subconjunto. Es la base para condensar grafos, resolver 2-SAT y detectar ciclos mutuos.

Complejidad: $O(V + E)$ tiempo y espacio (dos recorridos DFS).

Fundamento teórico: Kosaraju usa dos pasadas de DFS: (1) en el grafo original, registrar el orden de finalización (postorden) de cada nodo; (2) en el grafo transpuesto (aristas invertidas), recorrer los nodos en orden inverso de finalización. Cada árbol de DFS resultante en esta segunda pasada es exactamente una SCC, porque invertir las aristas separa los componentes que antes solo eran alcanzables en una dirección. [8]

Ejercicio aplicado

(CSES 1683 — Planets and Kingdoms) Un juego tiene n planetas conectados por m teletransportadores dirigidos. Dos planetas pertenecen al mismo "reino" si hay ruta en ambos sentidos entre ellos. Determinar, para cada planeta, a qué reino pertenece.

I/O: $n \leq 10^5$ y $m \leq 2 \cdot 10^5 \rightarrow$ lectura total + tokenización (patrón 2). Además, el DFS se implementa de forma iterativa (con pila explícita) para evitar RecursionError en Python con grafos grandes.

Código solución

```

import sys

def kosaraju(n, adj):
    visited = [False] * n
    order = [] # orden de finalizacion (postorden)
    for s in range(n):
        if visited[s]:
            continue
        visited[s] = True
        stack = [(s, iter(adj[s]))] # iterador
        # recuerda donde va en i parte de la lista
        while stack:
            node, it = stack[-1]
            advanced = False
            for nxt in it: # DFS iterativo
                con iteradores
                if not visited[nxt]:
                    visited[nxt] = True
                    stack.append((nxt, iter(adj[nxt])))
                    advanced = True
                    break
            if not advanced: # no quedan
                vecinos: cierra el nodo
                order.append(node)
                stack.pop()

    radj = [[] for _ in range(n)] # grafo
    # transpuesto (aristas invertidas)
    for u in range(n):
        for v in adj[u]:
            radj[v].append(u)

    comp = [0] * n
    visited = [False] * n
    c = 0
    for node in reversed(order): # orden inverso
        de finalizacion
        if visited[node]:
            continue
        c += 1
        visited[node] = True
        comp[node] = c
        stack = [node]
        while stack:
            u = stack.pop()
            for v in radj[u]:
                if not visited[v]:
                    visited[v] = True
                    comp[v] = c
                    stack.append(v)

    return c, comp

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); m = int(data[idx + 1]); idx += 2
    adj = [[] for _ in range(n)]
    for _ in range(m):
        a = int(data[idx]); b = int(data[idx + 1]); idx += 2
        adj[a - 1].append(b - 1)

    k, comp = kosaraju(n, adj)
    out = [str(k), " ".join(map(str, comp))]
    sys.stdout.write("\n".join(out))

solve()

```

4.5 Circuito Euleriano, Hierholzer (ruta postal)

Preguntas guía:

- ¿Piden recorrer TODAS las aristas exactamente una vez (no todos los nodos, como en Hamiltoniano)?
- ¿Se pide un camino o circuito que use cada calle/arista una sola vez?
- ¿Puedo verificar rápido la condición de existencia (grados pares para circuito; 0 ó 2 nodos de grado impar para camino) antes de construir la ruta?

Utilidad: Encontrar un recorrido que use cada arista del grafo exactamente una vez (circuito o camino euleriano). Aplicaciones: rutas de reparto/barrido que cubren todas las calles, o cualquier problema de "recorrer cada arista una vez".

Complejidad: $\mathcal{O}(V + E)$ con la versión iterativa basada en pila (evita recursión y eliminaciones costosas de aristas).

Fundamento teórico: Existe circuito euleriano si y solo si el grafo es conexo (en aristas) y todos los vértices tienen grado par. El algoritmo de Hierholzer avanza por aristas no usadas hasta atascarse; en ese momento inserta el vértice atascado en la ruta final y retrocede. El resultado, leído en el orden en que se van "atascando" los vértices, es un circuito válido. [9]

Ejercicio aplicado

(CSES 1691 — Mail Delivery) Dado un mapa de n cruces y m calles bidireccionales, hallar una ruta que empiece y termine en el cruce 1 (oficina postal) y recorra cada calle exactamente una vez, o determinar que es "IMPOSSIBLE".

I/O: $n \leq 10^5$, $m \leq 2 \cdot 10^5 \rightarrow$ lectura total + tokenización (patrón 2). Se evita `adj.erase()/remove()` ($\mathcal{O}(n)$ por operación) usando un arreglo de punteros `ptr[]` que salta aristas usadas: así la eliminación es $\mathcal{O}(1)$ amortizado.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); m = int(data[idx + 1]); idx += 2

    adj = [[] for _ in range(n + 1)]
    deg = [0] * (n + 1)
    for e in range(m):
        a = int(data[idx]); b = int(data[idx + 1]); idx += 2
        adj[a].append((b, e))
        adj[b].append((a, e))  # grafo no dirigido:
                               # ambos sentidos (e es id de arista)
        deg[a] += 1
        deg[b] += 1

    if deg[1] == 0 or any(deg[v] % 2 for v in range(1, n + 1)):
        print("IMPOSSIBLE")
        return

    used = [False] * m
```

```
ptr = [0] * (n + 1)  # guarda las ids de
                      # las aristas
path = []
stack = [1]
while stack:
    v = stack[-1]
    while ptr[v] < len(adj[v]) and used[adj[v][ptr[v]]]:
        ptr[v] += 1  # salta aristas
                      # ya usadas (O(1) amortizado)
    if ptr[v] == len(adj[v]):  # reviso todas
                              # las aristas de v y no queda ninguna disponible
        path.append(v)
        stack.pop()
    else:
        u, eid = adj[v][ptr[v]]
        used[eid] = True
        stack.append(u)  # avanza por la
                        # arista disponible

if len(path) - 1 != m:  # no se
    recorrieron todas las aristas
    print("IMPOSSIBLE")
    return

sys.stdout.write(" ".join(map(str, path)))

solve()
```

4.6 Puntos de Articulación, Tarjan (islas críticas)

Preguntas guía:

- ¿Piden identificar nodos cuya eliminación desconecta el grafo (o aumenta el número de componentes)?
- ¿El grafo es no dirigido y se pregunta por robustez/conectividad ante fallos de un solo nodo?
- ¿ n puede ser grande, por lo que hay que evitar recursión profunda en Python?

Utilidad: Encontrar los vértices de corte (articulation points) de un grafo no dirigido: nodos que, al eliminarse, aumentan el número de componentes conexas. Es la base para analizar robustez de redes (qué nodo, si falla, parte el sistema en pedazos).

Complejidad: $\mathcal{O}(V + E)$ con DFS iterativo (un solo recorrido con low-link).

Fundamento teórico: Se hace un DFS calculando, para cada nodo, su tiempo de descubrimiento (`disc`) y el menor tiempo de descubrimiento alcanzable usando a lo más una arista de retroceso (`low`). Un nodo u (no raíz) es punto de articulación si tiene un hijo v en el árbol DFS tal que `low[v] ≥ disc[u]`: eso significa que desde el subárbol de v no hay forma de "escapar" hacia arriba de u sin pasar por u . La raíz del DFS es un caso especial: es punto de articulación si tiene más de un hijo en el árbol. [10]

Ejercicio aplicado

Dadas varias pruebas, cada una con n islas y m puentes bidireccionales (grafo conexo), contar cuántas islas, al hundirse, desconectarían partes de la ciudad. La entrada termina con un caso 0 0.

I/O: Múltiples casos con centinela "0 0" (patrón 3 de la sección de E/S). $n \leq 10^4$, $m \leq 10^5 \rightarrow$ lectura total + tokenización

nización, y DFS iterativo con pila explícita (mismo motivo que en Kosaraju: evitar RecursionError/stack overflow).

Código solución

```
import sys

def find_articulation_points(n, adj):
    disc = [0] * n
    low = [0] * n
    visited = [False] * n
    is_ap = [False] * n
    timer = 1

    for start in range(n):
        if visited[start]:
            continue
        visited[start] = True
        disc[start] = low[start] = timer
        timer += 1
        root_children = 0
        # pila: [nodo, padre, iterador de vecinos,
        # ya_salto_arista_al_padre]
        stack = [[start, -1, iter(adj[start]), False]]
        while stack:
            frame = stack[-1]
            node, parent, it, _ = frame
            advanced = False
            for nxt in it:
                if nxt == parent and not frame[3]:
                    frame[3] = True # salta SOLO una
                    # vez la arista de vuelta
                    continue # (por si hay
                    # aristas paralelas)
                if not visited[nxt]:
                    visited[nxt] = True
                    disc[nxt] = low[nxt] = timer
                    timer += 1
                    if node == start:
                        root_children += 1
                    stack.append([nxt, node, iter(adj[nxt]), False])
                    advanced = True
                    break
            else:
                low[node] = min(low[node], disc[nxt])
                # arista de retroceso
                if not advanced:
                    stack.pop()
                if stack:
                    par = stack[-1]
                    par_node = par[0]
                    low[par_node] = min(low[par_node],
                    low[node])
                    # condicion clasica de punto de
                    # articulacion (no raiz)
                    if par_node != start and low[node] >=
                    disc[par_node]:
                        is_ap[par_node] = True
                    if root_children > 1: # caso especial: la
                    # raiz del DFS
                        is_ap[start] = True

        return is_ap

def main():
    data = sys.stdin.buffer.read().split()
    idx = 0
    out = []
    while True:
        n = int(data[idx]); m = int(data[idx + 1]); idx
        += 2
        if n == 0 and m == 0:
            break
```

```
adj = [[] for _ in range(n)]
for _ in range(m):
    u = int(data[idx]) - 1; v = int(data[idx +
1]) - 1; idx += 2
    adj[u].append(v)
    adj[v].append(u)
is_ap = find_articulation_points(n, adj)
out.append(str(sum(is_ap)))
sys.stdout.write("\n".join(out) + "\n")

main()
```

4.7 Bron–Kerbosch: Clique Máximo (grupo compatible)

Preguntas guía:

- ¿Necesito encontrar un clique máximo o todos los cliques máximos de un grafo no dirigido?
- ¿El número de vértices es relativamente pequeño (aproximadamente $n \leq 50$)?
- ¿La solución requiere explorar todas las posibles extensiones de un conjunto de vértices compatible?

Utilidad: Encontrar todos los cliques máximos o el clique máximo de un grafo no dirigido mediante backtracking con poda. Es la técnica estándar para resolver problemas de máxima compatibilidad, grupos completamente conectados y optimización sobre grafos pequeños.

Complejidad: $\mathcal{O}(3^{n/3})$ en el peor caso, que es óptimo para la enumeración de todos los cliques máximos; memoria $\mathcal{O}(n)$. En la práctica, el uso de pivote reduce considerablemente el número de llamadas recursivas.

Fundamento teórico: Durante la búsqueda se mantienen tres conjuntos:

- R : vértices que pertenecen al clique actual.
- P : vértices que todavía pueden añadirse al clique.
- X : vértices ya procesados, utilizados para evitar generar soluciones repetidas.

Cuando tanto P como X quedan vacíos, el conjunto R constituye un clique máximo. Para reducir la cantidad de ramas exploradas se selecciona un pivote u y únicamente se consideran los vértices de $P \setminus N(u)$, donde $N(u)$ representa los vecinos del pivote. Esta optimización disminuye significativamente el árbol de búsqueda. [11]

Ejercicio aplicado

Dado un grafo no dirigido, determinar el tamaño del mayor clique del grafo. Cada vértice representa una persona y existe una arista cuando dos personas son compatibles. Se desea encontrar el grupo más grande donde todos los integrantes sean compatibles entre sí.

I/O: Se reciben múltiples casos de prueba; cada caso especifica el número de vértices y aristas seguido por la lista de conexiones. El algoritmo resulta adecuado para grafos pequeños, donde una búsqueda exhaustiva con poda es suficiente para obtener la solución óptima.

Código solución

```
def bronk(R, P, X):
    global best
```

```

if not P and not X:
    best = max(best, len(R))
    return

if P or X:
    u = next(iter(P | X))
else:
    u = None

if u is None:
    candidates = list(P)
else:
    candidates = list(P - graph[u])

for v in candidates:
    bronk(
        R | {v},
        P & graph[v],
        X & graph[v]
    )

    P.remove(v)
    X.add(v)

n, m = map(int, input().split())
graph = [set() for _ in range(n)]
for _ in range(m):
    u, v = map(int, input().split())
    graph[u].add(v)
    graph[v].add(u)

best = 0
bronk(set(), set(range(n)), set())
print(best)

```

Observación: El algoritmo de Bron–Kerbosch explora únicamente conjuntos que pueden convertirse en un clique válido. La versión con pivote es la utilizada habitualmente en programación competitiva debido a que reduce considerablemente el número de llamadas recursivas. Aunque su complejidad en el peor caso sigue siendo exponencial, resulta extremadamente eficiente para grafos pequeños, siendo la solución estándar para problemas de clique máximo en ICPC.

4.8 Orden Topológico, Kahn + Propagación de Conteos (Digi Comp II)

Preguntas guía:

- ¿Tengo un DAG donde una cantidad enorme de "elementos" entra por un único origen y debo saber cuántos pasan por cada nodo, siendo imposible simularlos uno por uno?
- ¿La forma en que un nodo reparte su cantidad entre sus sucesores depende solo del TOTAL que lo atraviesa (y no del orden individual de llegada)?
- ¿Necesito procesar cada nodo únicamente cuando TODOS sus predecesores ya aportaron su parte, es decir, en orden topológico?
- ¿El resultado final por nodo depende solo de la paridad o de un residuo del total acumulado?

Utilidad: Sustituye la simulación individual de n recorridos sobre un DAG (inviabile cuando n es astronómico) por un único barrido en orden topológico que propaga CONTEOS agregados en lugar de elementos. El orden

topológico de Kahn garantiza que, al procesar un nodo, ya se conoce su total definitivo, de modo que el reparto hacia sus sucesores se calcula una sola vez, en tiempo lineal, sin importar cuán grande sea n .

Complejidad: $\mathcal{O}(V + E)$ para el orden topológico y la propagación, con $E = 2V$ aquí (cada nodo tiene exactamente dos aristas de salida).

Fundamento teórico: La observación clave es que el efecto de n bolas sobre un interruptor NO depende del orden en que llegan: si t bolas pasan por un vértice v , el interruptor alterna su estado en cada una, así que las bolas se reparten alternadamente entre las dos salidas. Si el estado inicial es L, salen $\lceil t/2 \rceil$ por la izquierda y $\lfloor t/2 \rfloor$ por la derecha (y al revés si el estado inicial es R); el estado FINAL del interruptor solo depende de la PARIDAD de t : se invierte si t es impar y queda igual si es par. Esto convierte un problema aparentemente secuencial en uno puramente aritmético. Para que el cálculo sea válido hace falta conocer el total t de cada vértice ANTES de procesarlo, lo cual es exactamente lo que garantiza el orden topológico: el algoritmo de Kahn mantiene un contador de grado de entrada y solo encola un vértice cuando todas sus aristas entrantes han sido consumidas, es decir, cuando todos sus predecesores ya sumaron su aporte a `count`. El enunciado garantiza que el grafo es acíclico (una bola siempre baja y nunca puede volver a un interruptor), condición necesaria para que exista dicho orden. Nótese que cuando $L = R$ el vértice destino recibe DOS unidades de grado de entrada y también se decrementa dos veces, por lo que el conteo sigue siendo consistente y el vértice no se encola prematuramente. [12]

Ejercicio aplicado

(Kattis digicomp2 — Digi Comp II) Una máquina Digi Comp II se modela como un DAG de m interruptores, cada uno con estado L/R y dos aristas de salida. Entran n bolas por el vértice 1; cada bola sigue la salida indicada por el estado del interruptor y lo invierte al pasar. Reportar el estado final de los m interruptores.

I/O: $m \leq 5 \cdot 10^5$ con 3 tokens por interruptor (carácter + dos enteros) $\Rightarrow$ lectura total + tokenización con índice `idx` (patrón 2), comparando el carácter directamente como `bytes` (`b'L'`) sin decodificar. $n \leq 10^{18}$ cabe sin problema en los enteros nativos de Python. La salida se arma en un `bytearray` de m posiciones escribiendo los códigos ASCII 76/82 ('L'/'R') y se emite con un único `sys.stdout.buffer.write`, evitando concatenar medio millón de strings. Verificado contra el caso de ejemplo y contra una simulación bola por bola en 500 DAGs aleatorios pequeños (incluyendo casos con $L = R$) sin discrepancias, más casos borde ($n = 0$, $m = 1$, cadena de profundidad máxima, todos los nodos apuntando al vértice final). En el peor caso ($m = 500\,000$, $n = 10^{18}$) se midieron ~ 1.43 s en una cadena de profundidad máxima y ~ 0.97 s en un DAG denso, frente a un límite de 7 segundos.

Código solución

```

import sys
from collections import deque

data = sys.stdin.buffer.read().split()
idx = 0
n = int(data[idx])
m = int(data[idx+1])
idx += 2

gamestate = [0]*(m+1)    # 1 = L, 2 = R
left = [0]*(m+1)
right = [0]*(m+1)
indegree = [0]*(m+1)

for i in range(1, m+1):
    c = data[idx]
    L = int(data[idx+1])
    R = int(data[idx+2])
    idx += 3
    gamestate[i] = 1 if c == b'L' else 2
    left[i] = L
    right[i] = R
    indegree[L] += 1      # si L == R, el destino suma 2 (
    # y luego se resta 2)
    indegree[R] += 1

count = [0]*(m+1)        # cuantas bolas pasan por cada
    # vertice
count[1] = n              # todas las bolas entran por el
    # vertice 1
final_state = gamestate[:]

queue = deque(v for v in range(m+1) if indegree[v]==0)

while queue:
    v = queue.popleft()
    if v == 0:
        continue        # el vertice 0 es el final: no
    # reparte nada

    total = count[v]
    half = total//2
    odd = total&1
    # el interruptor alterna: el reparto es mitad y mitad
    # , y la bola
    # sobrante (si total es impar) se va por el lado del
    # estado INICIAL
    if gamestate[v] == 1:
        left_count = half+odd
        right_count = half
    else:
        right_count = half+odd
        left_count = half
    if odd:
        # solo la paridad del total determina si el
        # estado queda invertido
        final_state[v] = 2 if gamestate[v] == 1 else 1

    lv, rv = left[v], right[v]
    count[lv] += left_count
    indegree[lv] -= 1
    if indegree[lv] == 0:    # ya recibio el aporte de
    # TODOS sus predecesores
        queue.append(lv)
    count[rv] += right_count
    indegree[rv] -= 1
    if indegree[rv] == 0:
        queue.append(rv)

out = bytearray(m)
for i in range(1, m+1):
    out[i-1] = 76 if final_state[i] == 1 else 82    # 76='
    # L', 82='R'
sys.stdout.buffer.write(bytes(out))

```

4.9 Union-Find / DSU con Path Halving (componentes conexas)

Preguntas guía:

- ¿Me piden cuántos grupos/islas/componentes separados hay en un grafo no dirigido, o si dos elementos están en el mismo grupo?
- ¿Las aristas llegan de forma incremental (una por una) y solo necesito consultar conectividad, sin nunca eliminar aristas?
- ¿Puedo tolerar aristas repetidas y lazos sin que el conteo se rompa?
- ¿Prefiero evitar construir listas de adyacencia y hacer DFS/BFS explícito?

Utilidad: Mantiene una partición de elementos en conjuntos disjuntos soportando dos operaciones: unir los conjuntos de dos elementos, y averiguar a qué conjunto pertenece un elemento. Contar componentes conexas es su aplicación más directa, pero la estructura es la base de Kruskal (árbol de recubrimiento mínimo), detección de ciclos en grafos no dirigidos, agrupamiento por equivalencias y problemas de conectividad incremental en general. A diferencia de DFS/BFS, no requiere materializar el grafo: procesa las aristas al vuelo, una por una.

Complejidad: $\mathcal{O}(M\alpha(N))$ para procesar M aristas sobre N nodos, donde α es la inversa de la función de Ackermann (menor que 5 para cualquier entrada imaginable), es decir prácticamente $\mathcal{O}(M)$ amortizado con compresión de caminos.

Fundamento teórico: Cada conjunto se representa como un árbol donde cada nodo apunta a su padre y la RAÍZ (el nodo que es su propio padre) actúa como identificador canónico del conjunto. Dos elementos pertenecen al mismo conjunto exactamente cuando su búsqueda de raíz termina en el mismo nodo, así que el número de componentes conexas es el número de nodos que son su propia raíz. Unir dos conjuntos cuesta una sola asignación: se hace que la raíz de uno apunte a la raíz del otro. La eficiencia depende de mantener los árboles poco profundos, y para eso se usa COMPRESIÓN DE CAMINOS: durante la búsqueda de la raíz, cada nodo visitado se reengancha más arriba en el árbol, de modo que las consultas futuras sobre ese nodo (y sobre todos sus descendientes en el camino) sean más cortas. La variante implementada aquí, *path halving*, hace que cada nodo apunte a su abuelo en lugar de a su padre mientras sube, lo que reduce la profundidad a la mitad en cada pasada con un solo bucle y sin recursión ni segunda pasada. El resultado es que la profundidad amortizada queda acotada por $\alpha(N)$, una función que crece tan lentamente que es constante en la práctica. Nótese que la estructura es naturalmente inmune a aristas repetidas y lazos: si a y b ya comparten raíz, la unión simplemente no hace nada, así que la entrada puede contener CE y EC (como en el ejemplo) o incluso AA sin afectar el conteo.

Ejercicio aplicado

(UVA 459 — Graph Connectivity) Dado el nombre del nodo mayor del grafo (una letra mayúscula, que implica los nodos desde A hasta esa letra) y una lista de aristas no dirigidas expresadas como pares de letras, contar cuántos subgrafos conexos maximales (componentes conexas) tiene el grafo.

I/O: La entrada separa los casos con LÍNEAS EN BLANCO, lo que normalmente forzaría a leer línea por línea; sin embargo, aquí la estructura por líneas es redundante: el nodo mayor siempre es un token de 1 carácter y cada arista es un token de 2 caracteres. Eso permite usar lectura total + tokenización (patrón 2) y distinguir el inicio de cada caso por la LONGITUD del token, consumiendo aristas mientras midan 2 — el parseo se vuelve inmune a cualquier cantidad de líneas en blanco, espacios extra o retornos de carro. Se opera directamente sobre bytes, restando 64 al código ASCII para mapear 'A'→1 hasta 'Z'→26 (a lo sumo 26 nodos, así que el rendimiento es irrelevante). La salida exige una línea en blanco entre casos consecutivos, lo que se logra con `"\n\n".join(out)`. Verificado contra el caso de ejemplo y contrastado con una fuerza bruta de BFS sobre listas de adyacencia en 120 tandas aleatorias (incluyendo lazos AA y aristas duplicadas), más casos borde de un solo nodo y de casos sin ninguna arista, sin discrepancias.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    t = int(data[idx]); idx += 1
    out = []
    for _ in range(t):
        # el nodo mayor es un token de 1 caracter; las
        # aristas son de 2
        n = data[idx][0] - 64          # 'A'=65 -> 1 nodo
        ... 'Z'=90 -> 26 nodos
        idx += 1
        padre = list(range(n + 1))

        def find(x):
            while padre[x] != x:
                padre[x] = padre[padre[x]]    # path
            halving: apunta al abuelo
            x = padre[x]
            return x

        # consume aristas mientras el token siguiente
        # tenga 2 caracteres:
        # así el parseo ignora por completo las líneas en
        # blanco del enunciado
        while idx < len(data) and len(data[idx]) == 2:
            tok = data[idx]; idx += 1
            a = find(tok[0] - 64)
            b = find(tok[1] - 64)
            if a != b:                # si ya comparten
                raiz (arista repetida
                padre[a] = b          # o lazo) no se
            # cada nodo que es su propia raiz representa una
            # componente
            out.append(str(sum(1 for v in range(1, n + 1) if
                find(v) == v)))
    sys.stdout.write("\n\n".join(out) + "\n")
```

solve()

4.10 Dijkstra con heapq (Caminos Mínimos desde un Origen)

Preguntas guía:

- ¿Me piden la distancia mínima desde un nodo origen a todos los demás en un grafo con pesos NO negativos?
- ¿Los pesos son variables, de modo que un BFS simple (que asume peso 1 por arista) no sirve?
- ¿El grafo es disperso (m del orden de n), con n y m de hasta unos pocos cientos de miles?
- ¿Puede haber aristas paralelas, lazos, o varias formas de llegar a un mismo nodo que mejoran su distancia más de una vez?

Utilidad: Calcula las distancias mínimas desde un origen a todos los nodos cuando los pesos son no negativos, procesando los nodos en orden creciente de distancia con una cola de prioridad. Es el algoritmo base de caminos mínimos: rutas en mapas, costo mínimo en grafos de estados, y como subrutina de problemas más grandes (varios orígenes con un supernodo, grafo con estados extra como "¿ya usé el cupón?", reconstrucción del camino guardando el padre de cada nodo). Con pesos negativos deja de ser correcto y hace falta Bellman-Ford.

Complejidad: $\mathcal{O}((n + m) \log m)$ con la versión de borrado perezoso: cada arista puede insertar a lo sumo una entrada en el heap, así que el heap tiene $\mathcal{O}(m)$ elementos en total.

Fundamento teórico: El invariante es que, cuando un nodo sale del heap con la menor distancia tentativa de todo el heap, esa distancia es definitiva. La razón es que los pesos son no negativos: cualquier otro camino hacia ese nodo tiene que pasar primero por algún nodo que todavía está en el heap, con distancia tentativa mayor o igual, y agregarle aristas de peso ≥ 0 no puede hacerlo más corto. Por eso los nodos se fijan en orden creciente de distancia, y al fijar u basta relajar sus aristas salientes: si $\text{dist}[u] + w < \text{dist}[v]$, se mejora $\text{dist}[v]$ y se inserta $(\text{dist}[v], v)$ en el heap. Como **heapq** no permite disminuir la prioridad de un elemento ya insertado, se usa **BORRADO PEREZOSO**: se inserta una entrada nueva con la distancia mejorada y la vieja queda en el heap; cuando una entrada sale con distancia mayor que $\text{dist}[u]$, es obsoleta y se descarta con un **continue**. Así un nodo puede estar varias veces en el heap, pero solo se procesa una vez, y el total de inserciones está acotado por el número de relajaciones exitosas, es decir por m . La hipótesis de pesos no negativos es imprescindible: con una arista negativa, un nodo ya fijado podría mejorar después, y el invariante se rompe. [13]

Ejercicio aplicado

(CSES 1671 — Shortest Routes I) Dado un grafo dirigido de n ciudades y m vuelos con longitud, calcular la distancia mínima desde la ciudad 1 a cada ciudad (se garantiza que todas son alcanzables).

I/O: $n \leq 10^5$, $m \leq 2 \cdot 10^5$ con tres enteros por arista $\Rightarrow$ lectura total + tokenización (patrón 2). Las distancias pueden llegar a $\sim 10^{14}$, sin problema para los enteros de Python. La salida son n números en una sola línea, unidos con un único `write`. Verificado contra el caso de ejemplo y contrastado con Bellman-Ford en 400 tandas aleatorias con lazos, aristas paralelas y pesos de 1 a 10^9 , sin discrepancias. Con $n = 10^5$ y $m = 2 \cdot 10^5$ tarda entre ~ 0.24 s y ~ 0.35 s en los tres grafos medidos: árbol más aristas aleatorias, una cadena de peso 1 con atajos caros desde el origen (cada nodo se inserta dos veces), y un grafo con muchas relajaciones sucesivas; el límite es de 1 segundo.

Código solución

```
import sys, heapq

def solve():
    data = sys.stdin.buffer.read().split()
    n, m = int(data[0]), int(data[1])
    adj = [[] for _ in range(n + 1)]
    for i in range(m):
        a, b, c = int(data[2+3*i]), int(data[3+3*i]), int(
            data[4+3*i])
        adj[a].append((b, c))
    INF = float('inf')
    dist = [INF]*(n + 1)
    dist[1] = 0
    pq = [(0, 1)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                # no se puede "bajar la prioridad" en
                # heapq: se inserta de nuevo
                heapq.heappush(pq, (nd, v))
    sys.stdout.write(" ".join(map(str, dist[1:])) + "\n")

solve()
```

Extras: versión con entradas codificadas en un entero — aproximadamente un 20% más rápida, útil si el grafo es más grande o el límite más ajustado. En lugar de tuplas, cada arista se guarda como `peso < 17 | destino` y cada entrada del heap como `distancia < 17 | nodo`: comparar enteros es más barato que comparar tuplas, y como la distancia ocupa los bits altos, el orden del heap sigue siendo por distancia. Requiere que 2^{SH} supere al mayor número de nodo; con $n \leq 10^5$ basta $SH = 17$. Verificada igual que la versión principal; en los mismos grafos tarda entre ~ 0.22 s y ~ 0.28 s.

```
import sys
from heapq import heappush, heappop

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])
    A = list(map(int, data[2:2 + 3*m:3]))
    B = list(map(int, data[3:3 + 3*m:3]))
    C = list(map(int, data[4:4 + 3*m:3]))
```

```
SH = 17 # 2^SH debe ser mayor que n
MASK = (1 << SH) - 1
adj = [[] for _ in range(n + 1)]
for a, e in zip(A, [(c << SH) | b for b, c in zip(B, C)]):
    adj[a].append(e) # arista: peso << SH | destino
INF = 1 << 62
dist = [INF]*(n + 1)
dist[1] = 0
pq = [(0, 1)] # distancia 0, nodo
# 1 -> (0 << SH) | 1
while pq:
    x = heappop(pq)
    u = x & MASK # bits bajos: nodo
    d = x >> SH # bits altos: distancia
    if d > dist[u]:
        continue
    for e in adj[u]:
        v = e & MASK
        nd = d + (e >> SH)
        if nd < dist[v]:
            dist[v] = nd
            heappush(pq, (nd << SH) | v)
    sys.stdout.write(" ".join(map(str, dist[1:])) + "\n")

solve()
```

4.11 BFS en Grilla con Reconstrucción del Camino (y 0-1 BFS)

Preguntas guía:

- ¿Me piden el camino más corto en una grilla o grafo donde todos los movimientos cuestan lo mismo?
- ¿Además de la distancia, debo imprimir el camino (la secuencia de movimientos o de nodos)?
- ¿La grilla llega a 10^6 celdas, de modo que en Python necesito evitar tuplas de coordenadas y chequeos de borde en cada paso?
- ¿Los movimientos cuestan 0 o 1 (por ejemplo, romper una pared cuesta 1 y caminar cuesta 0), caso en que sirve el 0-1 BFS de los extras?

Utilidad: El BFS recorre el grafo por capas desde el origen, así que la primera vez que alcanza un nodo lo hace por un camino de la menor cantidad de pasos posible. Guardando en cada nodo por dónde se llegó, se reconstruye el camino completo al final. Es la herramienta para laberintos, mínimo número de movimientos en un tablero, distancias en grafos no ponderados y, con varios orígenes en la cola inicial, distancia de cada celda a la fuente más cercana. Su variante 0-1 BFS resuelve caminos mínimos cuando cada arista pesa 0 o 1, sin el costo de un heap.

Complejidad: $\mathcal{O}(nm)$ para una grilla de $n \times m$ (cada celda entra a la cola una sola vez), y lo mismo para reconstruir el camino, que tiene a lo sumo nm pasos. El 0-1 BFS también es $\mathcal{O}(V + E)$.

Fundamento teórico: El BFS mantiene la invariante de que la cola contiene nodos de a lo sumo dos distancias consecutivas, d y $d + 1$, con todos los de distancia d adelante. Por eso los nodos se extraen en orden creciente de

distancia, y al descubrir un nodo por primera vez desde uno a distancia d , su distancia es exactamente $d + 1$: si existiera un camino más corto, lo habría descubierto antes algún nodo de una capa anterior. Esto permite marcar cada celda como visitada en el momento de agregarla a la cola, sin volver a mirarla. Para reconstruir, no hace falta guardar la celda padre: basta guardar la DIRECCIÓN con la que se entró (U, D, L, R), y desde el destino se camina hacia atrás deshaciendo cada movimiento hasta llegar al origen; el camino sale invertido y se da vuelta al final. Dos decisiones de implementación hacen que en Python recorra un millón de celdas con holgura: la grilla se APLANA en un solo arreglo de ancho $W = m + 2$, donde los vecinos de u son $u \pm 1$ y $u \pm W$, y se RODEA de un marco de paredes, de modo que un vecino fuera de la grilla es simplemente una pared y no hace falta comparar índices en cada paso. El 0-1 BFS generaliza la invariante: con aristas de peso 0 o 1, un nodo alcanzado por una arista de peso 0 tiene la misma distancia que el actual, así que se pone al FRENTE de la deque (`appendleft`), y uno alcanzado por peso 1 va al final. La deque sigue ordenada por distancia con solo dos valores distintos, lo que reemplaza al heap de Dijkstra por operaciones $\mathcal{O}(1)$. A diferencia del BFS simple, en el 0-1 BFS un nodo puede mejorar después de descubierto (primero por peso 1, luego por un camino con aristas de peso 0), por eso se compara contra su distancia actual en vez de marcarlo como visitado de una vez.

Ejercicio aplicado

(CSES 1193 — Labyrinth) Dado un laberinto de $n \times m$ con paredes, un inicio A y un destino B, decidir si existe un camino moviéndose en las cuatro direcciones y, si existe, imprimir su longitud mínima y los movimientos.

I/O: $n, m \leq 1000$ con filas de texto sin espacios $\Rightarrow$ lectura total + tokenización (patrón 2), donde cada fila del laberinto es un token que se concatena como `bytes` en la grilla aplanada. La conversión a pasable/no pasable se hace con `bytes.translate`, en C. Como se acepta cualquier camino óptimo, la verificación no compara texto sino que comprueba la respuesta YES/NO, que la longitud sea la mínima y que el camino recorrido no atraviese paredes y termine en B: se hizo en 500 laberintos aleatorios sin paredes, densos, con A pegado a B y con n o m igual a 1, sin fallos. En grillas de 1000×1000 tarda ~ 0.22 s sin paredes con A y B en esquinas opuestas (recorre todo), ~ 0.15 s en un laberinto en zigzag con un camino de casi 500 000 pasos (reconstrucción más larga), y ~ 0.22 s cuando B está encerrado y la respuesta es NO; el límite es de 1 segundo. La versión de libro, con tuplas de coordenadas, deque y chequeo de bordes, da lo mismo pero tarda hasta ~ 0.61 s. El 0-1 BFS de los extras se verificó contra Dijkstra en 600 grillas aleatorias y recorre una de 1000×1000 en ~ 0.42 s.

Código solución

```
import sys

def solve():
```

```
data = sys.stdin.buffer.read().split()
n = int(data[0]); m = int(data[1])
W = m + 2
# grilla aplanada con un marco de paredes: nunca hay
# que chequear bordes
g = bytearray(b"#" * W)
for i in range(n):
    g += b"#" + data[2 + i] + b"#"
g += b"#" * W
s = g.find(b"A"); t = g.find(b"B")
libre = bytearray(len(g))
libre[:] = g.translate(bytes.maketrans(b".AB#", b"\x01\x01\x00\x00"))
vino = bytearray(len(g)) # letra del
# movimiento con que se llegó (0 = no visitada)
libre[s] = 0
cola = [s]
U, D, L, R = -W, W, -1, 1 # vecinos en la
# grilla aplanada
for u in cola: # la lista crece
    # mientras se recorre: es la cola BFS
    if u == t:
        break
    # se marca al encolar: la primera vez que se
    # alcanza es con distancia mínima
    v = u + U
    if libre[v]: libre[v] = 0; vino[v] = 85; cola.
    append(v) # 'U'
    v = u + D
    if libre[v]: libre[v] = 0; vino[v] = 68; cola.
    append(v) # 'D'
    v = u + L
    if libre[v]: libre[v] = 0; vino[v] = 76; cola.
    append(v) # 'L'
    v = u + R
    if libre[v]: libre[v] = 0; vino[v] = 82; cola.
    append(v) # 'R'
if not vino[t]:
    sys.stdout.write("NO\n"); return
atras = {85: W, 68: -W, 76: 1, 82: -1} #
# desplazamiento que deshace cada movimiento
camino = bytearray()
u = t
while u != s: # del destino al
    # origen, deshaciendo movimientos
    c = vino[u]
    camino.append(c)
    u += atras[c]
camino.reverse()
sys.stdout.write("YES\n%d\n%s\n" % (len(camino),
    camino.decode()))

solve()
```

Extras: 0-1 BFS — caminos mínimos cuando cada movimiento cuesta 0 o 1. Ejemplo verificado: mínima cantidad de paredes a romper para ir de A a B, donde entrar a . cuesta 0 y entrar a # cuesta 1. Usa la misma grilla aplanada con marco (aquí el marco se marca con costo 2 para distinguirlo).

```
from collections import deque

def paredes_minimas(g, n, m):
    # g: lista de n filas (bytes) de largo m
    W = m + 2
    INF = 1 << 30
    costo = bytearray(b"\x02" * W) # 2 = fuera
    # de la grilla
    for fila in g:
        costo += b"\x02" + fila.translate(bytes.maketrans(
            (b".AB#", b"\x00\x00\x00\x01"))) + b"\x02"
```

```

costo += b"\x02"*W
plano = b"".join(b"#"+f+b"#"+f for f in g)
s = W + plano.find(b"A"); t = W + plano.find(b"B")
dist = [INF]*len(costo)
dist[s] = 0
dq = deque([s])
while dq:
    u = dq.popleft()
    du = dist[u]
    for v in (u - W, u + W, u - 1, u + 1):
        c = costo[v]
        if c == 2: continue
        nd = du + c
        if nd < dist[v]:                # un nodo
            puede mejorar despues de descubierto
            dist[v] = nd
            if c: dq.append(v)          # peso 1: al
            final de la deque
            else: dq.appendleft(v)     # peso 0: al
            frente (misma distancia)
return dist[t]

```

4.12 Grafo Bipartito por Coloreo (y DFS Iterativo Genérico)

Preguntas guía:

- ¿Me piden dividir los nodos en dos grupos de modo que ninguna arista quede dentro de un mismo grupo (2-coloreo)?
- ¿Debo detectar si el grafo tiene un ciclo de longitud IMPAR, que es exactamente lo que impide la división?
- ¿El grafo puede estar desconectado, de modo que hay que recorrer cada componente por separado?
- ¿Necesito un recorrido en profundidad con tiempos de entrada y salida (orden topológico, subárboles, puentes) sin recursión, porque la profundidad puede llegar a 10^5 ?

Utilidad: Decide si un grafo es bipartito y, si lo es, construye una asignación de dos colores válida, recorriendo cada componente y dando a cada nodo el color opuesto al de quien lo descubrió. Es la base de problemas de dos equipos, asignaciones con conflictos por pares, detección de ciclos impares y del paso previo al emparejamiento bipartito. El DFS iterativo genérico de los extras es la plantilla para cualquier recorrido en profundidad que necesite el orden real de visita (preorden, postorden, padre en el árbol DFS) sin el límite de recursión de Python.

Complejidad: $\mathcal{O}(n+m)$: cada nodo se colorea una vez y cada arista se revisa dos veces (una desde cada extremo).

Fundamento teórico: Un grafo es bipartito si y solo si no tiene ciclos de longitud impar. Si hay un ciclo impar, al recorrerlo alternando colores el último nodo debe tener el color opuesto al primero y a la vez el mismo, porque el ciclo tiene un número impar de aristas: no hay asignación posible. Recíprocamente, dentro de una componente conexa, fijar el color de un nodo determina el de todos los demás: el color de cada nodo queda fijado por la paridad de cualquier camino desde el origen, y la asignación es consistente exactamente cuando todos los caminos entre dos nodos tienen la misma paridad, lo que equivale a no tener ciclos impares. Por eso el algoritmo

no necesita "elegir": colorea el origen con 1, cada nodo descubierto con el color opuesto al de su descubridor, y si alguna arista une dos nodos ya coloreados con el mismo color, encontró un ciclo impar y responde IMPOSSIBLE. Como el color depende solo de la paridad y no del orden, da igual recorrer en profundidad o por capas; se usa una pila simple, que en Python es más barata que una deque. Cada componente es independiente, así que se inicia un recorrido nuevo desde cada nodo aún sin color. El DFS genérico de los extras sí necesita el orden exacto de profundidad: la pila guarda pares (nodo, iterador de vecinos), y el iterador recuerda en qué vecino quedó cada nodo, de modo que al volver a él se continúa donde se había dejado. Al descubrir un vecino no visitado se registra su entrada (preorden) y se lo apila; cuando el iterador de un nodo se agota, la cláusula `else` del `for` registra su salida (postorden). Así se reproducen exactamente los mismos tiempos que el DFS recursivo, sin su riesgo de desbordar la pila.

Ejercicio aplicado

(CSES 1668 — Building Teams) Dados n alumnos y m amistades, dividirlos en dos equipos de modo que no haya dos amigos en el mismo equipo, o informar que es imposible.

I/O: $n \leq 10^5$, $m \leq 2 \cdot 10^5$ con dos enteros por arista $\Rightarrow$ lectura total + tokenización (patrón 2), separando los extremos con slices de paso 2. Los colores se guardan en un `bytearray`, que ocupa un byte por nodo. Como se acepta cualquier asignación válida, la verificación no compara texto: en 600 grafos aleatorios de hasta 11 nodos (sin aristas, varias componentes, ciclos pares e impares) se comprobó que la respuesta es IMPOSSIBLE exactamente cuando ninguna de las 2^n asignaciones sirve, y que en los demás casos ninguna arista queda dentro de un equipo, sin fallos. Con $n = 10^5$ y $m = 2 \cdot 10^5$ tarda entre ~ 0.06 s y ~ 0.21 s: grafo bipartito aleatorio, camino largo con saltos, un único ciclo impar de 99 999 nodos, y 50 000 componentes; el límite es de 1 segundo. El DFS genérico de los extras se verificó contra un DFS recursivo en 500 grafos dirigidos y no dirigidos, obteniendo tiempos de entrada, de salida y padres idénticos, y tarda ~ 0.07 s en un camino de 10^5 nodos (donde la versión recursiva desbordaría la pila) y ~ 0.14 s en un grafo aleatorio de 10^5 nodos y $2 \cdot 10^5$ aristas.

Código solución

```

import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])
    adj = [[] for _ in range(n + 1)]
    for a, b in zip(map(int, data[2:2 + 2*m:2]), map(int,
        data[3:3 + 2*m:2])):
        adj[a].append(b)
        adj[b].append(a)
    color = bytearray(n + 1)          # 0 = sin color, 1
    o 2 = equipo
    for s in range(1, n + 1):        # una recorrida por
        cada componente
        if color[s]:
            continue

```

```

    color[s] = 1
    pila = [s] # el orden no
    importa: solo la paridad
    while pila:
        u = pila.pop()
        cu = color[u]
        otro = 3 - cu # 1 <-> 2
        for v in adj[u]:
            cv = color[v]
            if not cv:
                color[v] = otro
                pila.append(v)
            elif cv == cu: # arista dentro de
                un mismo equipo: ciclo impar
                sys.stdout.write("IMPOSSIBLE\n");
    return
    sys.stdout.write(" ".join(map(str, color[1:])) + "\n")
solve()

```

Extras: DFS iterativo genérico — plantilla con preorden, postorden, tiempos de entrada y salida y padre en el árbol DFS, recorriendo todas las componentes. Sirve tal cual para grafos dirigidos y no dirigidos. Con los tiempos se responde en $\mathcal{O}(1)$ si u es ancestro de v ($\text{tin}[u] \leq \text{tin}[v]$ y $\text{tout}[v] \leq \text{tout}[u]$), y ordenando los nodos por tout decreciente se obtiene un orden topológico de un DAG.

```

def dfs_iterativo(n, adj):
    tin = [-1]*(n + 1)
    tout = [-1]*(n + 1)
    padre = [0]*(n + 1)
    t = 0
    for raiz in range(1, n + 1):
        if tin[raiz] != -1:
            continue
        tin[raiz] = t; t += 1
        pila = [(raiz, iter(adj[raiz]))]
        while pila:
            u, it = pila[-1]
            for v in it: # el
                iterador recuerda por donde iba u
                if tin[v] == -1:
                    padre[v] = u
                    tin[v] = t; t += 1 # ----
                PREORDEN: entrar a v ----
                pila.append((v, iter(adj[v])))
                break # bajar a v
            antes de seguir con u
            else: # se
                agotaron los vecinos de u
                pila.pop()
                tout[u] = t; t += 1 # ----
                POSTORDEN: salir de u ----
    return tin, tout, padre

```

4.13 Árbol de Expansión Mínima (Kruskal + Union-Find)

Preguntas guía:

- ¿Me piden conectar todos los nodos de un grafo no dirigido con el menor costo total posible, eligiendo un subconjunto de aristas?
- ¿Debo detectar si el grafo es desconectado, caso en que no existe tal conjunto de aristas?
- ¿Las aristas vienen como lista y el grafo es disperso, de modo que ordenarlas por peso es barato?

- ¿Ya tengo un Union-Find a mano para saber si dos nodos ya quedaron conectados?

Utilidad: Encuentra el árbol de expansión mínima (MST): el conjunto de $n - 1$ aristas que conecta todos los nodos con costo total mínimo. Aplica a redes de costo mínimo (camino, cables, tuberías), a agrupamiento (cortar las $k - 1$ aristas más caras del MST deja k grupos lo más separados posible), y a problemas de "minimizar el máximo" en un camino, porque el MST también minimiza la arista más cara entre cualquier par de nodos. Kruskal es la variante natural cuando el grafo viene como lista de aristas.

Complejidad: $\mathcal{O}(m \log m)$ dominado por el ordenamiento de las aristas, más $\mathcal{O}(m \alpha(n))$ para las operaciones del Union-Find.

Fundamento teórico: La corrección de Kruskal se apoya en la PROPIEDAD DEL CORTE: para cualquier partición de los nodos en dos lados, la arista más barata que cruza de un lado al otro pertenece a algún MST. Si un MST no la usara, agregarla formaría un ciclo que cruza el corte al menos otra vez, por alguna arista de costo mayor o igual; cambiar esa arista por la nueva no aumenta el costo y sigue siendo un árbol. Kruskal recorre las aristas de la más barata a la más cara y toma una arista (a, b) solo si a y b están en componentes distintas: en ese momento, el corte que separa la componente de a del resto no fue cruzado por ninguna arista más barata (si hubiera existido, ya se habría tomado y a, b estarían conectados), así que la arista actual es la más barata de ese corte y la propiedad garantiza que es segura. Las aristas que unirían dos nodos ya conectados cerrarían un ciclo y se descartan. El Union-Find responde en tiempo casi constante si dos nodos ya están en la misma componente (misma raíz) y las une en una asignación. Cada arista aceptada reduce en uno la cantidad de componentes, así que al llegar a $n - 1$ aristas el árbol está completo y se puede cortar el recorrido; si al terminar hay menos, el grafo era desconectado. Con pesos repetidos puede haber varios MST distintos, pero todos tienen el mismo costo, que es lo que se pide. [14]

Ejercicio aplicado

(CSES 1675 — Road Reparation) Dadas n ciudades y m caminos con costo de reparación, elegir qué caminos reparar para que todas las ciudades queden conectadas con costo total mínimo, o informar que es imposible.

I/O: $n \leq 10^5$, $m \leq 2 \cdot 10^5$ con tres enteros por arista $\Rightarrow$ lectura total + tokenización (patrón 2), separando extremos y pesos con slices de paso 3. En vez de ordenar tuplas, se ordenan los ÍNDICES de las aristas con `key=C.__getitem__`, que ordena por peso sin crear tuplas y con la comparación en C. El costo total puede llegar a $\sim 10^{14}$, sin problema para los enteros de Python. Verificado contra el caso de ejemplo y contrastado con el algoritmo de Prim en 600 grafos aleatorios (conexos, desconectados, $n = 1$ y pesos pequeños con muchos empates), sin discrepancias. Con $n = 10^5$ y $m = 2 \cdot 10^5$ tarda entre ~ 0.15 s y ~ 0.24 s: grafo aleatorio conexo, un árbol formado

por las aristas más caras (el recorrido no puede cortarse temprano) y un grafo desconectado (hay que revisar todas las aristas para responder IMPOSSIBLE); el límite es de 1 segundo.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])
    A = list(map(int, data[2:2 + 3*m:3]))
    B = list(map(int, data[3:3 + 3*m:3]))
    C = list(map(int, data[4:4 + 3*m:3]))
    # se ordenan los índices de las aristas por peso, sin
    # crear tuplas
    orden = sorted(range(m), key=C.__getitem__)
    padre = list(range(n + 1))
    total = 0
    usadas = 0
    for e in orden:
        x = A[e]
        while padre[x] != x:          # find con path
            halving
            padre[x] = padre[padre[x]]
            x = padre[x]
        y = B[e]
        while padre[y] != y:
            padre[y] = padre[padre[y]]
            y = padre[y]
        if x != y:                   # una dos
            componentes distintas: arista segura
            padre[x] = y
            total += C[e]
            usadas += 1
            if usadas == n - 1:      # arbol completo:
                el resto sobra
                break
        # si x == y, la arista cerraria un ciclo y se
        # descarta
    sys.stdout.write((str(total) if usadas == n - 1 else
        "IMPOSSIBLE") + "\n")

solve()
```

4.14 Puentes y Árbol de Componentes 2-Arista-Conexas (Diámetro en una Pasada)

Preguntas guía:

- ¿Me piden aristas cuya eliminación desconecta el grafo (puentes), o cuántas aristas son obligatorias para ir de un nodo a otro?
- ¿El problema se simplifica si contraigo cada componente 2-arista-conexa a un solo nodo, quedando un árbol cuyas aristas son exactamente los puentes?
- ¿Después de esa contracción debo calcular algo típico de árboles, como el diámetro?
- ¿ n y m llegan a $3 \cdot 10^5$, de modo que en Python necesito un DFS sin recursión y evitar construir el árbol de puentes explícitamente?

Utilidad: Encuentra los puentes de un grafo no dirigido con el low-link de Tarjan y resume su estructura: al contraer cada componente 2-arista-conexa (conjunto de nodos que siguen conectados aunque se quite cualquier arista) el

grafo se vuelve un árbol, el ÁRBOL DE PUENTES. Las aristas obligatorias entre dos nodos son exactamente los puentes del camino entre sus componentes en ese árbol, lo que convierte preguntas de "cuántas aristas críticas" en preguntas sobre caminos en un árbol. Sirve para robustez de redes, aristas críticas, y para cualquier problema que se resuelva primero condensando por puentes.

Complejidad: $\mathcal{O}(n + m)$: un DFS y una pasada en orden inverso sobre los nodos, revisando cada lista de adyacencia una vez.

Fundamento teórico: En un DFS de un grafo no dirigido, toda arista que no pertenece al árbol DFS une un nodo con uno de sus ancestros (arista de retroceso). Sea $\text{tin}[u]$ el tiempo de entrada y $\text{low}[u]$ el menor tiempo de entrada alcanzable desde el subárbol de u usando a lo sumo una arista de retroceso. La arista del árbol entre p y su hijo u es un puente si y solo si $\text{low}[u] > \text{tin}[p]$: ninguna arista sale del subárbol de u hacia p o más arriba, así que quitarla lo desconecta. El low se calcula recorriendo los nodos en orden de entrada INVERSO: cuando se procesa u , todos sus hijos ya fueron procesados y le pasaron su low ; basta combinarlo con el menor tin entre sus vecinos, EXCLUYENDO al padre (la arista por la que se llegó no cuenta como retroceso; el enunciado garantiza que no hay aristas repetidas). Para calcular ese mínimo en C con $\min(\text{map}(\dots))$, el tin del padre se reemplaza temporalmente por infinito. La otra observación clave evita construir el árbol de puentes: en el árbol DFS, las componentes 2-arista-conexas son exactamente los pedazos que quedan al cortar los puentes, porque toda arista de retroceso cubre el camino del árbol entre sus extremos y por lo tanto ninguno de esos tramos es puente. Entonces el árbol de puentes es el árbol DFS con las aristas no-puente CONTRAÍDAS, y contraer aristas de peso 0 no cambia la longitud de ningún camino. Basta asignar peso 1 a los puentes y 0 al resto de las aristas del árbol DFS y calcular el diámetro ponderado, en la misma pasada inversa: $\text{baja}[u]$ es el camino más pesado que baja desde u , y en cada padre se combinan las dos mejores ramas. Por qué la respuesta es el diámetro: para s y t fijos, las aristas obligatorias son los puentes del único camino entre sus componentes en el árbol de puentes, y maximizar sobre s y t es buscar el camino más largo de ese árbol. [10]

Ejercicio aplicado

(Codeforces 1000E — We Need More Bosses) Dado un grafo conexo de n lugares y m pasajes, elegir s y t para maximizar la cantidad de pasajes por los que es obligatorio pasar para ir de s a t .

I/O: $n, m \leq 3 \cdot 10^5$ con dos enteros por arista $\Rightarrow$ lectura total + tokenización (patrón 2), separando extremos con slices de paso 2. El DFS es iterativo con una pila de iteradores de vecinos, porque la profundidad llega a $3 \cdot 10^5$. Verificado contra los dos casos de ejemplo y contrastado con una fuerza bruta que aplica literalmente la definición del enunciado (para cada par s, t y cada arista, comprobar si quitarla los desconecta) en 700 grafos conexos aleatorios,

incluyendo árboles, ciclos y mezclas, sin discrepancias. Con $n = m = 3 \cdot 10^5$ tarda entre $\sim 0.42s$ y $\sim 0.74s$ según la topología (camino donde todo es puente y el DFS alcanza profundidad $3 \cdot 10^5$, un único ciclo sin puentes, árbol aleatorio, cadena de triángulos), frente a un límite de 2 segundos; memoria pico ~ 113 MB de 256 MB.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])
    adj = [[] for _ in range(n + 1)]
    for a, b in zip(map(int, data[2:2 + 2*m:2]), map(int, data[3:3 + 2*m:2])):
        adj[a].append(b)
        adj[b].append(a)

    # ---- DFS iterativo: orden de entrada (preorden),
    # tin y padre ----
    tin = [0]*(n + 1)
    padre = [0]*(n + 1)
    orden = [1]
    tin[1] = 1
    t = 1
    nodos = [1]
    its = iter(adj[1])
    while its:
        for v in its[-1]:
            # el iterador
            recuerda por donde iba
            if not tin[v]:
                t += 1
                tin[v] = t
                padre[v] = nodos[-1]
                orden.append(v)
                nodos.append(v)
                its.append(iter(adj[v]))
                break
            else:
                its.pop()
                nodos.pop()

    # ---- pasada inversa: low, puentes y diametro con
    # pesos 0/1 ----
    low = tin[:n]
    baja = [0]*(n + 1)
    # arista-puentes en
    # el camino mas pesado que baja desde u
    mejor = 0
    INF = n + 5
    for u in reversed(orden):
        # los hijos se
        # procesan antes que el padre
        p = padre[u]
        if not p:
            continue
        # menor tin entre los vecinos de u SIN contar al
        # padre:
        # se oculta tin[p] un instante para que min(map
        # (...)) corra en C
        tp = tin[p]
        tin[p] = INF
        d = min(map(tin.__getitem__, adj[u]))
        tin[p] = tp
        lu = low[u]
        # ya trae el minimo
        de sus hijos
        if d < lu:
            lu = d
        if lu < low[p]:
            low[p] = lu
        # (p, u) es puente si el subarbol de u no llega a
        # ni mas arriba
        cand = baja[u] + (1 if lu > tp else 0)
```

```
# diametro: se combinan las dos mejores ramas que
# bajan desde p
if baja[p] + cand > mejor:
    mejor = baja[p] + cand
if cand > baja[p]:
    baja[p] = cand
print(mejor)

solve()
```

4.15 Orden Topológico (Kahn) y DP sobre DAG con Reconstrucción

Preguntas guía:

- ¿El grafo es dirigido y SIN ciclos (DAG), o debo detectar si tiene ciclos?
- ¿Me piden el camino más largo (o más corto, o la cantidad de caminos) entre dos nodos de un DAG?
- ¿El valor de cada nodo depende solo de los valores de sus predecesores, de modo que basta procesarlos en un orden donde todo predecesor va antes?
- ¿Además del valor óptimo debo imprimir el camino, guardando de qué nodo vino cada mejora?

Utilidad: El orden topológico ordena los nodos de un DAG de forma que toda arista va de un nodo anterior a uno posterior. Recorriendo los nodos en ese orden, cuando se llega a un nodo todos sus predecesores ya tienen su valor definitivo, así que cualquier DP que dependa solo de ellos se calcula en una pasada: camino más largo o más corto, cantidad de caminos, tareas con dependencias (tiempo mínimo de finalización), planificación de cursos. Kahn además detecta ciclos: si no logra ordenar todos los nodos, el grafo tiene uno.

Complejidad: $\mathcal{O}(n + m)$: cada nodo entra una vez a la cola y cada arista se revisa una vez.

Fundamento teórico: El algoritmo de Kahn mantiene el grado de entrada de cada nodo y una cola con los que tienen grado 0, es decir, sin predecesores pendientes. Al procesar un nodo, se "borran" sus aristas salientes restando 1 al grado de cada sucesor, y los que llegan a 0 entran a la cola. Un nodo entra a la cola exactamente cuando todos sus predecesores fueron procesados, así que el orden de la cola es topológico. Si el grafo tiene un ciclo, ningún nodo del ciclo llega nunca a grado 0 (cada uno espera al anterior), y la cola termina con menos de n nodos: esa es la detección de ciclos. Sobre ese orden, el DP del camino más largo es $dp[v] = \max(dp[u] + 1)$ sobre las aristas $u \rightarrow v$, y es correcto porque al procesar u su valor ya es definitivo. En este problema los caminos deben EMPEZAR en la ciudad 1, así que $dp[v] = 0$ significa "inalcanzable desde 1" y solo se propagan valores desde nodos con $dp > 0$; aun así Kahn tiene que recorrer todo el grafo, porque los nodos inalcanzables también descuentan grados de entrada, y sin ellos otros nodos nunca llegarían a 0. La ciudad 1 puede tener aristas entrantes desde nodos inalcanzables y por eso no estar al principio de la cola, pero ningún nodo procesado antes que ella es alcanzable desde ella en un DAG, así que su valor inicial 1 nunca

se pierde. Para reconstruir, cada vez que $dp[v]$ mejora se guarda el predecesor responsable, y desde el destino se sigue esa cadena hasta el origen. [12]

Ejercicio aplicado

(CSES 1680 — Longest Flight Route) Dado un DAG de n ciudades y m vuelos dirigidos, hallar la ruta de la ciudad 1 a la ciudad n que visita la mayor cantidad de ciudades e imprimirla, o informar que no existe.

I/O: $n \leq 10^5$, $m \leq 2 \cdot 10^5$ con dos enteros por arista $\Rightarrow$ lectura total + tokenización (patrón 2), separando extremos con slices de paso 2. Como se acepta cualquier ruta óptima, la verificación comprueba que la ruta empiece en 1, termine en n , use solo vuelos existentes y tenga la longitud máxima, calculada enumerando TODAS las rutas: se hizo en 600 DAGs aleatorios, incluyendo la ciudad 1 con aristas entrantes, n inalcanzable y varias rutas óptimas, sin fallos. Con $n = 10^5$ y $m = 2 \cdot 10^5$ tarda entre $\sim 0.08s$ y $\sim 0.19s$: una ruta de 10^5 ciudades, DAGs aleatorios con y sin camino a n (en uno de ellos la respuesta IMPOSSIBLE se confirmó con un recorrido independiente), y n inalcanzable; el límite es de 1 segundo. El toposort genérico de los extras se verificó contra un detector de ciclos por DFS en 2000 grafos con y sin ciclos (incluyendo lazos), comprobando además que cada orden devuelto respete todas las aristas, y tarda $\sim 0.04s$ con 10^5 nodos.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])
    adj = [[] for _ in range(n + 1)]
    grado = [0]*(n + 1) # grado de entrada
    for a, b in zip(map(int, data[2:2 + 2*m:2]), map(int, data[3:3 + 2*m:2])):
        adj[a].append(b)
        grado[b] += 1
    dp = [0]*(n + 1) # ciudades en la mejor ruta desde 1; 0 = inalcanzable
    dp[1] = 1
    padre = [0]*(n + 1)
    cola = [v for v in range(1, n + 1) if grado[v] == 0]
    for u in cola: # la lista crece mientras se recorre: orden topologico
        du = dp[u]
        if du: # alcanzable
            desde 1: propaga el DP
            du1 = du + 1
            for v in adj[u]:
                if du1 > dp[v]:
                    dp[v] = du1
                    padre[v] = u # de donde vino
            la mejora, para reconstruir
            grado[v] -= 1
            if not grado[v]:
                cola.append(v)
        else: # inalcanzable:
            igual descuenta grados,
            for v in adj[u]: # o sus
                grado[v] -= 1
                if not grado[v]:
```

```
                cola.append(v)
    if not dp[n]:
        sys.stdout.write("IMPOSSIBLE\n"); return
    camino = []
    v = n
    while v: # padre[1] = 0
        corta la cadena
        camino.append(v)
        v = padre[v]
    camino.reverse()
    sys.stdout.write("%d\n%s\n" % (len(camino), " ".join(map(str, camino))))

solve()
```

Extras: orden topológico genérico con detección de ciclos — la parte reutilizable de Kahn, separada del DP. Devuelve el orden, o None si el grafo tiene un ciclo; cualquier DP sobre el DAG se hace luego recorriendo orden (o reversed(orden) si el valor de un nodo depende de sus sucesores).

```
def toposort(n, adj):
    grado = [0]*(n + 1)
    for u in range(1, n + 1):
        for v in adj[u]:
            grado[v] += 1
    orden = [v for v in range(1, n + 1) if grado[v] == 0]
    for u in orden: # la lista crece mientras se recorre
        for v in adj[u]:
            grado[v] -= 1
            if not grado[v]:
                orden.append(v)
    # si hay un ciclo, sus nodos nunca llegan a grado 0 y quedan afuera
    return orden if len(orden) == n else None
```

4.16 Detección de Ciclo Negativo (Bellman-Ford + Ciclos en el Grafo de Padres)

Preguntas guía:

- ¿El grafo tiene pesos negativos y me piden detectar (e imprimir) un ciclo de peso total negativo?
- ¿Me sirve un origen virtual conectado a todos los nodos con peso 0, para encontrar ciclos negativos en cualquier parte del grafo y no solo los alcanzables desde un nodo?
- ¿Los límites son del orden de $n \cdot m \sim 10^7$, de modo que en Python necesito cortar temprano en vez de hacer siempre las n rondas?
- ¿Debo reconstruir el ciclo en orden, no solo decir que existe?

Utilidad: Bellman-Ford calcula caminos mínimos con pesos negativos relajando todas las aristas en rondas, y detecta ciclos negativos: si después de $n - 1$ rondas algo sigue mejorando, hay uno. Es la herramienta para grafos con pesos negativos (donde Dijkstra falla), para detectar arbitraje (ciclos de ganancia en cambio de monedas, usando $-\log$ de las tasas), y para sistemas de restricciones de diferencias $x_b - x_a \leq c$, que son factibles exactamente cuando el grafo asociado no tiene ciclos negativos. La

variante de esta entrada agrega una revisión periódica del grafo de padres, que en Python hace la diferencia entre terminar en milisegundos o recorrer todas las rondas.

Complejidad: $\mathcal{O}(nm)$ en el peor caso (hasta n rondas de m aristas), más $\mathcal{O}(n)$ por cada revisión del grafo de padres, que se hace cada 32 rondas.

Fundamento teórico: Inicializar todas las distancias en 0 equivale a agregar un origen virtual con aristas de peso 0 hacia cada nodo: así cualquier ciclo negativo es alcanzable y se detecta. Tras la ronda r , la distancia de cada nodo es a lo sumo el peso del mejor camino de r aristas desde el origen virtual. Sin ciclos negativos todo camino mínimo es simple y tiene a lo sumo n aristas (contando la virtual), así que tras $n-1$ rondas reales nada puede mejorar; si en la ronda n algo mejora, hay un ciclo negativo. Esperar a la ronda n es lo que en Python resulta caro, y el atajo es el grafo de padres: cada nodo apunta al nodo desde el que obtuvo su distancia actual. Un lema clásico (CLRS, capítulo 24) garantiza que CUALQUIER ciclo en ese grafo tiene peso negativo: al fijar el último padre del ciclo, su distancia bajó estrictamente, y sumando las desigualdades $\text{dist}[v] \geq \text{dist}[\text{padre}(v)] + w$ alrededor del ciclo se obtiene peso total < 0 . Además, si hay un ciclo negativo, el grafo de padres termina conteniendo un ciclo. Como cada nodo tiene a lo sumo un padre, el grafo de padres es un grafo funcional y se revisa en $\mathcal{O}(n)$: desde cada nodo no visitado se sigue la cadena de padres marcando con el número del recorrido actual; si se llega a un nodo marcado en ESTE mismo recorrido, se encontró un ciclo, y ese nodo pertenece a él. Revisando cada 32 rondas, un ciclo negativo se detecta apenas aparece en los punteros, en vez de esperar a la ronda n . Si no hay ciclos negativos, se corta cuando una ronda no mejora nada. Para imprimir el ciclo, se parte de un nodo que está en él y se siguen los padres hasta volver; eso lo recorre al revés, así que se invierte. [15]

Ejercicio aplicado

(CSES 1197 — Cycle Finding) Dado un grafo dirigido de n nodos y m aristas con pesos que pueden ser negativos, decidir si contiene un ciclo de peso negativo y, si lo tiene, imprimir sus nodos en orden.

I/O: $n \leq 2500$, $m \leq 5000$ con tres enteros por arista $\Rightarrow$ lectura total + tokenización (patrón 2), armando la lista de aristas con `zip` de tres slices de paso 3. Como se acepta cualquier ciclo negativo, la verificación comprueba que la respuesta sea YES exactamente cuando existe un ciclo negativo (según Floyd-Warshall) y que el ciclo impreso cierre, use aristas existentes y suma negativo: se hizo en 800 grafos aleatorios (600 de hasta 9 nodos y 200 de 30 a 60 nodos, donde la revisión periódica sí se activa), con lazos negativos, aristas repetidas y varios ciclos, sin fallos. Con $n = 2500$ y $m = 5000$ se midieron siete grafos adversos: los que tienen ciclo negativo (de 3 nodos escondido, o de 2500 nodos) terminan en ~ 0.02 s gracias a la revisión de padres, y el peor caso es una cadena negativa con las aristas listadas al revés, sin ciclo, que obliga a las $n-1$

rondas: ~ 0.53 s frente a un límite de 1 segundo. Se comparó con SPFA (Bellman-Ford con cola), que es más rápido en la mayoría de los casos pero llega a ~ 1.00 s en una cadena numerada al revés, conocida por forzar a SPFA a mejorar cada nodo de a una unidad por vuelta de la cola; por eso se eligió Bellman-Ford por rondas.

Código solución

```
import sys

def ciclo_en_padres(n, par):
    # grafo funcional (cada nodo tiene a lo sumo un padre)
    # devuelve un nodo que esta en un ciclo, o 0 si no hay ciclos
    estado = [0]*(n + 1) # 0 = no visto, -1 = terminado
    for s in range(1, n + 1):
        if estado[s]:
            continue
        x = s
        while x and not estado[x]:
            estado[x] = s
            x = par[x]
        if x and estado[x] == s: # volvio a un nodo de ESTE recorrido: ciclo
            return x
        x = s
        while x and estado[x] == s: # el recorrido no cerro ciclo: se marca terminado
            estado[x] = -1
            x = par[x]
    return 0

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])
    E = list(zip(map(int, data[2:2+3*m:3]), map(int, data[3:3+3*m:3]), map(int, data[4:4+3*m:3])))
    dist = [0]*(n + 1) # todo en 0: origen virtual conectado a todos
    par = [0]*(n + 1)
    x = 0
    for ronda in range(1, n + 1):
        cambio = 0
        for a, b, c in E:
            nd = dist[a] + c
            if nd < dist[b]:
                dist[b] = nd; par[b] = a; cambio = b
        if not cambio: # nada mejoro: no hay ciclo negativo
            break
        # todo ciclo en el grafo de padres es negativo: se busca cada 32 rondas
        if ronda == n or ronda % 32 == 0:
            x = ciclo_en_padres(n, par)
            if x or ronda == n:
                if not x: # respaldo: mejora en la ronda n => hay ciclo;
                    x = cambio # n pasos hacia atras caen dentro de el
                    for _ in range(n): x = par[x]
                break
    if not x:
        sys.stdout.write("NO\n"); return
    ciclo = [x]; y = par[x]
    while y != x: # los padres recorren el ciclo al reves
        ciclo.append(y); y = par[y]
    ciclo.append(x); ciclo.reverse()
    sys.stdout.write("YES\n" + " ".join(map(str, ciclo)) + "\n")
```

```
solve()
```

4.17 Distancias en Árbol con LCA por RMQ sobre el Preorden (Sparse Table)

Preguntas guía:

- ¿Me piden distancias (o el ancestro común más bajo, LCA) entre muchos pares de nodos de un árbol FIJO?
- ¿Puedo expresar la respuesta con profundidades: $\text{dist}(u, v) = \text{prof}[u] + \text{prof}[v] - 2\text{prof}[\text{LCA}]$?
- ¿Hay del orden de 10^5 consultas y en Python necesito que cada una cueste $\mathcal{O}(1)$ en vez de $\mathcal{O}(\log n)$ pasos interpretados?
- ¿Ya tengo una sparse table de mínimo en rango que puedo reutilizar?

Utilidad: Reduce el cálculo del LCA a un mínimo en rango sobre el arreglo de profundidades en preorden DFS, que una sparse table responde en tiempo constante. Con la profundidad del LCA se obtiene la distancia entre dos nodos, y con variantes menores se responde si un nodo es ancestro de otro o la longitud de caminos en árboles con pesos (usando la distancia ponderada desde la raíz en lugar de la profundidad). A diferencia del tour de Euler clásico, este arreglo mide n y no $2n - 1$, lo que reduce a la mitad el tiempo y la memoria de la tabla.

Complejidad: $\mathcal{O}(n \log n)$ para construir la sparse table, $\mathcal{O}(n)$ para el recorrido, y $\mathcal{O}(1)$ por consulta.

Fundamento teórico: En un preorden DFS, el subárbol de cada nodo ocupa un bloque contiguo de posiciones que empieza en el propio nodo. Sean $u \neq v$ con $\text{tin}[u] < \text{tin}[v]$, y w su LCA. Se afirma que la menor profundidad entre las posiciones $(\text{tin}[u], \text{tin}[v])$ es exactamente $\text{prof}[w] + 1$. Primero, todos esos nodos pertenecen al subárbol de w : el bloque de w contiene a u y a v , y por contigüidad también todo lo que queda entre ellos. Segundo, w mismo no está en el rango: si $w = u$, el rango empieza justo después de u ; si $w \neq u$, entonces $\text{tin}[w] < \text{tin}[u]$. Por lo tanto toda profundidad en el rango es al menos $\text{prof}[w] + 1$. Tercero, ese valor se alcanza: el hijo c de w que contiene a v cumple $\text{tin}[u] < \text{tin}[c] \leq \text{tin}[v]$ (si $w = u$, porque c está en el subárbol de u y no es u ; si no, porque el subárbol que contiene a u se recorre antes que el de c , dado que $\text{tin}[u] < \text{tin}[v]$), y su profundidad es $\text{prof}[w] + 1$. Con eso, $\text{dist}(u, v) = \text{prof}[u] + \text{prof}[v] - 2(\text{mín} - 1)$, y ni siquiera hace falta saber CUÁL nodo es el LCA, solo su profundidad, así que la tabla guarda enteros y no pares. Un recorrido con pila que al sacar un nodo apila todos sus hijos produce un preorden válido de un DFS (los hijos quedan en orden inverso, lo cual no importa), porque la pila termina el subárbol del último hijo apilado antes de pasar al siguiente: los subárboles quedan contiguos. [4]

Ejercicio aplicado

(CSES 1135 — Distance Queries) Dado un árbol de n nodos, responder q consultas de la distancia (cantidad de aristas) entre dos nodos.

I/O: $n, q \leq 2 \cdot 10^5$ con dos enteros por arista y por consulta $\Rightarrow$ lectura total + tokenización (patrón 2), separando aristas y consultas con slices de paso 2. Verificado contra el caso de ejemplo y contrastado con distancias por BFS desde cada consulta en 400 árboles aleatorios, caminos, estrellas y orugas, reenumerados al azar, incluyendo $n = 1$ y consultas con $a = b$, sin discrepancias. Con $n = q = 2 \cdot 10^5$ tarda entre $\sim 0.45\text{s}$ y $\sim 0.65\text{s}$ según la forma del árbol (aleatorio, estrella, binario completo, y un camino de profundidad $2 \cdot 10^5$, el más lento). Como comparación, binary lifting, verificado igual de correcto, tarda entre $\sim 0.75\text{s}$ y $\sim 1.40\text{s}$ en los mismos árboles, por los $\mathcal{O}(\log n)$ pasos interpretados de cada consulta: con límite de 1 segundo, esa versión no es segura en Python.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); q = int(data[1])
    adj = [[] for _ in range(n + 1)]
    fin = 2 + 2*(n - 1)  # fin de
                        # las aristas, inicio de las consultas
    for a, b in zip(map(int, data[2:fin:2]), map(int,
        data[3:fin:2])):
        adj[a].append(b)
        adj[b].append(a)
    # ---- preorden DFS con pila: los subárboles quedan
    # contiguos ----
    prof = [0]*(n + 1)
    visto = bytearray(n + 1)
    visto[1] = 1
    orden = []
    pila = [1]
    while pila:
        u = pila.pop()
        orden.append(u)
        pu = prof[u] + 1
        for v in adj[u]:
            if not visto[v]:
                visto[v] = 1
                prof[v] = pu
                pila.append(v)
    tin = [0]*(n + 1)
    for i, v in enumerate(orden):
        tin[v] = i
    # ---- sparse table de mínimo sobre las profundidades
    # en preorden ----
    fila = [prof[v] for v in orden]
    st = [fila]
    j = 1
    while (1 << j) <= n:
        mitad = 1 << (j - 1)
        prev = st[-1]
        st.append([x if x < y else y for x, y in zip(prev
            [:n - (1 << j) + 1], prev[mitad:])]
        j += 1
    out = []
    for a, b in zip(map(int, data[fin:2]), map(int, data
        [fin + 1:2])):
        if a == b:
            out.append(0); continue
        ta = tin[a]; tb = tin[b]
        if ta > tb: ta, tb = tb, ta
```

```

# minimo de profundidad en (ta, tb) = prof(LCA) +
1
1 = ta + 1; r = tb + 1 # rango
semiabierto [l, r)
k = (r - l).bit_length() - 1
t = st[k]
x = t[l]; y = t[r - (1 << k)]
mn = x if x < y else y
out.append(prof[a] + prof[b] - 2*(mn - 1))
sys.stdout.write("\n".join(map(str, out)) + "\n")

solve()

```

4.18 Corte Mínimo de Aristas con Flujo Máximo (Dinic)

Preguntas guía:

- ¿Me piden la mínima cantidad (o costo) de aristas a eliminar para separar dos nodos s y t ?
- ¿Además de cuántas, debo imprimir CUÁLES aristas forman el corte?
- ¿El problema se puede ver como "cuántos caminos que no comparten aristas hay de s a t " (teorema de Menger)?
- ¿Las aristas son no dirigidas, de modo que necesito representarlas en una red de flujo dirigida?

Utilidad: Calcula el flujo máximo de s a t en una red con capacidades, y a partir de él el corte mínimo: el conjunto de aristas de menor capacidad total cuya eliminación desconecta t de s . Con capacidad 1 por arista, el valor es la mínima cantidad de aristas a cortar y a la vez la máxima cantidad de caminos disjuntos en aristas. El flujo es la base de emparejamientos bipartitos, asignaciones con capacidades, cortes de imágenes y problemas de selección de proyectos.

Complejidad: $\mathcal{O}(V^2E)$ en general para Dinic; con capacidades unitarias, $\mathcal{O}(E\sqrt{E})$. Aquí el flujo está acotado por el grado de s ($\leq n - 1$) y cada camino aumentante cuesta $\mathcal{O}(n + m)$.

Fundamento teórico: El teorema de flujo máximo y corte mínimo dice que el valor del flujo máximo de s a t es igual a la capacidad del corte mínimo. Dinic lo calcula por fases: en cada fase, un BFS desde s sobre el grafo RESIDUAL (arcos con capacidad sobrante) asigna niveles, y luego se envía flujo solo por arcos que suben exactamente un nivel, usando un puntero por nodo que avanza sobre los arcos que ya no sirven para no revisarlos de nuevo en la misma fase. Enviar flujo por un arco reduce su capacidad y aumenta la del arco reverso, lo que permite "deshacer" decisiones en caminos futuros. Guardando los arcos en arreglos de a pares, el reverso del arco e es simplemente $e \oplus 1$. Una arista NO dirigida de capacidad c se representa con dos arcos $a \rightarrow b$ y $b \rightarrow a$, ambos de capacidad c , que son el reverso uno del otro: empujar flujo en un sentido libera capacidad en el otro, lo que es exactamente el comportamiento de una arista que se puede usar en cualquier dirección. Cuando ya no hay camino de s a t en el residual, el flujo es máximo, y el corte se

EXTRAE así: sea S el conjunto de nodos alcanzables desde s en el residual; toda arista original con un extremo en S y el otro fuera está saturada (si no, el otro extremo sería alcanzable), y estas aristas forman un corte cuya capacidad es exactamente el flujo, así que es mínimo. En la implementación se recorren las aristas originales y se imprimen las que tienen exactamente un extremo alcanzable. [16]

Ejercicio aplicado

(CSES 1695 — Police Chase) Dado un grafo no dirigido de n cruces y m calles, hallar la mínima cantidad de calles a cerrar para que no haya ruta entre el cruce 1 y el cruce n , e imprimir cuáles.

I/O: $n \leq 500$, $m \leq 1000$ con dos enteros por calle $\Rightarrow$ lectura total + tokenización (patrón 2), separando extremos con slices de paso 2. La DFS de Dinic es recursiva: la profundidad está acotada por la longitud de un camino, a lo sumo $n = 500$, así que basta subir el límite de recursión. Como se acepta cualquier corte mínimo, la verificación comprueba que la cantidad impresa sea el mínimo real, calculado probando TODOS los subconjuntos de calles, que las calles impresas existan, y que al cerrarlas 1 y n queden desconectados: se hizo en 400 grafos aleatorios, incluyendo 1 y n ya desconectados, calle directa entre 1 y n y varios cortes mínimos posibles, sin fallos. Con $n = 500$ y $m = 1000$ tarda $\sim 0.01s$ tanto con flujo alto (1 y n conectados a casi todos los nodos, corte de 498 calles) como con caminos aumentantes largos y en un grafo aleatorio.

Código solución

```

import sys
sys.setrecursionlimit(10000)

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])
    A = list(map(int, data[2:2 + 2*m:2])); B = list(map(
        int, data[3:3 + 2*m:2]))
    # arcos en arreglos: el arco e y su reverso e ^ 1 van
    # de a pares.
    # arista no dirigida = dos arcos de capacidad 1 que
    # son reverso uno del otro
    hacia = []; cap = []
    ady = [[] for _ in range(n + 1)]
    for a, b in zip(A, B):
        ady[a].append(len(hacia)); hacia.append(b); cap.
        append(1)
        ady[b].append(len(hacia)); hacia.append(a); cap.
        append(1)
    s, t = 1, n
    flujo = 0
    while True:
        # ---- BFS: niveles en el grafo residual ----
        nivel = [-1]*(n + 1)
        nivel[s] = 0
        cola = [s]
        for u in cola:
            for e in ady[u]:
                if cap[e] and nivel[hacia[e]] < 0:
                    nivel[hacia[e]] = nivel[u] + 1
                    cola.append(hacia[e])
            if nivel[t] < 0:
                break
        # no hay camino
        : flujo maximo
        # ---- DFS: caminos que suben de a un nivel ----

```

```

ptr = [0]*(n + 1) # arcos ya
descartados en esta fase
def dfs(u):
    if u == t:
        return True
    lista = ady[u]
    while ptr[u] < len(lista):
        e = lista[ptr[u]]
        v = hacia[e]
        if cap[e] and nivel[v] == nivel[u] + 1
and dfs(v):
            cap[e] -= 1
            cap[e ^ 1] += 1 # el reverso
gana capacidad
            return True
        ptr[u] += 1
    return False
while dfs(s):
    flujo += 1
# ---- corte: lo alcanzable desde s en el residual es
# un lado ----
alcanza = [False]*(n + 1)
alcanza[s] = True
pila = [s]
while pila:
    u = pila.pop()
    for e in ady[u]:
        if cap[e] and not alcanza[hacia[e]]:
            alcanza[hacia[e]] = True
            pila.append(hacia[e])
# calles con un extremo de cada lado: estan saturadas
# y suman el flujo
corte = [f"{a} {b}" for a, b in zip(A, B) if alcanza[
a] != alcanza[b]]
sys.stdout.write(str(len(corte)) + "\n" + "\n".join(
corte) + ("\n" if corte else ""))
solve()

```

4.19 2-SAT con Componentes Fuertemente Conexas (Kosaraju)

Preguntas guía:

- ¿Tengo variables booleanas y restricciones de la forma "a o b" con exactamente DOS literales cada una?
- ¿Las restricciones se pueden reescribir como "si pasa X, entonces debe pasar Y" (implicaciones), incluyendo exclusiones ("no ambas") y obligaciones ("x debe ser verdadera")?
- ¿Debo decidir si existe una asignación que las cumpla todas y, si existe, construir una?
- ¿Hay del orden de 10^5 variables y restricciones, de modo que probar asignaciones es imposible y el DFS debe ser iterativo?

Utilidad: Resuelve en tiempo lineal la satisfacibilidad de fórmulas en forma normal conjuntiva con dos literales por cláusula, que en general (con tres o más) es NP-completo. Aparece disfrazado en asignaciones con conflictos de a pares: elegir una de dos opciones por elemento sin que ciertas combinaciones choquen, orientar aristas, ubicar etiquetas sin solaparse, horarios con dos turnos posibles. Una restricción "exactamente uno de a, b" se expresa con dos cláusulas, y "x obligatoriamente verdadera" con la cláusula $(x \vee x)$.

Complejidad: $\mathcal{O}(n + m)$: el grafo tiene $2m$ nodos (dos literales por variable) y $2n$ aristas (dos implicaciones por cláusula), y Kosaraju lo recorre dos veces.

Fundamento teórico: La cláusula $(a \vee b)$ es equivalente a las dos implicaciones $\neg a \Rightarrow b$ y $\neg b \Rightarrow a$: si una parte falla, la otra tiene que valer. Se arma el GRAFO DE IMPLICACIONES con un nodo por literal y esas aristas; un camino de u a v significa que elegir u verdadero obliga a que v sea verdadero. La fórmula es insatisfacible si y solo si para alguna variable x los literales x y $\neg x$ están en la misma componente fuertemente conexa: en ese caso $x \Rightarrow \neg x$ y $\neg x \Rightarrow x$, y ningún valor de x sirve. Recíprocamente, si ninguna variable está en ese caso, una asignación válida se obtiene del orden topológico del grafo de componentes: a cada variable se le da el valor del literal cuya componente aparece MÁS TARDE en ese orden. La idea es que las implicaciones van "hacia adelante" en el orden topológico, así que eligiendo siempre el literal más avanzado nunca se activa una cadena que termine forzando lo contrario (y el grafo es simétrico: si hay camino $u \rightarrow v$ también lo hay $\neg v \rightarrow \neg u$, lo que garantiza la consistencia). Kosaraju calcula las componentes en dos pasadas: un DFS sobre el grafo que registra el orden de TERMINACIÓN de los nodos, y luego recorridos sobre el grafo TRANSPUESTO tomando los nodos por terminación decreciente. Cada recorrido de la segunda pasada descubre exactamente una componente, y las componentes salen numeradas en orden topológico del grafo original, que es justamente lo que pide la asignación: x es verdadera si $\text{comp}[x] > \text{comp}[\neg x]$. Con los literales codificados como $2x$ (positivo) y $2x + 1$ (negativo), la negación de cualquier literal es un XOR con 1. [17]

Ejercicio aplicado

(CSES 1684 — Giant Pizza) Hay n familiares y m ingredientes de pizza; cada familiar da dos deseos del tipo "el ingrediente x va" o "el ingrediente x no va". Elegir los ingredientes de modo que a cada familiar se le cumpla al menos un deseo, o informar que es imposible.

I/O: $n, m \leq 10^5$ con cuatro tokens por línea (signo, número, signo, número) $\Rightarrow$ lectura total + tokenización (patrón 2), separando los cuatro campos con slices de paso 4 y comparando el signo directamente como **bytes**. Ambas pasadas de Kosaraju son iterativas: la primera con una pila de iteradores de vecinos (necesita el orden real de terminación), la segunda con una pila simple (solo marca la componente, el orden no importa). Como se acepta cualquier pizza válida, la verificación comprueba que cada familiar tenga al menos un deseo cumplido y que IMPOSSIBLE salga exactamente cuando ninguna de las 2^m pizzas sirve: se hizo en 800 instancias con deseos repetidos, familiares que piden $+x$ y $-x$ a la vez, e instancias insatisfacibles, sin fallos. Con $n = m = 10^5$ tarda entre ~ 0.24 s y ~ 0.39 s: instancia aleatoria, una cadena de 10^5 implicaciones (el DFS alcanza esa profundidad) y la misma cadena cerrada en contradicción; el límite es de 1 segundo.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])
    N = 2*m                                     # literal +
    x = 2x, -x = 2x+1; negar = ^ 1
    g = [[] for _ in range(N)]
    gt = [[] for _ in range(N)]                # grafo
    transpuesto
    s1 = data[2::4]; x1 = data[3::4]; s2 = data[4::4]; x2
    = data[5::4]
    for i in range(n):
        a = 2*(int(x1[i]) - 1) + (s1[i] == b"-")
        b = 2*(int(x2[i]) - 1) + (s2[i] == b"-")
        # (a o b) == (no a => b) y (no b => a)
        g[a ^ 1].append(b); gt[b].append(a ^ 1)
        g[b ^ 1].append(a); gt[a].append(b ^ 1)
    # ---- pasada 1: orden de terminacion (DFS iterativo
    real) ----
    visto = bytearray(N)
    post = []
    for r in range(N):
        if visto[r]:
            continue
        visto[r] = 1
        nodos = [r]; its = [iter(g[r])]
        while its:
            for v in its[-1]:
                if not visto[v]:
                    visto[v] = 1
                    nodos.append(v); its.append(iter(g[v
            ]))
                break
        else:
            its.pop(); post.append(nodos.pop()) #
    termino: postorden
    # ---- pasada 2: transpuesto, por terminacion
    decreciente ----
    comp = [-1]*N
    c = 0                                       # las
    componentes salen en orden topologico
    for r in reversed(post):
        if comp[r] >= 0:
            continue
        comp[r] = c
        pila = [r]
        while pila:
            u = pila.pop()
            for v in gt[u]:
                if comp[v] < 0:
                    comp[v] = c
                    pila.append(v)
        c += 1
    res = []
    for x in range(m):
        ct = comp[2*x]; cf = comp[2*x + 1]
        if ct == cf:                           # x => no x
            y no x => x
            sys.stdout.write("IMPOSSIBLE\n"); return
        # se elige el literal cuya componente va mas
        tarde en el orden topologico
        res.append("+ " if ct > cf else "- ")
    sys.stdout.write(" ".join(res) + "\n")

solve()
```

5 Búsqueda, Greedy y Optimización

5.1 Búsqueda Binaria sobre la Respuesta + Two Pointers (k-ésima suma de pares)

Preguntas guía:

- ¿La respuesta es un número y existe una condición monótona (si X funciona, todo valor mayor/menor también funciona)?
- ¿Puedo verificar en $\mathcal{O}(n)$ o $\mathcal{O}(n \log n)$ si un valor candidato "cumple" la condición pedida?
- ¿Dentro de esa verificación necesito contar pares/elementos con una condición de suma sobre un arreglo ordenado? (ahí entra Two Pointers)

Utilidad: Cuando no es fácil calcular la respuesta directamente pero sí verificar si un candidato es válido, y esa validez es monótona, se "busca" la respuesta con búsqueda binaria sobre el rango de valores posibles. Two Pointers acelera el conteo en arreglos ordenados de $\mathcal{O}(n^2)$ a $\mathcal{O}(n)$.

Complejidad: Búsqueda binaria: $\mathcal{O}(\log(\text{rango}))$ iteraciones. Cada verificación con Two Pointers: $\mathcal{O}(n)$. Total: $\mathcal{O}(n \log(\text{rango}))$.

Fundamento teórico: La búsqueda binaria sobre la respuesta funciona porque el conjunto de valores "válidos" forma un intervalo continuo (monotonía), reduciendo el espacio de búsqueda a la mitad en cada paso. Two Pointers explota que, en un arreglo ordenado, mover un extremo cambia la condición de suma de forma predecible, evitando recorrer todas las parejas.

Ejercicio aplicado

Dados N precios de categorías de joyas, un cliente que es el K-ésimo en la fila debe comprar dos joyas de categorías distintas sin repetir una combinación ya usada por otro cliente. Hallar el mínimo dinero que garantiza poder comprar: la K-ésima suma más pequeña entre todos los pares de categorías distintas.

I/O: N no está acotado explícitamente pero puede ser grande $\rightarrow$ lectura total + tokenización (patrón 2) para el arreglo de precios.

Código solución

```
import sys

def count_pairs_leq(prices, n, limit):
    # cuenta cuantos pares cumplen arr[i] + arr[j] <= limit
    # usando dos punteros sobre un arreglo YA ORDENADO
    count = 0
    left, right = 0, n - 1
    while left < right:
        if prices[left] + prices[right] <= limit:
            count += right - left  # todos los pares
            con "left" fijo
            left += 1
        else:
            right -= 1
    return count

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
```

```
prices = list(map(int, data[2:2 + n]))
prices.sort()

low = prices[0] + prices[1]  # menor suma
                             posible
high = prices[n - 2] + prices[n - 1]  # mayor suma
                                     posible

while low < high:
    mid = low + (high - low) // 2
    if count_pairs_leq(prices, n, mid) >= k:
        high = mid  # mid cumple:
                    intenta algo menor
    else:
        low = mid + 1  # mid no cumple:
                      sube el limite

print(low)

solve()
```

5.2 Enumeración por Bitmask (bits perdidos de un mensaje)

Preguntas guía:

- ¿Hay un pequeño número ($\leq \sim 20$) de decisiones binarias desconocidas (bits, elecciones sí/no) y hace falta probar combinaciones?
- ¿El espacio total de combinaciones (2^k) es lo bastante chico para probarlas todas dentro del límite de tiempo?
- ¿Cada combinación se puede armar y evaluar rápido a partir de su representación binaria?

Utilidad: Representar y recorrer exhaustivamente todas las configuraciones posibles de un conjunto pequeño de decisiones binarias. Cada máscara de bits codifica una asignación distinta de valores, permitiendo generar candidatos de forma sistemática y verificar cuál satisface la condición del problema.

Complejidad: $\mathcal{O}(2^k \cdot C)$, con k = número de incógnitas binarias y C = costo de evaluar una combinación.

Fundamento teórico: Cualquier asignación de k bits desconocidos corresponde biunívocamente a un entero entre 0 y $2^k - 1$. Recorriendo ese rango y extrayendo el bit j con $(\text{mascara} \gg j) \& 1$, se reconstruye cada combinación posible sin necesidad de recursión ni backtracking explícito. Es la enumeración más directa de un espacio de búsqueda combinatorio pequeño.

Ejercicio aplicado

Un mensaje M' y un divisor N' se recibieron con hasta 16 bits en total marcados como perdidos (*). Reconstruir un mensaje M compatible (M completando los '*' de M') tal que exista un N (completando los '*' de N') donde N divide a M exactamente.

I/O: $|M'| \leq 500$ pero el total de '*' (en M' y N' combinados) es $\leq 16 \rightarrow$ lectura total con readline es más que suficiente; el costo real está en las 2^{16} combinaciones, no en la entrada.

Código solución

```
import sys
```

```

input = sys.stdin.readline

def generate(strings):
    pos = [] # posiciones donde existen bits desconocidos
    for i, c in enumerate(strings):
        if c == '?':
            pos.append(i)
    k = len(pos)

    ans = []
    for mask in range(1 << k): # cada mascara representa una asignacion distinta de los k bits perdidos
        cur = strings[:]
        for j in range(k):
            if (mask >> j) & 1:
                cur[pos[j]] = '1'
            else:
                cur[pos[j]] = '0'
        ans.append(''.join(cur))
    return ans

M = list(input().strip())
N = list(input().strip())

messages = generate(M)
divisors = generate(N)

for m in messages: # probar todas las combinaciones posibles
    value_m = int(m, 2)
    for n in divisors:
        value_n = int(n, 2)

        if value_n == 0:
            continue # la division entre cero no existe
        if value_m % value_n == 0: # la primera combinacion valida es respuesta
            print(m)
            sys.exit()

```

5.3 Búsqueda Binaria sobre la Respuesta: Maximizar el Mínimo (muralla con refuerzo)

Preguntas guía:

- ¿Piden maximizar un mínimo (o minimizar un máximo) tras aplicar UNA modificación elegida de un conjunto de opciones?
- ¿Puedo verificar rápido, para un valor objetivo fijo, si ALGUNA elección lo logra?
- ¿La factibilidad es monótona: si un objetivo H es alcanzable, cualquier objetivo menor también lo es?

Utilidad: Resolver problemas de "maximizar el mínimo" cuando la modificación disponible tiene una forma estructurada (aquí, un refuerzo en forma de escalera), buscando binariamente el mejor valor objetivo y verificando factibilidad con una condición cerrada en $\mathcal{O}(n)$ por candidato.

Complejidad: $\mathcal{O}(n \log(\max x + K))$.

Fundamento teórico: Para un objetivo H fijo, una posición i con $x_i < H$ tiene una *deficiencia* $d_i = H - x_i$.

Si se refuerza en la posición p , la posición i recibe bonificación $K - (p - i)$ solo si $p - K + 1 \leq i \leq p$; para que alcance H , se necesita $p \leq i + K - d_i$. Además, ninguna posición fuera del rango de bonificación puede ser deficiente, lo que obliga a $p \geq R$ (la posición deficiente más a la derecha). Así, el conjunto de posiciones p válidas es la intersección de intervalos $[i, i + K - d_i]$ sobre todas las posiciones deficientes i ; como estos intervalos tienen extremo izquierdo creciente en i , la intersección es no vacía si y solo si $R \leq K + \min_i (i + x_i) - H$ (tomando el mínimo solo entre las posiciones deficientes). Esta condición se evalúa en $\mathcal{O}(n)$, y como la factibilidad de H es monótona (un refuerzo que logra mínimo H automáticamente logra cualquier $H' < H$), se puede buscar binariamente el mayor H factible.

Ejercicio aplicado

Dada una muralla de N segmentos con alturas iniciales, y un refuerzo que agrega K bloques al segmento elegido, $K - 1$ al anterior, y así sucesivamente, hallar la mayor altura mínima posible tras aplicar el refuerzo en un único segmento.

I/O: $N \leq 10^5 \rightarrow$ lectura total con tokenización (patrón 2). Verificado contra fuerza bruta en 300 casos aleatorios pequeños.

Código solución

```

import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
    x = list(map(int, data[2:2 + n]))

    def feasible(h):
        # deficientes: posiciones (1-indexadas) por debajo de h
        r = -1 # posicion deficiente mas a la derecha
        best = None # min(i + x[i]) entre las deficientes (1-indexado)
        for i in range(n):
            if x[i] < h:
                pos = i + 1
                r = pos
                val = pos + x[i]
                if best is None or val < best:
                    best = val
        if r == -1:
            return True # nadie por debajo de h: cualquier refuerzo sirve
        # interseccion de rangos validos de p: [pos, pos+K-d] para cada deficiente
        # no vacia <=> r <= K + best - h
        return r <= k + best - h

    lo, hi = 1, max(x) + k
    while lo < hi:
        mid = (lo + hi + 1) // 2 # busca el mayor H factible
        if feasible(mid):
            lo = mid
        else:
            hi = mid - 1
    print(lo)

solve()

```

5.4 Huffman: Fusión Greedy con Heap (teñir telas)

Preguntas guía:

- ¿Debo partir repetidamente un grupo en dos subgrupos (o fusionar repetidamente dos grupos en uno) hasta llegar a elementos individuales, pagando en cada paso el peso total del grupo involucrado?
- ¿El costo total equivale a $\sum w_i \cdot d_i$, donde d_i es la profundidad del elemento i en el árbol binario resultante, y me piden minimizarlo?
- ¿Los elementos pueden reordenarse libremente, de modo que cualquier estructura de árbol binario es alcanzable?

Utilidad: Construye el árbol binario que minimiza la suma ponderada de profundidades $\sum w_i d_i$, equivalente a minimizar el costo acumulado de fusionar (o partir) grupos donde cada operación cuesta el peso total del grupo resultante. Es la solución estándar para compresión por códigos de longitud variable, y aparece disfrazado en muchos problemas de "combinar/dividir pagando la suma" (unir cuerdas, mezclar archivos, partir lotes).

Complejidad: $\mathcal{O}(n \log n)$ por caso de prueba, domina por las $n - 1$ operaciones sobre el heap binario.

Fundamento teórico: El proceso de dividir repetidamente un conjunto en dos partes hasta llegar a singletons define un árbol binario cuyas hojas son los elementos. Cada división paga la suma de los pesos del grupo dividido, es decir, la suma de las hojas de ese subárbol; como un elemento i es pagado una vez por cada nodo interno que lo tiene por debajo, su contribución total es $w_i \cdot d_i$ con d_i su profundidad. Por lo tanto el costo total es $\sum_i w_i d_i$ y el problema es exactamente el de Huffman, que se resuelve construyendo el árbol de abajo hacia arriba: en cada paso se fusionan los DOS pesos menores. La optimalidad descansa en dos lemas. (i) *Lema de intercambio:* en algún árbol óptimo, los dos pesos menores $a \leq b$ pueden colocarse como hermanos en el nivel más profundo — si no lo estuvieran, intercambiarlos con las hojas más profundas no aumenta el costo, pues mover un peso menor hacia mayor profundidad y uno mayor hacia menor profundidad tiene delta $(w_{\text{mayor}} - w_{\text{menor}})(d_{\text{menor}} - d_{\text{mayor}}) \leq 0$. (ii) *Subestructura óptima:* reemplazar esas dos hojas hermanas por un único nodo de peso $a + b$ produce una instancia con un elemento menos cuyo costo óptimo difiere del original exactamente en la constante $a + b$, de modo que resolver óptimamente la instancia reducida resuelve óptimamente la original. La inducción sobre estos dos lemas da la corrección del greedy. Nótese que el enunciado permite disponer los elementos en cualquier orden en la banda, lo cual garantiza que CUALQUIER estructura de árbol binario (no solo particiones contiguas de un orden fijo) es realizable, así que no hay restricción adicional sobre la forma del árbol. [18]

Ejercicio aplicado

Dadas N telas con sus tamaños, todas inicialmente del mismo color, y un proceso que en cada paso toma todas las telas de un color y las reparte en dos colores nuevos pagando la suma de sus tamaños, calcular la tinta mínima total necesaria para que todas las telas terminen con colores distintos.

I/O: Hasta $T \leq 100$ casos con $\sum N_i \leq 10^5$ y tamaños hasta $10^5 \Rightarrow$ lectura total + tokenización con índice `idx` (patrón 2), ya que son muchos números sueltos repartidos en líneas de longitud variable. El caso $N = 1$ se atiende por separado (costo 0: no hace falta ninguna división). La respuesta puede superar los 32 bits, pero los enteros nativos de Python no requieren cuidado especial. Verificado contra el caso de ejemplo y contrastado con una fuerza bruta que explora TODAS las particiones recursivas posibles (cualquier subconjunto, no solo contiguos) en 200 casos aleatorios con $N \leq 8$, sin discrepancias. Prueba de estrés con $\sum N = 10^5$: ~ 0.14 s con un solo caso de $N = 10^5$, ~ 0.06 s repartido en 100 casos, y ~ 0.07 s con todos los tamaños iguales (máximo de empates en el heap).

Código solución

```
import sys
import heapq

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    t = int(data[idx]); idx += 1
    out = []
    for _ in range(t):
        n = int(data[idx]); idx += 1
        sizes = data[idx:idx + n]
        idx += n
        if n == 1:
            out.append("0") # una sola tela: ya
                           # tiene color unico, no se gasta tinta
            continue
        heap = [int(x) for x in sizes]
        heapq.heapify(heap) # heapify es O(n), mas
                           # rapido que n heappush
        total = 0
        while len(heap) > 1:
            a = heapq.heappop(heap)
            b = heapq.heappop(heap)
            s = a + b
            total += s # cada fusion paga el
                     # peso del grupo resultante
            heapq.heappush(heap, s)
            out.append(str(total))
    sys.stdout.write("\n".join(out) + "\n")

solve()
```

6 Programación Dinámica

6.1 DP de Particionamiento sobre Residuos (compra con redondeo)

Preguntas guía:

- ¿Debo partir una secuencia en grupos contiguos minimizando una suma de costos por grupo, con un límite en la cantidad de cortes?
- ¿El costo de cada grupo es (suma del grupo) más un ajuste que depende SOLO de la suma módulo algo pequeño (redondeo, paridad, comisión por tramos)?
- ¿El DP directo sobre posiciones sería $\mathcal{O}(n^2d)$ y los límites lo hacen inviable en Python?
- ¿Puedo reemplazar el estado "posición del último corte" por "residuo del prefijo en el último corte", colapsando n estados en unos pocos?

Utilidad: Reduce drásticamente el espacio de estados de un DP de particionamiento cuando el costo de cada bloque se descompone en una parte TELESCÓPICA (que se cancela globalmente y no depende de dónde se corte) más una parte que solo depende de un residuo módulo un número pequeño. El estado deja de ser "dónde fue el último corte" (n posibilidades) y pasa a ser "qué residuo tenía el prefijo en el último corte" (10 posibilidades), convirtiendo un $\mathcal{O}(n^2d)$ en $\mathcal{O}(ndM)$ con M el módulo.

Complejidad: $\mathcal{O}(n \cdot d \cdot M)$ con $M = 10$, es decir $\mathcal{O}(nd)$ para efectos prácticos, frente a $\mathcal{O}(n^2d)$ del DP directo.

Fundamento teórico: Sea P_i la suma de prefijos y $\text{red}(x)$ el redondeo de x al múltiplo de 10 más cercano (con 5 hacia arriba). Escribiendo $x = 10q + m$ con $m = x \bmod 10$, se tiene $\text{red}(x) = x + \delta(m)$, donde $\delta(m) = -m$ si $m < 5$ y $\delta(m) = 10 - m$ si $m \geq 5$; es decir, el redondeo es la identidad MÁS un ajuste que depende únicamente del residuo. Si los cortes son $0 = c_0 < c_1 < \dots < c_k = n$ y $\Delta_t = P_{c_t} - P_{c_{t-1}}$ es la suma del grupo t , el costo total es

$$\begin{aligned} \sum_{t=1}^k \text{red}(\Delta_t) &= \sum_{t=1}^k \Delta_t + \sum_{t=1}^k \delta(\Delta_t \bmod 10) \\ &= P_n + \sum_{t=1}^k \delta((r_{c_t} - r_{c_{t-1}}) \bmod 10), \end{aligned}$$

donde la primera suma TELESCOPIA a la constante P_n (independiente de los cortes) y $r_i = P_i \bmod 10$. La consecuencia clave es que el término a minimizar depende de las posiciones de corte ÚNICAMENTE a través de los residuos de sus prefijos, porque $\Delta_t \bmod 10 = (r_{c_t} - r_{c_{t-1}}) \bmod 10$. Por eso el estado del DP puede ser $(j, r) =$ "ya se formaron j grupos y el último corte estaba en una posición con residuo r ", guardando el ajuste acumulado mínimo; la posición exacta del corte es irrelevante más allá de su residuo y de que sea anterior a la posición actual. Un detalle de implementación importante: todos los candidatos de la posición i deben calcularse a partir del estado ANTERIOR a actualizarla, para no encadenar un corte con otro de la misma posición (grupo vacío); dicho sea de paso, permitir grupos vacíos sería inofensivo

en cuanto al valor óptimo, pues $\delta(0) = 0$ y equivale a gastar un divisor sin ganancia, lo cual el propio enunciado admite al decir "hasta d divisores".

Ejercicio aplicado

Dados n precios en el orden en que están sobre la banda y hasta d divisores, partir la compra en a lo sumo $d + 1$ grupos contiguos y pagar cada grupo por separado, donde el total de cada grupo se redondea al múltiplo de 10 más cercano. Minimizar el monto total pagado.

I/O: $n \leq 2000$, $d \leq 20$, precios hasta 10^4 , en dos líneas de muchos números sueltos $\Rightarrow$ lectura total + tokenización (patrón 2). Verificado contra los dos casos de ejemplo y contrastado con el DP directo $\mathcal{O}(n^2d)$ en 400 casos aleatorios pequeños, sin discrepancias, más casos borde (un solo ítem, un ítem que redondea hacia arriba, más divisores que ítems, todos los precios múltiplos de 10). Sobre el rendimiento conviene ser explícito: el DP directo, implementado en Python puro, tarda ~ 5.60 s en el peor caso ($n = 2000$, $d = 20$), justo POR ENCIMA del límite de 5 segundos; el DP sobre residuos resuelve el mismo caso en ~ 0.04 s (unas 140 veces más rápido) y ambos coinciden en la respuesta. Es decir, aquí la reducción de estados no es una optimización cosmética sino la diferencia entre TLE y AC.

Código solución

```
import sys

data = sys.stdin.buffer.read().split()
n = int(data[0]); divs = int(data[1])
acumulado = 0
pref = [0]*(n+1)
for i in range(n):
    acumulado += int(data[2+i])
    pref[i+1] = acumulado
total = acumulado # parte telescopica:
                 constante

ajuste = [0,-1,-2,-3,-4,5,4,3,2,1]

INF = float('inf')
max_grupos = divs + 1 # a lo
                    sumo d+1 grupos
# ajuste_min[j][r] = ajuste minimo acumulado tras formar
# j grupos,
# con el ultimo corte en una posicion cuyo prefijo tiene
# residuo r
ajuste_min = [[INF]*10 for _ in range(max_grupos+1)]
ajuste_min[0][pref[0] % 10] = 0

mejor = INF
for i in range(1, n+1):
    residuo_curr = pref[i] % 10
    # se calculan TODOS los candidatos con la g anterior,
    # para no
    # encadenar un corte con otro de esta misma posicion
    # (grupo vacio)
    cand = [INF]*(max_grupos+1)
    for j in range(1, max_grupos+1):
        fila = ajuste_min[j-1]
        minimum = INF
        for r in range(10):
            v = fila[r]
            if v < INF:
                v += ajuste[(residuo_curr - r) % 10]
                if v < minimum: minimum = v
```

```

cand[j] = minimum
for j in range(1, max_grupos+1):
    if cand[j] < ajuste_min[j][residuo_curr]:
        ajuste_min[j][residuo_curr] = cand[j]
if i == n:
    mejor = min(cand[1:])      # el ultimo grupo debe
    cerrar en n
print(total + mejor)

```

6.2 Distancia de Edición y LCS (DP Bit-Paralelo)

Preguntas guía:

- ¿Me piden el mínimo de inserciones, borrados y reemplazos para transformar una cadena en otra (distancia de Levenshtein)?
- ¿Me piden la subsecuencia común más larga (LCS), o la distancia usando solo inserciones y borrados, que es $n + m - 2 \cdot \text{LCS}$?
- ¿La tabla $\mathcal{O}(nm)$ tiene decenas de millones de celdas y el DP clásico en Python no entra en el tiempo?
- ¿Puedo aprovechar que celdas vecinas de la tabla difieren en a lo sumo 1, para codificar una columna entera en bits?

Utilidad: Calcula la distancia de edición (y, con una variante, la LCS) entre dos cadenas. El DP clásico llena una tabla de $n \times m$ celdas, lo cual es simple pero inviable en Python cuando n, m llegan a varios miles. La versión bit-paralela representa una columna entera de la tabla como enteros de m bits y la actualiza con unas quince operaciones de bits por carácter; como Python ejecuta esas operaciones sobre enteros grandes en C, el bucle interpretado pasa de nm iteraciones a solo n . Es la misma idea que sirve para búsqueda aproximada de patrones (con k errores) y para LCS en tiempo casi lineal.

Complejidad: $\mathcal{O}(nm)$ el DP clásico; $\mathcal{O}(n \cdot \lceil m/w \rceil)$ el bit-paralelo, con $w = 64$ el tamaño de palabra. En Python esto se traduce en n iteraciones del bucle, cada una con operaciones sobre enteros de m bits resueltas en C.

Fundamento teórico: El DP clásico define $D[i][j]$ como la distancia entre los prefijos $a[0..i]$ y $b[0..j]$, con $D[i][0] = i$, $D[0][j] = j$ y $D[i][j] = \min(D[i-1][j] + 1, D[i][j-1] + 1, D[i-1][j-1] + [a_{i-1} \neq b_{j-1}])$: borrar, insertar, o reemplazar (gratis si las letras coinciden). La observación que habilita el paralelismo es que dos celdas vecinas de la tabla, horizontal o verticalmente, difieren en EXACTAMENTE $-1, 0$ o $+1$: agregar un carácter a una de las cadenas nunca cambia la distancia en más de una operación. Por lo tanto, en lugar de guardar los valores de una columna, alcanza con guardar sus DIFERENCIAS verticales, codificadas en dos máscaras de bits: Pv marca las filas donde la diferencia es $+1$ y Mv las filas donde es -1 (las demás valen 0). El algoritmo de Myers, adaptado por Hyrö a la distancia global, deriva de la recurrencia unas pocas fórmulas de bits que calculan, a partir de la columna anterior y de la máscara Eq de posiciones de a donde aparece la letra actual de b , las diferencias horizontales

(Ph, Mh) y luego la nueva columna. El paso no evidente es la SUMA (Eq & Pv) + Pv: el acarreo de la suma binaria se propaga a lo largo de tramos consecutivos de bits, y eso modela exactamente cómo una coincidencia en la diagonal se propaga hacia abajo por la columna, algo que celda por celda requeriría un bucle. El valor final no se reconstruye entero: se lleva solo `score`, el valor de la última fila, que empieza en m y en cada columna sube o baja según el bit más alto de Ph o Mh. El desplazamiento $(\text{Ph} \ll 1) \mid 1$ introduce un $+1$ en la fila 0 porque $D[0][j] = j$ crece en uno por columna; esa es la diferencia con la búsqueda aproximada de patrones, donde se introduce un 0 para permitir que la coincidencia empiece en cualquier lugar del texto. Para la LCS la situación es aún más simple: las diferencias verticales son solo 0 o 1, así que basta una máscara, y la actualización se reduce a una suma y una resta. En Python hay un detalle adicional: el operador $\sim$ sobre enteros produce negativos (es $-x - 1$), por lo que toda negación y todo desplazamiento se recortan con $\& \text{mask}$ a m bits. [19]

Ejercicio aplicado

(CSES 1639 — Edit Distance) Dadas dos cadenas en mayúsculas de longitudes $n, m \leq 5000$, calcular su distancia de edición con inserciones, borrados y reemplazos.

I/O: Dos líneas, una cadena cada una $\Rightarrow$ `sys.stdin.buffer.readline` (patrón 1), trabajando sobre `bytes` para que cada letra sea un entero usable directamente como clave de la tabla `peq`. Sobre el rendimiento conviene ser explícito, porque es la razón de ser de esta versión: el DP clásico, incluso con filas rodadas y sin llamadas a `min`, tarda $\sim 3.3\text{s}$ en el caso 5000×5000 , muy por encima del límite de 1 segundo; la versión bit-paralela resuelve el mismo caso en $\sim 0.03\text{s}$, unas 125 veces más rápido, con la misma respuesta. Verificado contra el caso de ejemplo y contrastado con el DP clásico en 6069 casos, incluyendo alfabetos de 1 a 26 letras, cadenas iguales, prefijos, cadenas sin letras en común, el clásico KITTEN/SITTING, y 60 pares de entre 65 y 300 caracteres para ejercitar el acarreo entre palabras de 64 bits, sin fallos.

Código solución

```

import sys

def edit_distance(a, b):
    m = len(a)
    mask = (1 << m) - 1      # recorta a m bits: ~x en
                             # Python da negativos
    alto = 1 << (m - 1)      # bit de la ultima fila
                             # de la tabla
    # peq[c] = mascara de las posiciones de a donde
    # aparece la letra c
    peq = {}
    for i, c in enumerate(a):
        peq[c] = peq.get(c, 0) | (1 << i)
    Pv = mask                # columna 0: D[i][0] = i,
                             # todas las diferencias son +1
    Mv = 0
    score = m                # valor de la ultima fila
    , D[m][0] = m
    for c in b:

```

```

Eq = peq.get(c, 0)
Xv = Eq | Mv
# la SUMA propaga las coincidencias de la
diagonal hacia abajo por acarreo
Xh = (((Eq & Pv) + Pv) ^ Pv) | Eq
Ph = Mv | (~Xh | Pv) & mask # diferencias
horizontales +1
Mh = Pv & Xh # diferencias
horizontales -1
if Ph & alto:
    score += 1
elif Mh & alto:
    score -= 1
# el /1 mete el +1 de la fila 0 (D[0][j] = j):
distancia GLOBAL
Ph = ((Ph << 1) | 1) & mask
Mh = (Mh << 1) & mask
Pv = Mh | (~Xv | Ph) & mask # nueva columna
de diferencias verticales
Mv = Ph & Xv
return score

def solve():
    a = sys.stdin.buffer.readline().strip()
    b = sys.stdin.buffer.readline().strip()
    print(edit_distance(a, b))

solve()

```

Extras: LCS bit-paralela — la subsecuencia común más larga con la misma técnica. Como las diferencias verticales de la tabla de LCS son solo 0 o 1, basta una máscara y la actualización se reduce a una suma y una resta. Verificada contra una tabla de referencia en 5050 casos; en 5000×5000 tarda ~ 0.004 s.

```

def lcs_bits(a, b):
    m = len(a)
    mask = (1 << m) - 1
    peq = {}
    for i, c in enumerate(a):
        peq[c] = peq.get(c, 0) | (1 << i)
    V = mask
    for c in b:
        U = V & peq.get(c, 0)
        V = ((V + U) | (V - U)) & mask
    return m - bin(V).count("1") # ceros de V = largo
                                # de la LCS
                                # (en Python 3.10+: V
                                .bit_count())

# con solo insertar y borrar (sin reemplazo): distancia =
n + m - 2 * lcs_bits(a, b)

```

6.3 Subsecuencia Creciente Más Larga (Arreglo de Colas + Búsqueda Binaria)

Preguntas guía:

- ¿Me piden la longitud de la subsecuencia creciente (o decreciente, o no decreciente) más larga de un arreglo?
- ¿El DP clásico $\mathcal{O}(n^2)$ es inviable porque n llega a 10^5 o más?
- ¿Me basta con la LONGITUD, o también necesito reconstruir la subsecuencia?
- ¿El problema pide estrictamente creciente o admite iguales? (eso decide entre `bisect_left` y `bisect_right`)

Utilidad: Calcula la longitud de la subsecuencia creciente más larga en tiempo $\mathcal{O}(n \log n)$ manteniendo, para cada longitud posible, el menor valor con el que puede terminar una subsecuencia de esa longitud. Aparece disfrazado en problemas de apilar cajas, plataformas o sobres (ordenar por una dimensión y hacer LIS sobre la otra), en el mínimo número de sistemas de misiles (por el teorema de Dilworth, es la longitud de la subsecuencia no creciente más larga) y en particiones en cadenas monótonas.

Complejidad: $\mathcal{O}(n \log n)$: una búsqueda binaria por elemento sobre un arreglo de a lo sumo n posiciones.

Fundamento teórico: Se mantiene un arreglo `colas` donde `colas[k]` es el MENOR valor con el que puede terminar una subsecuencia creciente de longitud $k+1$ entre los elementos procesados. Ese arreglo es estrictamente creciente: una subsecuencia de longitud $k+1$ contiene una de longitud k que termina antes y con un valor menor, así que el mejor final para k es menor que el mejor para $k+1$; gracias a eso se puede buscar en él binariamente. Al procesar un elemento x se busca la primera posición i con `colas[i]  $\geq$  x`: si no existe, x extiende la subsecuencia más larga conocida y se agrega al final; si existe, x es un final mejor (o igual) para longitud $i+1$ y reemplaza a `colas[i]`. Reemplazar nunca pierde soluciones: no cambia la longitud máxima actual, pero deja el arreglo con finales más chicos, lo que solo puede facilitar extensiones futuras. La longitud de la LIS es el tamaño final del arreglo. Es importante notar que `colas` NO es una subsecuencia válida del arreglo original, solo un resumen de los mejores finales por longitud; para reconstruir la subsecuencia hay que guardar además, para cada elemento, en qué posición del arreglo se ubicó y quién lo precedía. La elección entre `bisect_left` y `bisect_right` es lo que distingue creciente Estricta de no decreciente: con `bisect_left`, un valor igual reemplaza al existente en vez de extender, que es exactamente lo que impide contar dos elementos iguales en la misma subsecuencia. [20]

Ejercicio aplicado

(CSES 1145 — Increasing Subsequence) Dado un arreglo de n enteros, hallar la longitud de la subsecuencia estrictamente creciente más larga.

I/O: $n \leq 2 \cdot 10^5$ con muchos números sueltos $\Rightarrow$ lectura total + tokenización (patrón 2), convirtiendo con `map(int, ...)` directamente sobre el iterador del bucle para no materializar una lista intermedia. Verificado contra el caso de ejemplo y contrastado con el DP clásico $\mathcal{O}(n^2)$ en 600 casos aleatorios, incluyendo arreglos crecientes, decrecientes, todos iguales y con valores repetidos, sin discrepancias. Con $n = 2 \cdot 10^5$ tarda entre ~ 0.04 s y ~ 0.08 s en todos los perfiles medidos (aleatorio, creciente donde la LIS es n , decreciente, todos iguales y sierra); el límite es de 1 segundo. Como referencia de por qué no sirve el DP clásico: con $n = 5000$ ya tarda ~ 1.18 s, lo que extrapolado a $n = 2 \cdot 10^5$ daría del orden de media hora.

Código solución

```

import sys
from bisect import bisect_left

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    # colas[k] = menor valor con el que puede terminar
    # una subsecuencia
    # creciente de largo k+1; queda siempre ordenado, por
    # eso se puede biseccionar
    colas = []
    for x in map(int, data[1:1+n]):
        i = bisect_left(colas, x)      # bisect_left =>
        # creciente Estricta
        if i == len(colas):
            colas.append(x)           # x extiende la
            # subsecuencia mas larga
        else:
            colas[i] = x              # x es un final
            # mejor para ese largo
    print(len(colas))                # el largo del
            # arreglo es la respuesta

solve()

```

6.4 Mochila 0/1 con DP Bit-Paralelo (SWAR)

Preguntas guía:

- ¿Debo elegir un subconjunto de objetos, cada uno usable a lo sumo UNA vez, maximizando un valor sin pasarme de una capacidad?
- ¿El producto (cantidad de objetos) $\times$ (capacidad) llega a 10^7 o más, de modo que el DP celda por celda en Python es inviable?
- ¿Los valores del DP son enteros acotados y chicos, de modo que puedo empaquetar toda la tabla en un entero grande?
- ¿La transición es "desplazar la tabla por el peso, sumar el valor y quedarme con el máximo"?

Utilidad: Resuelve la mochila 0/1 (elegir objetos con peso y valor sin repetir, maximizando valor bajo una capacidad) cuando el DP clásico es correcto pero demasiado lento en Python. La idea general — empaquetar una fila entera del DP en un solo entero y hacer la transición con operaciones de bits — aplica a cualquier DP unidimensional cuyos valores sean enteros chicos y cuya transición sea un desplazamiento más un máximo o un mínimo: mochila, subset-sum con valores, cambio de monedas con conteo acotado.

Complejidad: $\mathcal{O}(nx)$ operaciones elementales, pero ejecutadas dentro de C: el bucle interpretado hace solo $\mathcal{O}(n)$ iteraciones, cada una con unas once operaciones sobre un entero de $x \cdot W$ bits.

Fundamento teórico: El DP clásico recorre los objetos y mantiene $dp[j]$ = mejor valor alcanzable con capacidad j , actualizando $dp[j] = \max(dp[j], dp[j - h] + s)$ para el objeto de peso h y valor s . Que cada objeto se use a lo sumo una vez se garantiza usando siempre la fila ANTERIOR del lado derecho (de ahí que la versión con listas recorra j de mayor a menor, o tome una copia por

slicing). Visto como operación sobre el arreglo completo, la transición es: tomar el arreglo, sumarle s a todas sus entradas, DESPLAZARLO h posiciones hacia arriba (las posiciones nuevas quedan en cero, que es lo correcto: con capacidad menor que h el objeto no se puede comprar), y quedarse con el máximo elemento a elemento entre eso y el arreglo original. Si el arreglo se empaqueta en un entero con un campo de W bits por posición, "sumar s a todas las entradas" es multiplicar por la constante de unos y sumar, y "desplazar h posiciones" es un corrimiento de Wh bits: ambas operaciones ya existen en los enteros de Python y corren en C. El máximo elemento a elemento se hace con la técnica SWAR: si cada campo reserva su bit más alto como GUARDIA y todos los valores caben por debajo, entonces en $D = (X \mid G_0) - Y$ no hay préstamos entre campos, y el bit de guardia de cada campo queda encendido exactamente donde $X_j \geq Y_j$, con los bits bajos valiendo $X_j - Y_j$. Expandiendo esas guardias a una máscara de campo completo mediante $M = G - (G \gg (W - 1))$, resulta $\max = Y + (D \& M)$: donde X ganaba se le suma la diferencia y donde no, queda Y . La elección de W depende de la cota de los valores: aquí el total de páginas es a lo sumo $1000 \times 1000 = 10^6 < 2^{20}$, y el candidato intermedio $dp + s$ tampoco pasa de 2^{20} , así que bastan 20 bits de valor más 1 de guardia. Esa cota es AJUSTADA: si el problema permitiera más objetos o valores más grandes, habría que subir W o el método daría resultados corruptos en silencio. [21]

Ejercicio aplicado

(CSES 1158 — Book Shop) Dados n libros con precio y cantidad de páginas y un presupuesto x , hallar la máxima cantidad de páginas comprable sin repetir libros ni pasarse del presupuesto.

I/O: $n \leq 1000$, $x \leq 10^5$, precios y páginas hasta 1000 $\Rightarrow$ lectura total + tokenización (patrón 2). La constante de unos se construye con el truco del repunit, $\text{ONES} = (2^{Wc} - 1) / (2^W - 1)$ con c campos, en una sola división. Verificado contra el caso de ejemplo y contrastado con una fuerza bruta que prueba todos los subconjuntos en 400 casos con $n \leq 12$, sin discrepancias, y además contra una implementación con listas y `map` en el caso máximo, con idéntico resultado. Sobre el rendimiento conviene ser explícito: esa versión con listas, que ya usa `map` en C y evita bucles por celda, tarda ~ 14.6 s con $n = 1000$ y $x = 10^5$; la versión bit-paralela resuelve el mismo caso en ~ 0.56 s, unas 25 veces más rápido, con un límite de 1 segundo. Los otros perfiles medidos dan ~ 0.41 s (todos los precios iguales a 1) y ~ 0.60 s (todos iguales a 1000).

Código solución

```

import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); x = int(data[1])
    H = list(map(int, data[2:2+n])); S = list(map(int,
        data[2+n:2+2*n]))
    W = 21
    # 20 bits de

```

```

    valor (paginas <= 10^6) + 1 de guardia
W1 = W - 1
campos = x + 1
MASK = (1 << (W*campos)) - 1
ONES = MASK // ((1 << W) - 1)          # un 1 en el
    bit bajo de cada campo (repunit)
GO = ONES << W1                        # bit de
    guardia de cada campo
X = 0                                  # dp
empaquetado: campo j = paginas con presupuesto j
for h, s in zip(H, S):
    if h > x:
        continue
    # candidato: dp con s paginas mas, desplazado h
    campos.
    # los h campos bajos quedan en 0 = no alcanza el
    presupuesto para este libro
    Y = ((X + s*ONES) << (W*h)) & MASK
    D = (X | GO) - Y                    # sin
    prestamos entre campos: guardia = (X_j >= Y_j)
    G = D & GO
    M = G - (G >> W1)                  # expande
    cada guardia a todo su campo
    X = Y + (D & M)                    # maximo
    campo a campo
print((X >> (W*x)) & ((1 << W) - 1))

solve()

```

6.5 Conteo de Caminos en Grilla con Obstáculos (DP por Filas)

Preguntas guía:

- ¿Me piden CONTAR caminos en una grilla donde solo se avanza hacia la derecha y hacia abajo?
- ¿Hay casillas bloqueadas, de modo que la fórmula combinatoria con binomios no aplica?
- ¿El valor de cada celda depende solo de la celda de arriba y la de la izquierda, permitiendo guardar una sola fila?
- ¿El resultado se pide módulo un primo porque crece exponencialmente?

Utilidad: Cuenta caminos monótonos en una grilla con obstáculos mediante un DP que recorre las celdas en un orden donde las dependencias ya están calculadas. Es el DP bidimensional más simple y la base de muchas variantes: caminos con costo mínimo o máximo en vez de conteo, caminos que deben pasar por ciertas celdas, o grillas con más direcciones permitidas. Cuando NO hay obstáculos, el mismo conteo sale por combinatoria con un solo binomio, así que el DP se justifica precisamente por los bloqueos.

Complejidad: $\mathcal{O}(n^2)$ en tiempo y $\mathcal{O}(n)$ en memoria, guardando una sola fila.

Fundamento teórico: Sea $dp[i][j]$ la cantidad de caminos desde la esquina superior izquierda hasta la celda (i, j) . Como solo se puede llegar desde arriba o desde la izquierda, y esos dos conjuntos de caminos son disjuntos (se distinguen por el último movimiento), vale $dp[i][j] = dp[i-1][j] + dp[i][j-1]$, con $dp[i][j] = 0$ si la celda tiene una trampa. Recorriendo las filas de arriba hacia abajo y cada fila de izquierda a derecha, ambas dependencias ya están

calculadas cuando se necesita la celda actual; además, la de arriba es la fila anterior y la de la izquierda es la celda recién calculada, así que basta un arreglo de una fila más una variable con el valor de la izquierda. Un detalle de implementación que evita casos especiales: en lugar de inicializar la primera fila a mano, se arranca con una fila virtual que vale 1 en la primera columna y 0 en el resto; al procesar la fila 0, la celda $(0, 0)$ recibe ese 1 y el resto se propaga solo, incluido el caso en que la propia casilla de salida esté bloqueada. Como la cantidad de caminos crece exponencialmente (sin obstáculos es $\binom{2n-2}{n-1}$), todo se lleva módulo $10^9 + 7$, y como el enunciado solo pide el conteo final, no hace falta guardar la tabla completa.

Ejercicio aplicado

(CSES 1638 — Grid Paths I) Dada una grilla de $n \times n$ donde algunas casillas tienen trampas, contar módulo $10^9 + 7$ los caminos de la esquina superior izquierda a la inferior derecha moviéndose solo a la derecha o hacia abajo.

I/O: $n \leq 1000$ con filas de texto sin espacios $\Rightarrow$ lectura total + tokenización (patrón 2), donde cada fila es un token que se recorre como `bytes` comparando contra 42, el código ASCII de *. Verificado contra el caso de ejemplo y contrastado con un conteo recursivo exacto (sin módulo, con memoización) en 500 grillas aleatorias, incluyendo grillas sin trampas, muy densas, con la casilla inicial o final bloqueada y con $n = 1$, sin discrepancias. Con $n = 1000$ (un millón de celdas) tarda entre $\sim 0.05s$ y $\sim 0.09s$ según la densidad de trampas, frente a un límite de 1 segundo. Se probó también una versión que reemplaza el bucle interno por sumas de prefijos con `accumulate` sobre los tramos sin trampas, ejecutadas en C: da lo mismo y es marginalmente más rápida sin trampas ($\sim 0.06s$), pero más lenta cuando abundan ($\sim 0.17s$) porque se fragmenta en muchos tramos cortos, así que no se justifica la complicación.

Código solución

```

import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    MOD = 10**9 + 7
    prev = [0]*n
    prev[0] = 1                                # fila virtual: la
    entrada a la casilla (0,0)
    for i in range(n):
        fila = data[1 + i]
        cur = [0]*n
        izq = 0                                # caminos que llegan
        desde la izquierda
        for j in range(n):
            if fila[j] == 42:                  # 42 = '*', trampa:
                no se puede pasar
                izq = 0
            else:
                izq = (izq + prev[j]) % MOD    # desde la
                izquierda + desde arriba
                cur[j] = izq
        prev = cur
    print(prev[n-1] % MOD)

```

6.6 DP en Árboles con Recorrido Iterativo (emparejamiento máximo)

Preguntas guía:

- ¿El grafo es un ÁRBOL y me piden optimizar algo que se puede decidir subárbol por subárbol?
- ¿El estado de cada nodo se resume en pocos casos (usado / libre, tomado / no tomado, pintado de tal color)?
- ¿La respuesta de un nodo depende solo de las respuestas de sus hijos, permitiendo procesar de las hojas hacia la raíz?
- ¿La profundidad puede llegar a 10^5 o más, obligando a un recorrido iterativo en vez de recursivo?

Utilidad: Es la plantilla del DP sobre árboles: se recorre el árbol de las hojas hacia la raíz y cada nodo combina los resultados de sus hijos según un estado pequeño. Con dos estados por nodo se resuelven emparejamiento máximo, conjunto independiente máximo (con o sin pesos), cobertura mínima de vértices y coloreos; con más estados, variantes como "a lo sumo k hijos elegidos". Todas comparan la estructura de esta entrada: un orden que garantiza hijos antes que padres, y una combinación aditiva con una corrección por el caso especial.

Complejidad: $O(n)$ en tiempo y memoria: cada nodo y cada arista se visitan una vez.

Fundamento teórico: Para cada nodo v se definen dos valores sobre su subárbol: $dp_0[v]$, el mejor emparejamiento dejando a v SIN emparejar, y $dp_1[v]$, el mejor dejándolo emparejado con alguno de sus hijos. Si v queda libre, cada hijo aporta lo mejor que pueda por su cuenta: $dp_0[v] = \sum_c \max(dp_0[c], dp_1[c])$. Si v se empareja con un hijo c , ese hijo debe estar libre, así que aporta $dp_0[c] + 1$ en lugar de $\max(dp_0[c], dp_1[c])$, y el resto de los hijos sigue aportando su máximo; por lo tanto

$$dp_1[v] = dp_0[v] + \max_c (dp_0[c] + 1 - \max(dp_0[c], dp_1[c])).$$

Escrito así, no hace falta recorrer los hijos dos veces ni guardar listas de hijos: basta acumular en el padre, mientras se procesan los hijos, la suma de los máximos y por separado la mayor de esas diferencias. La respuesta del nodo es $\max(dp_0[v], dp_1[v])$, que equivale a sumar la diferencia solo si es positiva. Hay dos detalles de implementación que evitan problemas. Primero, el ORDEN: se construye un recorrido por capas desde la raíz, que garantiza que todo nodo aparece después de su padre; recorrerlo AL REVÉS garantiza que los hijos se procesan antes, que es lo que el DP necesita, y evita la recursión, inviable con profundidad $2 \cdot 10^5$. Segundo, no hace falta separar hijos de padre en la lista de adyacencia: al marcar el padre de cada nodo durante la construcción del orden, el recorrido inverso ya sabe a quién aportar.

Ejercicio aplicado

(CSES 1130 — Tree Matching) Dado un árbol de n nodos, hallar la máxima cantidad de aristas que se pueden elegir sin que dos compartan un nodo (emparejamiento máximo).

I/O: $n \leq 2 \cdot 10^5$ con dos enteros por arista $\Rightarrow$ lectura total + tokenización (patrón 2), separando extremos con slices de paso 2. Se atiende aparte $n = 1$, donde no hay aristas que leer. Verificado contra el caso de ejemplo y contrastado con una fuerza bruta que prueba todos los subconjuntos de aristas en 500 árboles de hasta 11 nodos (camino, estrellas, orugas, binarios y aleatorios, con los nodos reenumerados al azar y las aristas mezcladas), sin discrepancias. Con $n = 2 \cdot 10^5$ tarda entre ~ 0.19 s y ~ 0.26 s en todas las formas medidas, incluido un camino de profundidad $2 \cdot 10^5$ donde una versión recursiva desbordaría la pila; el límite es de 1 segundo.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    if n == 1:
        print(0); return
    adj = [[] for _ in range(n + 1)]
    fin = 1 + 2*(n - 1)
    for a, b in zip(map(int, data[1:fin:2]), map(int, data[2:fin:2])):
        adj[a].append(b)
        adj[b].append(a)
    # orden por capas: cada nodo aparece despues de su padre
    padre = [0]*(n + 1)
    orden = [1]; padre[1] = 1 # la raiz se marca
    # a si misma para no revisitarla
    for u in orden: # la lista crece
        # mientras se recorre
        for v in adj[u]:
            if not padre[v]:
                padre[v] = u
                orden.append(v)
    libre = [0]*(n + 1) # dp0: mejor emparejamiento del subarbol con el nodo LIBRE
    ganan = [0]*(n + 1) # mayor (dp0[hijo] + 1 - max(dp0, dp1)[hijo]) entre los hijos
    for v in reversed(orden): # al revés: los hijos ya estan procesados
        # max(dp0[v], dp1[v]): emparejar a v con un hijo solo si la diferencia suma
        mejor = libre[v] + (ganan[v] if ganan[v] > 0 else 0)
        if v == 1:
            print(mejor); return
        p = padre[v]
        libre[p] += mejor # aporte de v si el padre queda libre
        # si el padre se empareja CON v, v debe quedar libre: aporta dp0[v] + 1
        cand = libre[v] + 1 - mejor
        if cand > ganan[p]:
            ganan[p] = cand

solve()
```

6.7 DP de Dígitos (Conteo en un Rango con Restricciones)

Preguntas guía:

- ¿Me piden CONTAR cuántos números de un rango $[a, b]$ cumplen una propiedad, con b tan grande que recorrerlos uno por uno es imposible?
- ¿La propiedad se puede verificar leyendo el número dígito a dígito, recordando solo un resumen chico del prefijo (último dígito, suma parcial, resto)?
- ¿Puedo reducir el rango a dos conteos desde cero, es decir, $f(b) - f(a - 1)$?
- ¿Necesito distinguir los ceros a la izquierda de los ceros que son parte del número?

Utilidad: Cuenta números en un rango que cumplen una propiedad decidible dígito a dígito, construyéndolos de izquierda a derecha y memorizando estados repetidos. Sirve para condiciones sobre dígitos adyacentes, sumas de dígitos, divisibilidad, dígitos prohibidos, cantidad de apariciones de un dígito, o combinaciones de todas ellas: lo único que cambia es qué se guarda en el estado.

Complejidad: $\mathcal{O}(D \cdot E \cdot 10)$ donde $D \leq 19$ es la cantidad de dígitos y E el tamaño del estado (aquí 10 valores del último dígito por dos banderas), es decir unos pocos miles de operaciones por consulta.

Fundamento teórico: Contar en un rango se reduce a contar desde cero: $|[a, b]| = f(b) - f(a - 1)$, donde $f(x)$ cuenta los válidos en $[0, x]$. Para calcular $f(x)$ se construye el número dígito a dígito de izquierda a derecha, eligiendo en cada posición un dígito y llevando tres piezas de información. La primera es el estado propio de la propiedad: aquí, el ÚLTIMO dígito colocado, que es todo lo que hace falta para saber si el siguiente lo repite. La segunda es la bandera de PEGADO al límite: si todos los dígitos elegidos hasta ahora coinciden con los de x , el dígito actual no puede pasarse del dígito correspondiente de x ; en cuanto se elige uno estrictamente menor, el resto de las posiciones queda libre de 0 a 9 y esa bandera se apaga para siempre. La tercera es la bandera de EMPEZADO, que distingue los ceros a la izquierda (que no son dígitos del número) de los ceros reales: mientras no se empezó, un cero no cuenta como dígito y por lo tanto no puede "repetirse" con el siguiente, que es justamente por qué la comparación de repetición se hace solo cuando la bandera está encendida. Sin esa distinción, un número como 101 se rechazaría al comparar contra los ceros de relleno. Como muchas ramas distintas llegan al mismo estado (posición, último, empezado, pegado), memorizarlo convierte una exploración exponencial en unos pocos miles de estados. El caso $a = 0$ se resuelve solo: $f(-1) = 0$ por la guarda inicial. Nótese que el número 0 se cuenta como válido (nunca enciende la bandera de empezado y llega al final sin repetir nada), que es lo correcto.

Ejercicio aplicado

(CSES 2220 — Counting Numbers) Dados a y b con $0 \leq a \leq b \leq 10^{18}$, contar cuántos enteros del rango no

tienen dos dígitos adyacentes iguales.

I/O: Una sola línea con dos enteros $\Rightarrow$ lectura total + tokenización (patrón 2); 10^{18} entra sin problema en los enteros de Python. La recursión tiene profundidad 19 como máximo, así que no hace falta tocar el límite de recursión, y la caché se limpia entre las dos llamadas porque los estados dependen de los dígitos de x . Verificado contra el caso de ejemplo y contrastado con un conteo directo número por número en 310 rangos, incluyendo $a = b$, $a = 0$, rangos de un solo elemento y cruces de decena y centena, sin discrepancias. Con los límites máximos tarda ~ 0.013 s, frente a un límite de 1 segundo.

Código solución

```
import sys
from functools import lru_cache

def contar(x):
    # cuantos numeros de 0 a x no tienen dos digitos
    # adyacentes iguales
    if x < 0:
        return 0
    s = str(x)
    n = len(s)

    @lru_cache(maxsize=None)
    def ir(i, ult, empezo, pegado):
        if i == n:
            return 1
        # pegado: todos los digitos previos son iguales a
        # los de x,
        # asi que este no puede pasarse del dígito
        # correspondiente
        tope = int(s[i]) if pegado else 9
        total = 0
        for d in range(tope + 1):
            if empezo and d == ult:  # dígito
                # repetido consecutivo
                continue
            # empezo se enciende con el primer dígito no
            # nulo: antes de eso
            # los ceros son relleno y no cuentan como
            # dígitos del numero
            total += ir(i + 1, d, empezo or d > 0, pegado)
        and d == tope
        return total

    r = ir(0, -1, False, True)
    ir.cache_clear()  # los estados
    # dependen de los digitos de x
    return r

def solve():
    a, b = map(int, sys.stdin.buffer.read().split())
    print(contar(b) - contar(a - 1))  # contar(-1) = 0
    # cubre el caso a = 0

solve()
```

6.8 DP de Intervalos (Corte Óptimo de una Barra)

Preguntas guía:

- ¿Debo partir una secuencia o un segmento en trozos, y el costo de cada partición depende del TRAMO completo que se está partiendo?

- ¿La respuesta de un tramo se arma eligiendo dónde hacer el primer (o último) corte y sumando las respuestas de los dos sub-tramos?
- ¿Los sub-tramos son siempre intervalos contiguos, de modo que el estado es un par (inicio, fin)?
- ¿El costo del tramo NO depende de dónde se corte, solo de sus extremos? (eso habilita la optimización de Knuth)

Utilidad: Es la plantilla del DP sobre intervalos: el estado es un segmento y la transición prueba todos los puntos interiores donde partirlo. Resuelve multiplicación óptima de cadenas de matrices, corte óptimo de barras o cuerdas, partición en palíndromos, fusión de pilas de piedras y árboles binarios de búsqueda óptimos. Lo que cambia entre esos problemas es solo el costo asociado al tramo; la estructura del recorrido es idéntica.

Complejidad: $\mathcal{O}(n^3)$ en la versión directa, o $\mathcal{O}(n^2)$ con la optimización de Knuth de los extras.

Fundamento teórico: Se agregan centinelas en los extremos, de modo que los puntos de corte quedan en un arreglo $p[0] = 0 < p_1 < \dots < p_n < p[n+1] = L$. Sea $dp[i][j]$ el costo mínimo de hacer todos los cortes estrictamente entre $p[i]$ y $p[j]$. La observación central es que, sin importar en qué orden se corte, el PRIMER corte de ese tramo cuesta siempre su longitud completa $p[j] - p[i]$, porque en ese momento el tramo todavía está entero. Elegido ese primer corte en k , el problema se separa en dos subproblemas independientes, los tramos $[i, k]$ y $[k, j]$, que nunca vuelven a interactuar. Por lo tanto

$$dp[i][j] = (p[j] - p[i]) + \min_{i < k < j} (dp[i][k] + dp[k][j]),$$

con $dp[i][i+1] = 0$ porque un tramo sin cortes interiores no cuesta nada. El orden de cálculo es por LONGITUD de intervalo creciente: al resolver un tramo, sus dos sub-tramos son más cortos y ya están listos. La optimización de Knuth se apoya en que este costo cumple la desigualdad cuadrangular (el costo del tramo depende solo de sus extremos y es monótono al agrandarlo), lo que garantiza que el punto de corte óptimo es MONÓTONO: si $opt[i][j]$ es el mejor k , entonces $opt[i][j-1] \leq opt[i][j] \leq opt[i+1][j]$. Recorriendo solo ese rango, la suma total de iteraciones telescopia y el algoritmo baja a $\mathcal{O}(n^2)$. [22]

Ejercicio aplicado

(UVA 10003 — Cutting Sticks) Dada una barra de longitud l y n posiciones donde hay que cortarla, donde cada corte cuesta la longitud del trozo que se está cortando, hallar el costo mínimo total. La entrada termina con $l = 0$.

I/O: Varios casos con centinela $l = 0$ (patrón 3), leídos con lectura total + tokenización porque son números sueltos repartidos en varias líneas. $l < 1000$ y $n < 50$, así que la tabla es de a lo sumo 52×52 . Verificado contra los dos casos de ejemplo y contrastado con una fuerza bruta que simula TODOS los órdenes de corte posibles (la definición literal del enunciado) en 254 casos con hasta 7 cortes, incluyendo $n = 0$, sin discrepancias. El enunciado no acota la cantidad de casos de prueba, así que se midió

el costo por caso con $n = 49$: ~ 1.14 ms por caso, es decir ~ 1.1 s con 1000 casos y ~ 5.7 s con 5000. La versión con la optimización de Knuth de los extras, verificada igual contra la fuerza bruta y contra esta, da ~ 0.35 ms por caso (~ 1.75 s con 5000), unas 3.4 veces más rápido: conviene usarla si la entrada real resulta ser grande.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    pos = 0
    out = []
    while pos < len(data):
        l = int(data[pos]); pos += 1
        if l == 0:
            break
        n = int(data[pos]); pos += 1
        p = [0] + [int(v) for v in data[pos:pos+n]] + [l]
        # centinelas en los extremos
        pos += n
        m = n + 2
        INF = float('inf')
        dp = [[0]*m for _ in range(m)] # dp[i][j]:
        # cortar lo que hay entre p[i] y p[j]
        # se resuelve por tramos cada vez mas largos: los
        # sub-tramos ya estan listos
        for salto in range(2, m):
            for i in range(m - salto):
                j = i + salto
                largo = p[j] - p[i] # el PRIMER
                # corte del tramo siempre cuesta esto
                mejor = INF
                for k in range(i + 1, j): # donde
                    # hacer ese primer corte
                    v = dp[i][k] + dp[k][j]
                    if v < mejor: mejor = v
                dp[i][j] = mejor + largo
            out.append("The minimum cutting is %d." % dp[0][
                m-1])
    sys.stdout.write("\n".join(out) + "\n")

solve()
```

Extras: optimización de Knuth — reemplaza el bucle interno completo por un rango acotado, bajando de $\mathcal{O}(n^3)$ a $\mathcal{O}(n^2)$. Aplica cuando el costo del tramo depende solo de sus extremos y es monótono, condición que aquí se cumple; si no se cumple, da respuestas incorrectas, así que conviene verificarla contra la versión cúbica antes de confiar en ella.

```
dp = [[0]*m for _ in range(m)]
opt = [[0]*m for _ in range(m)] # mejor
# punto de corte de cada tramo
for i in range(m - 1):
    opt[i][i+1] = i + 1
for salto in range(2, m):
    for i in range(m - salto):
        j = i + salto
        # el corte optimo es monotono: basta
        # mirar entre estos dos
        lo = opt[i][j-1]; hi = opt[i+1][j]
        mejor = None; arg = lo
        fila = dp[i]
        for k in range(lo, hi + 1):
            v = fila[k] + dp[k][j]
            if mejor is None or v < mejor:
                mejor = v; arg = k
```

```
dp[i][j] = mejor + p[j] - p[i]  
opt[i][j] = arg
```

7 Matemáticas y Simulación

7.1 Reducción Binaria tipo Josefo (fila con detector alternado)

Preguntas guía:

- ¿El problema describe un proceso repetitivo/alternado (cada segundo elemento se "marca") sobre n elementos, con n muy grande ($\geq 10^9$)?
- ¿Simular paso a paso sería $\mathcal{O}(n)$, demasiado lento, pero el patrón se repite de forma autosimilar al dividir por la mitad?
- ¿Puedo replantear el problema de tamaño n como un subproblema de tamaño $\sim n/2$ aplicando la misma regla?

Utilidad: Resolver en $\mathcal{O}(\log n)$ problemas de simulación con patrones alternados sobre secuencias enormes, sin simular cada paso. Es la misma idea del clásico Problema de Josefo: en vez de eliminar uno por uno, se "salta" de una ronda completa a la mitad del problema restante.

Complejidad: $\mathcal{O}(\log n)$ por consulta (cada ronda aprox. divide n a la mitad).

Fundamento teórico: En cada "ronda" del proceso, la mitad de los elementos son descartados/activados según la paridad de su posición. Reformulando la posición y el tamaño del problema tras cada ronda ($n' = n - \text{procesados}$, $i' = \text{nueva posición relativa}$) se llega a una recurrencia que converge en $\mathcal{O}(\log n)$ pasos en vez de $\mathcal{O}(n)$.

Ejercicio aplicado

Hay N personas en fila y un detector de metales defectuoso que se activa de forma alternada (la 1ª persona no lo activa, la 2ª sí, la 3ª no, ...; quien lo activa va al final de la fila). Dado que el coach está en la posición i , determinar en qué turno (entre quienes no activan el detector) pasará.

I/O: $T \leq 1000$ casos, cada uno con solo dos enteros $\rightarrow$ readline por línea (patrón 1) es suficiente y más simple.

Código solución

```
import sys
input = sys.stdin.buffer.readline

def safe_position(n, i):
    # turno "seguro" (que no activa el patron) en que pasara
    # la posicion i
    res = 0
    while i % 2 == 0:          # mientras i cae en la
        mitad "activada"      # cuenta los turnos seguros
        res += i // 2
        de esta ronda
        n -= i // 2            # reduce el problema a la
        mitad restante
        i = n                  # la posicion i se
        convierte en el nuevo n
    res += (i + 1) // 2        # suma los turnos seguros
    de la ronda final
    return res

def solve():
    t = int(input())
    out = []
    for _ in range(t):
        n, i = map(int, input().split())
        out.append(str(safe_position(n, i)))
```

```
sys.stdout.write("\n".join(out))
```

```
solve()
```

7.2 Fuerza Bruta / Simulación Directa (paradoja del cumpleaños)

Preguntas guía:

- ¿El tamaño de la entrada es pequeño ($N \leq 10^3$ o similar) y no hay una fórmula cerrada evidente?
- ¿Puedo calcular la respuesta iterando directamente sobre el espacio del problema sin que se dispare el tiempo?
- ¿El problema es más de matemáticas/probabilidad que de estructuras de datos?

Utilidad: Reconocer cuándo NO hace falta un algoritmo sofisticado: con restricciones pequeñas, iterar directamente (simulando la definición del problema) es la solución más simple, rápida de programar y menos propensa a errores. Identificar esto ahorra tiempo valioso en el contrarreloj.

Complejidad: Depende del problema; en este caso $\mathcal{O}(n)$ con aritmética de punto flotante.

Fundamento teórico: Se calcula directamente la probabilidad de que NO haya coincidencia de cumpleaños entre k personas, multiplicando factores $(1 - (k-1)/n)$ sucesivamente (extensión del problema del cumpleaños), deteniéndose apenas esa probabilidad cae por debajo de 0.5 (la probabilidad de coincidencia supera el 50%).

Ejercicio aplicado

Dado N (número de días del año), hallar el mínimo número de personas necesarias en una sala para que la probabilidad de que dos compartan cumpleaños supere el 50%.

I/O: Entrada de un solo entero $\rightarrow$ readline (patrón 1), no requiere nada especial.

Código solución

```
import sys
input = sys.stdin.buffer.readline

n = int(input())
prob_no_coincidencia = 1.0
for k in range(1, n + 2):
    prob_no_coincidencia *= (n - (k - 1)) / n
    if prob_no_coincidencia < 0.5:
        print(k)
        break
```

7.3 Simulación Lineal con Máximo de Pre-fijo (paredes que se acercan)

Preguntas guía:

- ¿La respuesta depende de un solo recorrido lineal del arreglo, calculando un máximo/mínimo/acumulado por posición?
- ¿No hay dependencias entre posiciones más allá de una comparación directa fila por fila?

- ¿El problema, aunque tenga una narrativa elaborada, se reduce matemáticamente a una fórmula simple tras identificar la variable clave?

Utilidad: Resolver directamente problemas donde, tras modelar bien la situación (aquí: el "cuello de botella" es la fila con menor espacio libre entre dos paredes), un solo recorrido $\mathcal{O}(n)$ calculando un máximo/mínimo basta. Es clave en competencia reconocer cuándo un problema de apariencia compleja se reduce a esto.

Complejidad: $\mathcal{O}(h)$ tiempo, $\mathcal{O}(h)$ espacio (o $\mathcal{O}(1)$ extra si se procesa en streaming).

Fundamento teórico: Las dos paredes se mueven a 1 m/s cada una. En la fila i , el espacio libre es $w - (l[i] + r[i])$ y se cierra a razón de 2 m/s (ambas paredes se acercan). El tiempo en que la fila i se cierra es $(w - l[i] - r[i]) / 2$. Las paredes chocan en la fila que se cierra primero: la de menor espacio libre inicial, equivalente al máximo de $(l[i] + r[i])$.

Ejercicio aplicado

Una sala de H filas y ancho W tiene una pared este y una pared oeste que se acercan a 1 m/s cada una desde que alguien entra. Cada fila i tiene un ancho ya ocupado por cada pared. Determinar en cuánto tiempo chocan las paredes.

I/O: $H \leq 10^5$, dos arreglos de tamaño $H \rightarrow$ lectura total + tokenización (patrón 2).

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    h = int(data[0]); w = int(data[1])
    left = list(map(int, data[2:2 + h]))
    right = list(map(int, data[2 + h:2 + 2 * h]))
    max_sum = max(left[i] + right[i] for i in range(h))
    ans = (w - max_sum) / 2

if ans.is_integer():           # verifica parseo de float a
    int                         int
    print(int(ans))
else:
    print(ans)
solve()
```

7.4 Simulación de Eventos con Colas (escala doble)

Preguntas guía:

- ¿Hay un recurso compartido con estado (dirección, disponibilidad) que cambia según el orden de llegada de eventos con timestamps?
- ¿Los eventos ya vienen ordenados (o se pueden ordenar) y conviene procesarlos con colas separadas por tipo?
- ¿El estado siguiente depende de comparar el próximo evento contra un tiempo límite (fin de uso actual del recurso)?

Utilidad: Simular un sistema con estado (aquí: una dirección de movimiento y un instante de fin de uso) que evoluciona a medida que llegan eventos de distintos tipos, usando una cola por tipo de evento en vez de simular minuto a minuto.

Complejidad: $\mathcal{O}(n)$ - cada persona se encola y desencola una sola vez.

Fundamento teórico: Se mantienen dos colas (una por dirección deseada) y un estado (dirección activa, instante en que el recurso queda libre). En cada paso se mira el evento más próximo entre ambas colas: si coincide con la dirección activa, extiende el fin de uso; si es la dirección contraria, esa persona debe esperar (se cuenta pero no se procesa aún); si ya no hay eventos antes de que el recurso quede libre, se decide si invertir la dirección (si había gente esperando) o terminar ese ciclo de uso.

Ejercicio aplicado

N personas llegan a una escalera doble (10 segundos por cruce) en tiempos distintos, cada una queriendo ir en una de dos direcciones. Determinar el instante en que la última persona termina de usarla.

I/O: $n \leq 10^4 \rightarrow$ lectura total + tokenización (patrón 2); se usan dos deque (collections.deque) en vez de queue, ya que en Python deque tiene $\mathcal{O}(1)$ real para popleft().

Código solución

```
import sys
from collections import deque

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); idx += 1
    left = deque()      # personas que quieren ir en
                        # direccion 0
    right = deque()     # personas que quieren ir en
                        # direccion 1
    for _ in range(n):
        t = int(data[idx]); d = int(data[idx + 1]); idx += 2
        (left if d == 0 else right).append(t)

    end = 0             # instante en que el recurso queda
                        # libre
    ans = 0
    waiting = 0         # cuantos esperan en direccion
                        # contraria a la activa
    direction = -1      # -1 = detenida

    while left or right or direction != -1:
        # mira el proximo evento disponible entre ambas
        # colas
        next_time, who = None, -1
        if left and (next_time is None or left[0] <
next_time):
            next_time, who = left[0], 0
        if right and (next_time is None or right[0] <
next_time):
            next_time, who = right[0], 1

        if direction == -1:
            # arranca detenida: la toma quien llegue
            # primero
            direction = who
            end = next_time + 10
            (left if who == 0 else right).popleft()
```

```

elif who != -1 and next_time < end:
    if who == direction:
        end = max(end, next_time + 10) # se
        sube mientras sigue en marcha
    else:
        waiting += 1 #
        direccion contraria: debe esperar
        (left if who == 0 else right).popleft()
    else:
        # no hay mas eventos antes de que el recurso
        quede libre
        if waiting > 0:
            direction = 1 - direction # invierte
            sentido para atender a los que esperan
            end += 10
            waiting = 0
        else:
            ans = end
            direction = -1 # se detiene

print(ans)
solve()

```

7.5 Simulación XOR con Detección de Ciclos (interruptores y bombillas)

Preguntas guía:

- ¿El estado del sistema se puede representar como un vector de bits, y cada operación disponible es un XOR con una máscara fija?
- ¿Las operaciones se aplican en un orden CÍCLICO fijo y predecible (no libre elección)?
- ¿Necesito detectar si el proceso entra en un ciclo infinito sin nunca alcanzar el estado objetivo?

Utilidad: Simular un proceso determinista donde el estado (aquí, qué bombillas están encendidas) cambia por XOR con máscaras fijas aplicadas en orden cíclico, deteniéndose al llegar al estado objetivo o detectando que nunca se alcanzará. Es aplicable a cualquier proceso de estado finito con transiciones deterministas.

Complejidad: $\mathcal{O}(N)$ en la práctica (ver fundamento teórico), a pesar de que el espacio de estados aparenta ser exponencial en M .

Fundamento teórico: Presionar los N interruptores en orden fijo $1..N$ corresponde a aplicar sucesivamente XOR con N máscaras fijas. Después de un CICLO COMPLETO (los N interruptores), el efecto neto sobre el estado es XOR con una constante fija $C = \text{XOR de todas las máscaras de los interruptores}$, sin importar el estado en que se esté. Como $v \oplus C \oplus C = v$, el estado al INICIO de cada ciclo solo puede tomar 2 valores posibles (el original, o el original $\oplus C$), alternando entre ambos. Esto acota el número de estados distintos alcanzables (y por lo tanto el número de pasos antes de repetir un estado o llegar a 0) a como mucho, $2N$: por eso la detección de ciclos con un conjunto de estados visitados nunca necesita más de un puñado de iteraciones, sin importar cuán grande sea M .

Ejercicio aplicado

Un conserje presiona N interruptores en orden cíclico $1, 2, \dots, N, 1, 2, \dots$. Cada interruptor alterna un subconjunto fijo de M bombillas. Dado el estado inicial de las bombillas encendidas, determinar cuántas veces hay que

presionar interruptores hasta que TODAS las bombillas queden apagadas simultáneamente, o -1 si nunca ocurre.

I/O: $N, M \leq 1000 \rightarrow$ readline por línea (patrón 1) es más que suficiente, dado que el propio algoritmo termina en a lo más $\sim 2N$ iteraciones (verificado: $< 0.03s$ incluso en el peor caso $N=M=1000$).

Código solución

```

import sys
input = sys.stdin.buffer.readline

n, m = map(int, input().split())
ldata = list(map(int, input().split())) # ldata[0]=
        cantidad, resto=bombillas encendidas

switch = []
for i in range(n):
    sdata = list(map(int, input().split()))
    tmp = 0
    for j in range(1, sdata[0] + 1):
        aux = 1 << (m - sdata[j]) # bit que
        representa esa bombilla
        tmp += aux
    switch.append(tmp) # mascara XOR
        de este interruptor

bulbs = 0
for j in range(1, ldata[0] + 1):
    aux = 1 << (m - ldata[j])
    bulbs += aux # estado
        inicial como bitmask

idx = 0
res = 0
seen = set() # detecta si (
        estado, interruptor actual) se repite
while True:
    if bulbs == 0:
        break # todas las
        bombillas apagadas: listo
    if (bulbs, idx) in seen:
        res = -1 # ya visitamos
        este estado exacto: nunca terminara
        break
    seen.add((bulbs, idx))
    info = switch[idx]

    bulbs ^= info # aplica el
        interruptor actual

    if idx + 1 == n:
        idx = 0 # vuelve al
        primer interruptor (orden ciclico)
    else:
        idx += 1
        res += 1

sys.stdout.write(str(res))

```

7.6 Invariante de Paridad por Ciclos de Permutación (rompecabezas deslizante)

Preguntas guía:

- ¿Me preguntan si una configuración es ALCANZABLE mediante ciertos movimientos, sin pedir la secuencia de movimientos?

- ¿Puedo encontrar una cantidad que NO cambie con ningún movimiento válido (un invariante), y comparar su valor inicial con el del estado objetivo?
- ¿El espacio de estados es astronómico, de modo que una búsqueda (BFS/DFS) es imposible y hace falta un argumento matemático?
- ¿Hay casos degenerados donde el invariante deja de ser suficiente y el problema se vuelve otro?

Utilidad: Decide alcanzabilidad sin explorar el espacio de estados, buscando una función invariante bajo los movimientos permitidos: si dos configuraciones difieren en su valor, ninguna sucesión de movimientos lleva de una a la otra. Es la técnica para problemas de "¿se puede llegar de A a B?" con operaciones definidas: rompecabezas deslizantes, fichas que se intercambian, coloreos, monedas que se voltean de a pares. Aquí el invariante es la paridad de la permutación combinada con la paridad de la distancia del hueco a su posición final.

Complejidad: $\mathcal{O}(mn)$ por caso de prueba, con una sola pasada para la descomposición en ciclos.

Fundamento teórico: Cada movimiento intercambia el hueco con una pieza vecina, es decir, aplica una TRANSPOSICIÓN a la permutación, lo que invierte su paridad. A la vez, el hueco se mueve una casilla en horizontal o vertical, lo que cambia en exactamente 1 su distancia Manhattan a la esquina inferior derecha, invirtiendo también esa paridad. Por lo tanto la SUMA de ambas paridades no cambia nunca: es un invariante. En la configuración objetivo la permutación es la identidad (paridad par) y el hueco está en la esquina (distancia 0, par), así que la suma vale 0; una configuración es resoluble solo si su suma es par, y para tableros con al menos dos filas y dos columnas esa condición además es suficiente. Para calcular la paridad de la permutación conviene evitar contar inversiones: toda permutación se descompone de forma única en ciclos disjuntos, y un ciclo de longitud ℓ es producto de $\ell - 1$ transposiciones, así que la paridad es $(\sum (\ell_i - 1)) \bmod 2 = (N - \# \text{ciclos}) \bmod 2$, calculable en una pasada lineal siguiendo cada ciclo, sin estructuras auxiliares ni $\mathcal{O}(N \log N)$. Hay un caso degenerado que el argumento del enunciado no cubre: cuando el tablero tiene UNA sola fila o UNA sola columna, un movimiento sigue invirtiendo ambas paridades, pero la condición deja de ser suficiente, porque las piezas solo pueden desplazarse y nunca cambian su orden relativo. En ese caso la configuración es resoluble si y solo si las piezas, ignorando el hueco, ya están en orden creciente. Esto se verificó explorando con BFS todas las configuraciones alcanzables: en un tablero 1×4 solo 4 de las 24 permutaciones son resolubles, mientras que el criterio de paridad declararía resolubles a 12.

Ejercicio aplicado

Dadas varias configuraciones de rompecabezas deslizantes de $m \times n$ (la pieza mn representa el hueco), decidir para cada una si puede llevarse, deslizando piezas, a la configuración ordenada $1, 2, \dots, mn$.

I/O: Varios casos hasta agotar la entrada, cada uno con m, n y luego mn números, con $m \cdot n \leq 10^5$ y cantidad de casos no acotada $\Rightarrow$ lectura total + tokenización con índice (patrón 2 combinado con el 4: se consumen casos mientras queden tokens). Verificado contra los cuatro casos de ejemplo y, exhaustivamente, contra un BFS que explora todas las configuraciones realmente alcanzables desde la posición resuelta en 14 tableros (1×2 hasta 3×3 , incluyendo $1 \times n$ y $n \times 1$), cubriendo las 362 880 permutaciones del 3×3 , sin discrepancias. Tiempos medidos: $\sim 0.04s$ con un único caso de 10^5 piezas, $\sim 0.27s$ con diez casos máximos (5.7 MB de entrada), $\sim 0.05s$ con 20 000 casos pequeños y $\sim 0.03s$ con una fila de 10^5 piezas.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    pos = 0
    total = len(data)
    out = []
    while pos + 1 < total:
        # se consumen
        # casos hasta agotar la entrada
        m = int(data[pos]); n = int(data[pos+1]); pos += 2
        N = m*n
        p = list(map(int, data[pos:pos+N])); pos += N
        if m == 1 or n == 1:
            # una sola fila (o columna): las piezas solo
            # se desplazan, nunca
            # cambian de orden relativo -> solo sirve si
            # ya estan ordenadas
            resto = [v for v in p if v != N]
            out.append("Y" if resto == list(range(1, N))
                       else "N")
            continue
        # paridad de la permutacion = (N - numero de
        # ciclos) mod 2,
        # porque un ciclo de largo L son L-1
        # transposiciones
        visto = bytearray(N + 1)
        ciclos = 0
        for i in range(1, N + 1):
            if visto[i]:
                continue
            ciclos += 1
            j = i
            while not visto[j]:
                # recorre el
                # ciclo que contiene a i
                visto[j] = 1
                j = p[j - 1]
            par = (N - ciclos) & 1
            idx = p.index(N)
            # posicion del
            # hueco (la pieza mn)
            dist = (m - 1 - idx // n) + (n - 1 - idx % n)
            # distancia Manhattan a la esquina
            # cada movimiento invierte AMBAS paridades: su
            # suma es invariante
            out.append("Y" if not ((par + dist) & 1) else "N")
        sys.stdout.write("\n".join(out) + "\n")

solve()
```

7.7 Descomposición en Fracciones Unitarias (Búsqueda Acotada tipo Erdős-Straus)

Preguntas guía:

- ¿Me piden escribir una fracción como suma de un número FIJO de fracciones unitarias $1/x$?
- ¿Se permiten denominadores repetidos? (eso habilita identidades triviales como $1/q = 1/2q + 1/2q$)
- ¿Puedo acotar el primer denominador y reducir el problema a descomponer el resto en menos términos?
- ¿Los límites del enunciado son chicos, de modo que una búsqueda acotada con verificación exacta es preferible a buscar una fórmula cerrada?

Utilidad: Resuelve problemas de representación de racionales como suma de fracciones unitarias (fracciones egipcias) fijando el primer denominador dentro de un rango acotado y reduciendo el resto a un subproblema con un término menos. La idea general — acotar la primera elección por monotonía, restar y recurrir — sirve para cualquier descomposición de este tipo, y en particular para el caso $4/n$ de la conjetura de Erdős-Straus, que no tiene fórmula cerrada conocida pero sí solución para todo n del rango pedido.

Complejidad: $\mathcal{O}(1)$ amortizado por caso en la práctica: el peor n del rango completo se resolvió en 0.21 ms, y las respuestas se memorizan por si n se repite.

Fundamento teórico: Como $1/x$ es el mayor de los tres sumandos o al menos uno de ellos lo es, se puede suponer $1/x \geq \frac{1}{3} \cdot \frac{4}{n}$, y como además $1/x < 4/n$, el primer denominador cumple $\lceil n/4 \rceil \leq x \leq \lceil 3n/4 \rceil$: un rango chico donde basta recorrer. Fijado x , queda $\frac{p}{q} = \frac{4}{n} - \frac{1}{x}$ con $p = 4x - n$ y $q = nx$, y se reduce la fracción. Ahí aparecen tres atajos que explotan que los denominadores pueden repetirse: si $p = 0$, ya se tiene $4/n = 1/x$ y basta partirlo con $\frac{1}{x} = \frac{1}{2x} + \frac{1}{4x} + \frac{1}{4x}$; si $p = 1$, el resto $1/q$ se parte en $\frac{1}{2q} + \frac{1}{2q}$; y si $p = 2$, el resto $2/q$ es directamente $\frac{1}{q} + \frac{1}{q}$. Solo cuando $p \geq 3$ hace falta buscar: se necesita $\frac{p}{q} = \frac{1}{y} + \frac{1}{z}$, y como $1/y < p/q$ obliga a $y > q/p$, mientras que $1/y \geq \frac{1}{2} \cdot \frac{p}{q}$ obliga a $y \leq 2q/p$, el candidato y vive en un rango acotado; para cada uno, $\frac{1}{z} = \frac{p}{q} - \frac{1}{y} = \frac{py-q}{qy}$, así que sirve exactamente cuando $py - q$ divide a qy . Empíricamente el primer y útil aparece muy pronto, por eso alcanza con explorar unos cientos de candidatos por cada x . Vale notar qué tan chico es el caso realmente difícil: con identidades directas se cubren todos los n salvo los que cumplen $n \equiv 1 \pmod{12}$ (833 de los 9998 del rango), que son justamente los casos duros de la conjetura, y para ellos la búsqueda encontró solución en todos los casos.

Ejercicio aplicado

Para cada n de la entrada ($2 \leq n < 10^4$, terminando en 0), hallar enteros positivos x, y, z menores que 10^{16} tales que $\frac{4}{n} = \frac{1}{x} + \frac{1}{y} + \frac{1}{z}$.

I/O: Varios casos, uno por línea, con centinela $n = 0$ (patrón 3), leídos con lectura total + tokenización porque son números sueltos. Las respuestas se memorizan en un

diccionario por si un mismo n aparece muchas veces, ya que la cantidad de casos no está acotada. Toda la aritmética es entera: la verificación usa divisibilidad exacta, nunca flotantes, que a esta escala darían falsos positivos. Verificado EXHAUSTIVAMENTE para los 9998 valores posibles de n , comprobando con aritmética exacta de fracciones que $\frac{1}{x} + \frac{1}{y} + \frac{1}{z}$ es idénticamente $\frac{4}{n}$ y que los tres denominadores son positivos y menores que 10^{16} : sin fallos y sin casos sin solución. Tiempos: $\sim 0.03s$ resolviendo los 9998 valores distintos, $\sim 0.15s$ con 200 000 casos aleatorios, y $\sim 0.14s$ con 200 000 casos restringidos a los duros ($n \equiv 1 \pmod{12}$).

Código solución

```
import sys
from math import gcd

def solve():
    data = sys.stdin.buffer.read().split()
    memo = {}
    out = []
    for tok in data:
        n = int(tok)
        if n == 0:
            break
        r = memo.get(n)
        if r is None:
            x0 = -(-n // 4)  # techo de n
            # 4/n < 4/n obliga x > n/4
            for x in range(x0, x0 + 300):
                p = 4*x - n; q = n*x  # falta p/q
                # 4/n - 1/x
                if p == 0:  # 4/n = 1/x
                    exacto: se parte en tres
                    r = (2*x, 4*x, 4*x); break
                g = gcd(p, q); p //= g; q //= g
                if p == 1:  # 1/q = 1/(2
                    q) + 1/(2q)
                    r = (x, 2*q, 2*q); break
                if p == 2:  # 2/q = 1/q
                    + 1/q
                    r = (x, q, q); break
                y0 = q // p + 1  # 1/y < p/q
                # 1/y >= (p/q)/2 acota por arriba: y <= 2
                q/p
                for y in range(y0, min(y0 + 400, 2*q // p
                    + 1)):
                    d = p*y - q  # 1/z = p/q
                    - 1/y = d/(q*y)
                    if d > 0 and (q*y) % d == 0:
                        r = (x, y, q*y // d); hallado =
1; break
                if hallado: break
            memo[n] = r  # la entrada
            # puede repetir el mismo n
            out.append("%d %d %d" % r)
    sys.stdout.write("\n".join(out) + "\n")

solve()
```

8 Construcción y Combinatoria

8.1 Constructivo por Invariante de Paridad (permutación con signos)

Preguntas guía:

- ¿El problema pide construir explícitamente una configuración (o probar que es imposible), en vez de optimizar o contar?
- ¿Puedo idear una construcción simple (simetrías, intercambios, patrones repetidos) en vez de buscar entre todas las opciones?
- ¿La imposibilidad de otros valores se puede demostrar con un invariante (paridad, módulo, suma fija) que ninguna elección puede romper?

Utilidad: Resolver problemas donde no hace falta ningún algoritmo de búsqueda: basta con proponer una construcción explícita y justificar, con un argumento de invariante, que el resultado contrario (o cualquier otro valor puntual) nunca puede ocurrir sin importar qué elecciones se hagan.

Complejidad: $\mathcal{O}(n)$ por caso - construir y emitir la respuesta.

Fundamento teórico: Se particiona $\{1, \dots, n\}$ en pares consecutivos $(2k-1, 2k)$ y se intercambian entre sí. Cada par contribuye al contador un valor relacionado con $V_k = (2k-1)(2k)$, que siempre es par; y ese aporte entra a la suma final como $-V_k$, 0 o V_k (las tres opciones son pares). Como la suma de términos pares es par, el resultado nunca puede ser impar - en particular, nunca puede ser 1.

Ejercicio aplicado

(Codeforces 2246A — farmpiggy and Subset Sum) Dado un entero par n , construir una permutación p de longitud n tal que, sin importar qué signo (sumar, restar o ignorar) se le asigne a cada término $i \cdot p_i$ al acumular un contador, el resultado final nunca pueda ser exactamente 1.

I/O: $t \leq 25, n \leq 50$ (par) $\rightarrow$ entrada pequeña, readline por línea (patrón 1) es suficiente.

Código solución

```
import sys
input = sys.stdin.buffer.readline

t = int(input())
out = []
for _ in range(t):
    n = int(input())
    p = []
    for i in range(1, n + 1):
        # empareja (1,2), (3,4), ..., (n-1,n) e
        # intercambia cada pareja:
        # el aporte de cada pareja al contador siempre es
        # par (ver teoría)
        if i % 2 == 1:
            p.append(i + 1)      # posición impar ->
            # apunta a la siguiente
        else:
            p.append(i - 1)      # posición par -> apunta
            # a la anterior
    out.append(' '.join(map(str, p)))
```

```
sys.stdout.write('\n'.join(out) + '\n')
```

8.2 Constructivo por Construcción Explícita (suma divisible por cada elemento)

Preguntas guía:

- ¿Piden construir explícitamente n valores con una relación de divisibilidad o proporción exacta entre ellos y su suma?
- ¿La condición se puede reescribir como una ecuación entre fracciones que deben sumar exactamente 1?
- ¿Existe una descomposición numérica conocida (como una fracción egipcia) que se pueda extender o escalar para cualquier n ?

Utilidad: Construir directamente arreglos o valores que cumplan una condición de divisibilidad o proporción exacta, usando descomposiciones numéricas conocidas en vez de búsquedas o heurísticas. Es frecuente en problemas "constructivos" donde la solución es una fórmula cerrada, no un algoritmo de recorrido.

Complejidad: $\mathcal{O}(n)$ por caso.

Fundamento teórico: Si cada $a_i = S / k_i$ (S = suma total), la condición " S divisible por cada a_i " equivale a que los k_i sean enteros positivos distintos con $\sum(1/k_i) = 1$. Partiendo de la identidad $1 = 1/2 + 1/3 + 1/6$ y extendiendo el término $1/6$ con una cadena de potencias de 2, se obtiene una familia de k_i distintos para cualquier $n \geq 3$: la construcción cerrada $\{1, 2, 3, 6, 12, 24, \dots, 3 \cdot 2^{n-3}\}$. Los casos $n=1$ y $n=2$ se resuelven aparte (trivial e imposible, respectivamente).

Ejercicio aplicado

(Codeforces 2246B — ezraft and Array) Dado n , construir un arreglo de n enteros positivos distintos tal que la suma total sea divisible por cada uno de sus elementos, o determinar que no es posible.

I/O: $t \leq 50, n \leq 50 \rightarrow$ entrada pequeña, readline por línea (patrón 1).

Código solución

```
import sys
input = sys.stdin.buffer.readline

t = int(input())
out = []
for _ in range(t):
    n = int(input())
    if n == 1:
        out.append('1')          # caso base: un
        # unico elemento
    elif n == 2:
        out.append('-1')         # a1/a2 y a2/a1
        # con ambos positivos    # obliga a1 ==
        # a2 (no son distintos)
    else:
        # construccion cerrada a partir de 1 = 1/2 + 1/3
        # + 1/6:
        # {1, 2} mas una cadena de potencias de 2
        # escaladas por 3,
        # que nunca "chocan" con los divisores de 1 o 2
        vals = [1, 2] + [3 * (1 << k) for k in range(n - 2)]
```

```
out.append(' '.join(map(str, vals)))
sys.stdout.write('\n'.join(out) + '\n')
```

8.3 Conteo por Simetría de Binomios (subsecuencias con suma alternada 0)

Preguntas guía:

- ¿Piden contar subconjuntos o subsecuencias que cumplan una condición de suma o balance?
- ¿Hay valores repetidos en la entrada, de forma que agruparlos por valor simplifique el conteo?
- ¿Se puede usar $C(c, \text{par}) = C(c, \text{impar}) = 2^{c-1}$ para separar el conteo por grupo y combinar con una potencia de 2?

Utilidad: Contar eficientemente subconjuntos o subsecuencias que satisfacen una condición combinatoria, evitando enumerar los 2^n subconjuntos, aprovechando la simetría de los coeficientes binomiales para reducir el problema a una subsecuencia mucho más pequeña: solo los valores distintos.

Complejidad: $\mathcal{O}(n)$ por caso (la entrada ya viene no-decreciente).

Fundamento teórico: Dentro de un grupo de c copias iguales, elegir una cantidad par de ellas no cambia la suma alternada acumulada (aporta 0) y elegir una cantidad impar sí la cambia (aporta $\pm \text{valor}$). Como $C(c, \text{par}) = C(c, \text{impar}) = 2^{c-1}$, cada grupo aporta el mismo peso sin importar el modo elegido, así que el problema se reduce a contar subconjuntos de suma alternada 0 sobre los valores DISTINTOS, multiplicando el resultado por 2^{n-m} al final (m = cantidad de valores distintos). En esa secuencia reducida, agrupando términos de dos en dos se demuestra que la única forma de obtener alternada 0 con más de un elemento es usando exactamente 3 valores: -1 y un par de enteros positivos consecutivos.

Ejercicio aplicado

(Codeforces 2246C — Omar and Alternating Sums) Dado un arreglo no decreciente de n valores (cada uno -1 o un entero positivo), contar cuántas subsecuencias tienen suma alternada (primer término positivo, segundo negativo, tercero positivo, ...) igual a 0, módulo $10^9 + 7$.

I/O: $t \leq 10^4$, $\sum n \leq 2 \cdot 10^5 \rightarrow$ volumen moderado; se usa readline por línea (patrón 1), adecuado porque el costo dominante está en el procesamiento $\mathcal{O}(n)$, no en la lectura.

Código solución

```
import sys
input = sys.stdin.buffer.readline

MOD = 10**9 + 7

t = int(input())
out = []
for _ in range(t):
    n = int(input())
    a = list(map(int, input().split()))

    # el arreglo ya viene no-decreciente: colapsar a
    # valores distintos.
```

```
# cada grupo de c repetidos aporta un factor 2^(c-1)
# independiente
# del resto (C(c,par) == C(c,impar) == 2^(c-1))
distinct = []
for x in a:
    if not distinct or distinct[-1] != x:
        distinct.append(x)

m = len(distinct)
has_neg1 = 1 if distinct[0] == -1 else 0 # -1 es el
# unico valor <= 0 posible

pos = [v for v in distinct if v > 0]
count_consec = 0
for i in range(len(pos) - 1):
    if pos[i + 1] - pos[i] == 1: # pares
        # de enteros positivos consecutivos
        count_consec += 1

# en la secuencia reducida, la suma alternada solo
# puede dar 0 con:
# el subconjunto vacio, o exactamente {-1, v, v+1}
# para cada pareja
# consecutiva (si hay un -1 disponible)
reduced = (1 + (count_consec if has_neg1 else 0)) %
MOD
ans = reduced * pow(2, n - m, MOD) % MOD #
# reincorpora los repetidos
out.append(str(ans))

sys.stdout.write('\n'.join(out) + '\n')
```

8.4 Greedy por Alternancia con Límite de Racha (dónde dormir cada noche)

Preguntas guía:

- ¿Hay que construir explícitamente una secuencia que respete un límite de repeticiones CONSECUTIVAS de cada elemento (sin límite en el total de usos)?
- ¿Un elemento puede reutilizarse libremente después de "descansar" al menos un turno?
- ¿Basta con dos "recursos" (el mejor y el segundo mejor) para romper cualquier racha, sin necesidad de planear la secuencia completa de antemano?

Utilidad: Construir una secuencia válida de longitud D usando N opciones con límites de racha consecutiva, sin necesidad de una búsqueda ni un algoritmo sofisticado: alternar entre las DOS opciones con mayor límite es siempre suficiente (si el problema es factible), porque cualquier racha se puede "romper" con un solo turno de la segunda opción antes de retomar la primera.

Complejidad: $\mathcal{O}(D)$.

Fundamento teórico: El problema es imposible si ningún lugar admite ni una sola noche (todos los límites son 0), o si solo UN lugar tiene límite positivo y ese límite es menor que D (no hay forma de "descansar" de él). En cualquier otro caso (al menos dos lugares con límite > 0), siempre es posible: se usa el lugar de mayor límite hasta agotar su racha, se intercala un solo día en el segundo mejor lugar (lo cual reinicia la racha del primero), y se repite este patrón hasta completar los D días. Este patrón simple es suficiente porque no hay ningún límite en el

número TOTAL de veces que se puede volver a un lugar, solo en la racha consecutiva.

Ejercicio aplicado

Kai necesita dormir D días seguidos, alternando entre N lugares, donde el lugar i tolera a lo más d_i noches CONSECUTIVAS antes de expulsarlo (pero puede volver después). Construir un plan válido día a día, o determinar que es imposible.

I/O: $N, D \leq 10^5 \rightarrow$ lectura total con tokenización (patrón 2). Verificado contra 200 casos aleatorios comparando factibilidad y validando la secuencia construida.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); d = int(data[1])
    arr = list(map(int, data[2:2 + n]))

    order = sorted(range(n), key=lambda i: -arr[i])
    best_idx = order[0]
    best_val = arr[best_idx]

    if best_val == 0:
        # ningun lugar
        # es utilizable
        print(-1)
        return

    second_val = arr[order[1]] if n > 1 else 0

    if second_val == 0:
        # solo hay UN lugar utilizable: hay que quedarse
        # ahí todos los D dias seguidos
        if best_val >= d:
            print(' '.join([str(best_idx + 1)] * d))
        else:
            print(-1)
        return

    second_idx = order[1]
    out = []
    remaining = d
    while remaining > 0:
        if remaining == 1:
            out.append(second_idx + 1)
            # un dia
            # de "descanso" en el segundo mejor lugar
            break

        out.append(second_idx + 1)
        # un dia de "
        # descanso" en el segundo mejor lugar
        remaining -= 1

        take = min(best_val, remaining)
        out.extend([best_idx + 1] * take)
        remaining -= take
    sys.stdout.write(' '.join(map(str, out)) + '\n')

solve()
```

8.5 Stars and Bars con Factoriales Precalculados (días de espera entre oasis)

Preguntas guía:

- ¿Debo distribuir una cantidad fija de elementos idénticos entre varios lugares?
- ¿Cada lugar puede recibir cero o más elementos?

- ¿El orden dentro de cada grupo no importa, únicamente cuántos elementos recibe cada uno?

Utilidad: Contar el número de formas de distribuir una cantidad fija de elementos indistinguibles entre varios grupos independientes. Es una de las aplicaciones más comunes de la técnica de *Stars and Bars* (estrellas y barras), utilizada frecuentemente en problemas de combinatoria, programación competitiva y teoría de conteo.

Complejidad: Preprocesamiento de factoriales $\mathcal{O}(MAX)$; cada consulta $\mathcal{O}(1)$; espacio $\mathcal{O}(MAX)$.

Fundamento teórico: Si el viaje mínimo hasta el oasis destino requiere O días y debe durar exactamente T días, existen $extra = T - O$ días de espera que pueden distribuirse libremente antes de salir o después de cualquiera de los D oasis recorridos, es decir, entre $D + 1$ posiciones distintas. Como los días de espera son indistinguibles y cada posición puede recibir cualquier cantidad de ellos, el problema se reduce directamente al principio de *Stars and Bars*, obteniendo un total de $\binom{extra+D}{D}$ planes posibles.

Ejercicio aplicado

Se conocen las posiciones de todos los oasis de una ruta. Para cada consulta se recibe un oasis destino y el número exacto de días que debe durar el viaje. Calcular cuántos planes distintos existen para distribuir los días de espera manteniendo siempre el mismo recorrido.

I/O: $5 \leq N \leq 20$, $1 \leq M \leq 10$. Las distancias a los oasis no superan los 60 días, por lo que todos los factoriales necesarios pueden precomputarse una única vez antes de responder las consultas; cada consulta se responde en tiempo constante.

Código solución

```
import sys
input = sys.stdin.buffer.readline
cant, queries = map(int, input().split()) # numero de
# oasis y consultas
oasis = list(map(int, input().split())) # distancia
# minima hasta cada oasis

MAX = 80
factorial = [1] * (MAX + 1) # factoriales para responder
# combinaciones en O(1)

for i in range(1, MAX + 1):
    factorial[i] = factorial[i - 1] * i

def combination(n, k):
    return factorial[n] // (factorial[k] * factorial[n - k])

for _ in range(queries):
    destiny, time = map(int, input().split())
    extraTime = time - oasis[destiny - 1] # dias extra
    # que deben distribuirse como esperas
    res = combination(extraTime + destiny, destiny) #
    # Stars and Bars
    sys.stdout.write(str(res) + "\n")
```

9 Algoritmos de Cadenas (Strings)

9.1 Rotación Mínima, Booth (huellas duplicadas por rotación)

Preguntas guía:

- ¿Dos strings se consideran "iguales" si uno es una rotación cíclica del otro?
- ¿Necesito una forma canónica única para comparar/deduplicar strings bajo rotación, en tiempo lineal (no $\mathcal{O}(n^2)$) probando todas las rotaciones)?
- ¿El string puede ser largo (miles de caracteres) y hay muchos para procesar?

Utilidad: Encontrar la rotación lexicográficamente menor de un string en $\mathcal{O}(n)$, como forma canónica para comparar o deduplicar strings equivalentes bajo rotación cíclica. Evita el enfoque ingenuo de generar y comparar las n rotaciones ($\mathcal{O}(n^2)$).

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico: El algoritmo de Booth adapta la función de fallo (como en KMP) sobre el string duplicado ($s + s$). Mantiene un índice candidato k a la rotación mínima y, al procesar cada carácter, usa la función de fallo para saltar comparaciones ya conocidas, actualizando k solo cuando encuentra evidencia de que otra posición produce una rotación menor. El resultado equivale a comparar todas las rotaciones en un solo recorrido lineal. [23]

Ejercicio aplicado

Dado un conjunto de huellas dactilares codificadas como strings (equivalentes bajo rotación cíclica), eliminar duplicados representando cada grupo por su rotación canónica (la lexicográficamente menor).

I/O: Hasta 100 strings de hasta 5000 caracteres por caso, múltiples casos hasta el centinela "0" → lectura total con `sys.stdin.read().split()` (patrón 2). Verificado: ~1.4s para 5 casos $\times$ 100 strings $\times$ 5000 caracteres, y correcto contra fuerza bruta en 3000 strings aleatorios.

Código solución

```
import sys

def least_rotation(s):
    # índice de inicio de la rotación lexicográficamente
    # menor (algoritmo de Booth)
    ss = s + s
    n = len(ss)
    fail = [-1] * n
    k = 0
    for j in range(1, n):
        sj = ss[j]
        i = fail[j - k - 1]
        while i != -1 and sj != ss[k + i + 1]:
            if sj < ss[k + i + 1]:
                k = j - i - 1
                i = fail[i]
            if sj != ss[k + i + 1]:
                if sj < ss[k + i + 1]:
                    k = j
                fail[j - k] = -1
            else:
                fail[j - k] = i + 1
        return k
```

```
def canonical(s):
    k = least_rotation(s)
    return s[k:] + s[:k]

def solve():
    data = sys.stdin.read().split()
    idx = 0
    out = []
    while idx < len(data):
        n = int(data[idx]); idx += 1
        if n == 0:
            break
        seen = set()
        for _ in range(n):
            m = int(data[idx]); f = data[idx + 1]; idx += 2
            c = canonical(f)
            if c not in seen:
                # dedup
                # preservando orden de aparición
                seen.add(c)
                out.append(c)
        sys.stdout.write("\n".join(out) + "\n")

solve()
```

9.2 Normalización + Comparación con el Reverso (palíndromos)

Preguntas guía:

- ¿Piden verificar si un texto es un palíndromo ignorando mayúsculas, espacios y puntuación?
- ¿Basta con normalizar el string (filtrar solo los caracteres relevantes, unificar case) y comparar contra su reverso?
- ¿El caso de un string vacío (tras filtrar) debe considerarse un palíndromo válido?

Utilidad: Resolver directamente problemas de verificación de palíndromos con reglas de normalización, filtrando primero y comparando el resultado contra su versión invertida.

Complejidad: $\mathcal{O}(n)$.

Fundamento teórico: Un string es palíndromo si es igual a su reverso. Al normalizar primero (quedarse solo con letras, todas en minúscula), la comparación directa `filtered == filtered[::-1]` captura exactamente la definición pedida, incluyendo el caso borde de que un string vacío (o sin letras) es trivialmente un palíndromo.

Ejercicio aplicado

Dada una oración, determinar si es un palíndromo ignorando mayúsculas, espacios, y cualquier carácter que no sea una letra del alfabeto inglés.

I/O: Múltiples líneas hasta el centinela "*" (patrón 3), $N < 1000$ por línea → se recorre línea por línea con `for line in sys.stdin`, ya que es texto libre (no se puede tokenizar con `.split()`).

Código solución

```
import sys

def solve():
    out = []
    for raw in sys.stdin:
        line = raw.rstrip('\n').rstrip('\r')
```

```

if line == '*':
    break
filtered = [c.lower() for c in line if c.isalpha()]
out.append('Y' if filtered == filtered[:-1] else 'N')
sys.stdout.write("\n".join(out) + "\n")

solve()

```

9.3 KMP sobre Codificación en Preorden (subárboles con la misma forma)

Preguntas guía:

- ¿Piden contar/encontrar nodos de un árbol cuyo subárbol tiene la MISMA FORMA que otro árbol patrón, ignorando por completo las etiquetas (valores, nombres de variable, operadores)?
- ¿Los árboles son binarios completos (todo nodo interno tiene exactamente 2 hijos) y el orden izquierda/derecha de los hijos SÍ importa para la comparación?
- ¿Los árboles vienen dados como texto con una gramática recursiva fácil de recorrer linealmente (ej. expresiones totalmente parentizadas)?
- ¿|patrón| y |texto| son grandes (decenas o cientos de miles), descartando comparar subárbol por subárbol de forma ingenua?

Utilidad: Reduce un problema de comparación estructural de árboles (¿este subárbol tiene la misma forma que aquel otro?) a un problema clásico de coincidencia de patrones en cadenas. Es aplicable a cualquier familia de árboles binarios completos donde solo interesa la forma (posición y arity de los nodos), no las etiquetas: sirve tanto para árboles de expresión como para cualquier otra estructura jerárquica que se pueda serializar de la misma manera.

Complejidad: $\mathcal{O}(|\alpha| + |\beta|)$ por caso de prueba, tanto para la codificación como para la búsqueda de coincidencias con KMP.

Fundamento teórico: Todo árbol binario completo (cada nodo interno tiene exactamente 2 hijos, las hojas 0) admite una codificación en preorden como una cadena binaria: cada hoja emite un símbolo '0' y cada nodo interno emite un símbolo '1' seguido de la codificación de su hijo izquierdo y luego la de su derecho. Esta codificación es una biyección: dos árboles son estructuralmente idénticos si y solo si sus cadenas codificadas son idénticas carácter a carácter. Además, como el recorrido es en preorden, la codificación del subárbol enraizado en cualquier nodo v es exactamente una SUBCADENA CONTIGUA de la codificación completa, que comienza en la posición de v y cuya longitud es el tamaño de ese subárbol — esto se cumple porque el proceso de decodificación es determinista (depende únicamente de la secuencia de bits leída, nunca de lo que venga después), así que si dos subcadenas de igual longitud L son iguales carácter a carácter, sus procesos de decodificación son idénticos paso a paso y por

tanto ambas "cierran" el árbol exactamente tras consumir L símbolos, produciendo árboles idénticos. Como además cada nodo del árbol contribuye exactamente un símbolo en el recorrido preorden, TODAS las posiciones de la cadena de texto corresponden a un nodo válido, así que contar nodos v cuyo subárbol coincide con el patrón α se reduce exactamente a contar cuántas veces aparece (con solapamiento permitido) la cadena codificada de α dentro de la cadena codificada de β , lo cual se resuelve en tiempo lineal con el autómata de fallos de Knuth-Morris-Pratt (KMP), reanudando la búsqueda en `fail[m-1]` tras cada coincidencia para permitir solapamientos. [24]

Ejercicio aplicado

Dadas expresiones aritméticas α y β totalmente parentizadas (operadores binarios, variables de una letra, números), contar cuántos nodos del árbol de sintaxis de β tienen un subárbol con la misma forma que el árbol de α , ignorando etiquetas (variables, números y operadores).

I/O: Varios casos de prueba hasta EOF, cada uno de dos líneas sin espacios internos, con $1 \leq |\alpha|, |\beta| \leq 400000 \rightarrow$ como ninguna expresión contiene blancos, se usa lectura total + tokenización (patrón 2: `sys.stdin.buffer.read().split()`), donde cada token es directamente una expresión completa. La codificación se hace con un único barrido lineal por carácter (sin recursión ni pila, evitando cualquier límite de profundidad de Python) y la búsqueda usa KMP clásico. Verificado contra los ejemplos dados (salidas 3 y 2) y contrastado con un verificador de fuerza bruta (parseo real del árbol + comparación de formas) en 300 casos aleatorios pequeños, sin discrepancias. Con un caso cercano al peor caso teórico ($|\alpha| = |\beta| \approx 400000$) el tiempo medido fue de 0.11s, y con varios casos de prueba encadenados (texto total >1.6 MB) 0.2s, muy por debajo de límites típicos.

Código solución

```

import sys

def encode(tok):
    # codifica la expresion (bytes) en preorden: '1' =
    # nodo interno, '0' = hoja
    # un solo barrido lineal, sin recursion ni pila
    out = bytearray()
    i, n = 0, len(tok)
    ap = out.append
    while i < n:
        c = tok[i]
        if c == 40:                # '(' -> abre un nodo
            interno
            ap(49)                  # emite '1'
            i += 1
        elif c == 41:              # ')' -> solo delimita,
            no aporta simbolo
            i += 1
        elif c == 43 or c == 45 or c == 42 or c == 47: #
            + - * / (operador)
            i += 1
        elif 97 <= c <= 122:       # variable (una sola
            letra a-z)
            ap(48)                  # emite '0' (hoja)
            i += 1
        else:                      # numero: puede tener
            varios digitos, es UNA sola hoja

```

```

        ap(48)
        i += 1
        while i < n and 48 <= tok[i] <= 57:
            i += 1
    return bytes(out)

def kmp_count(pattern, text):
    # cuenta ocurrencias de pattern en text, permitiendo
    # solapamiento
    m = len(pattern)
    if m == 0 or m > len(text):
        return 0
    fail = [0] * m
    k = 0
    for i in range(1, m):
        while k > 0 and pattern[i] != pattern[k]:
            k = fail[k - 1]
        if pattern[i] == pattern[k]:
            k += 1
        fail[i] = k
    count = 0
    k = 0
    for c in text:
        while k > 0 and c != pattern[k]:
            k = fail[k - 1]
        if c == pattern[k]:
            k += 1
        if k == m:
            count += 1
            k = fail[k - 1] # permite coincidencias
    # solapadas
    return count

def solve():
    data = sys.stdin.buffer.read().split()
    out = []
    for i in range(0, len(data), 2):
        alpha, beta = data[i], data[i + 1]
        out.append(str(kmp_count(encode(alpha), encode(
            beta))))
    sys.stdout.write("\n".join(out) + "\n")

solve()

```

9.4 Suffix Array + LCP de Kasai (subcadena repetida más larga)

Preguntas guía:

- ¿Me piden la subcadena más larga que aparece al menos dos veces en un texto (permitiendo solapamiento)?
- ¿Necesito comparar prefijos comunes entre TODAS las subcadenas del texto, y enumerarlas explícitamente sería cuadrático o peor?
- ¿El problema se reduce a una consulta sobre prefijos comunes de sufijos (repeticiones, subcadena común, cantidad de subcadenas distintas)?

Utilidad: El suffix array ordena lexicográficamente todos los sufijos de una cadena, y el arreglo LCP guarda la longitud del prefijo común entre cada par de sufijos ADYACENTES en ese orden. Juntos permiten responder en tiempo casi lineal preguntas sobre repeticiones y subcadenas comunes que de forma ingenua requerirían enumerar las $\mathcal{O}(n^2)$ subcadenas: subcadena repetida más larga, cantidad de subcadenas distintas, subcadena común

más larga entre varios textos, y consultas de prefijo común entre sufijos arbitrarios.

Complejidad: $\mathcal{O}(n \log^2 n)$ para construir el suffix array por duplicación (log n rondas, cada una con un ordenamiento), y $\mathcal{O}(n)$ amortizado para el arreglo LCP con el algoritmo de Kasai.

Fundamento teórico: Toda subcadena de S es un PREFIJO de algún sufijo de S , así que una subcadena que aparece dos veces corresponde a un prefijo común de dos sufijos distintos, y su longitud máxima es $\max_{i \neq j} \text{lcp}(\text{suf}_i, \text{suf}_j)$. La clave es que no hace falta examinar los $\mathcal{O}(n^2)$ pares: al ordenar los sufijos lexicográficamente, la función lcp cumple $\text{lcp}(\text{sa}_i, \text{sa}_j) = \min_{i < t \leq j} \text{lcp}(\text{sa}_{t-1}, \text{sa}_t)$ para $i < j$ (un mínimo sobre un rango del arreglo LCP), porque un prefijo común a dos sufijos lo es también de todos los sufijos intercalados entre ellos en el orden. Como un mínimo sobre un rango nunca supera a ninguno de sus elementos, el máximo global sobre todos los pares se alcanza necesariamente en un par ADYACENTE, y la respuesta es simplemente $\max(\text{lcp})$. La construcción del suffix array por duplicación se apoya en que, conociendo el rango de cada sufijo por sus primeros k caracteres, el rango por $2k$ caracteres se obtiene ordenando por el par $(\text{rank}_k[i], \text{rank}_k[i + k])$ — de ahí las $\lceil \log n \rceil$ rondas. El algoritmo de Kasai calcula los LCP en orden de POSICIÓN en el texto (no de rango) usando el invariante de que, si lcp del sufijo que empieza en i vale h , entonces el del sufijo que empieza en $i + 1$ vale al menos $h - 1$: al quitar el primer carácter de dos cadenas que comparten h caracteres, siguen compartiendo $h - 1$. Por eso h solo decrece en una unidad por iteración pero puede crecer arbitrariamente, dando un total amortizado de $\mathcal{O}(n)$. [25], [26]

Ejercicio aplicado

Dados T textos, para cada uno hallar la longitud de la subcadena más larga que aparece al menos dos veces en él (las dos apariciones pueden solaparse).

I/O: T casos, cada uno una cadena en su propia línea con $|S| \leq 5000$ sobre el alfabeto minúsculo $\Rightarrow$ como las cadenas no contienen espacios, se usa lectura total + tokenización (patrón 2) y cada token es directamente una cadena completa. Se atiende aparte el caso $|S| < 2$ (respuesta 0: no puede haber repetición). Verificado contra el caso de ejemplo y contrastado con una fuerza bruta (enumerar todas las subcadenas de cada longitud en un conjunto) en 158 casos, incluyendo alfabeto binario, cadenas de un solo carácter repetido y otros casos borde, sin discrepancias. El enunciado no acota T , así que se midió el peor caso razonable: una cadena de $|S| = 5000$ tarda $\sim 0.02\text{s}$ (tanto aleatoria como en el peor caso para Kasai, "a" repetida 5000 veces), y $T = 100$ cadenas de longitud máxima $\sim 1.04\text{s}$.

Código solución

```

import sys

def suffix_array(s):

```

```

n = len(s)
# ronda inicial: ordenar los sufijos por su primer
# caracter
sa = sorted(range(n), key=lambda i: s[i])
rank = [0]*n
r = 0
prev = s[sa[0]]
for idx, i in enumerate(sa):
    if s[i] != prev:
        r += 1
        prev = s[i]
    rank[i] = r
k = 1
# duplicacion: si ya se conoce el rango por k
# caracteres,
# ordenar por el par (rank[i], rank[i+k]) da el rango
# por 2k
while r < n - 1:
    # corta apenas todos
    # los rangos son distintos
    key = [0]*n
    for i in range(n):
        # el par se empaqueta en un solo entero para
        # ordenar mas rapido
        key[i] = rank[i] * (n + 1) + (rank[i+k] + 1
    if i + k < n else 0)
    sa.sort(key=lambda i: key[i])
    new_rank = [0]*n
    r = 0
    prev = key[sa[0]]
    for i in sa:
        if key[i] != prev:
            r += 1
            prev = key[i]
        new_rank[i] = r
    rank = new_rank
    k <= 1
return sa, rank

def kasai(s, sa, rank):
    n = len(s)
    lcp = [0]*n
    h = 0
    # se recorre por POSICION en el texto: al avanzar de
    # i a i+1,
    # el lcp baja como mucho en 1, asi que h nunca se
    # reinicia a 0
    for i in range(n):
        if rank[i] > 0:
            j = sa[rank[i]-1]
            # sufijo anterior en
            # orden lexicografico
            while i + h < n and j + h < n and s[i+h] == s
            [j+h]:
                h += 1
            lcp[rank[i]] = h
            if h:
                h -= 1
        else:
            h = 0
    return lcp

def solve():
    data = sys.stdin.buffer.read().split()
    t = int(data[0])
    out = []
    for i in range(1, t+1):
        s = data[i].decode()
        n = len(s)
        if n < 2:
            out.append("0")
            continue
        sa, rank = suffix_array(s)
        lcp = kasai(s, sa, rank)
        # el maximo lcp sobre pares adyacentes ya es el
        # maximo global

```

```

        out.append(str(max(lcp)))
        sys.stdout.write("\n".join(out) + "\n")

solve()

```

Extras: qué subcadena es y dónde aparece — variantes de `solve()` para cuando el problema pide, además de la longitud, QUÉ subcadena es y DÓNDE aparece (`lcp_detailed`: todas las subcadenas de longitud máxima con sus posiciones) o cuál es la subcadena más larga que aparece el MAYOR número de veces (`lcp_repeated`, con mínimo en ventana deslizante sobre el arreglo LCP). Se llaman con `(s, sa, lcp, n)` en lugar de `max(lcp)`.

```

from collections import deque, Counter

def blocks(s, sa, lcp, n, L):
    # bloques maximales con lcp >= L -> (subcadena,
    # ocurrencias)
    res = []
    p = 1
    while p < n:
        if lcp[p] >= L:
            ini = p
            while p < n and lcp[p] >= L:
                p += 1
            occ = sorted(sa[ini-1:p])
            res.append((s[sa[ini]:sa[ini]+L], occ))
        else:
            p += 1
    return res

def lcp_detailed(s, sa, lcp, n):
    L = max(lcp)
    if L == 0:
        return "0"
    partes = [f"'{sub}' x{len(occ)} en {occ}" for sub,
    occ in blocks(s, sa, lcp, n, L)]
    return f"[L] -> " + " | ".join(partes)

def lcp_repeated(s, sa, lcp, n):
    # ninguna subcadena se repite mas que su primer
    # caracter
    c = max(Counter(s).values())
    if c < 2:
        return "0"
    w = c - 1
    # ventana de w valores
    # de lcp = c sufijos
    dq = deque()
    # minimo de ventana
    # deslizante
    mejorL = mejorA = -1
    for p in range(1, n):
        while dq and lcp[dq[-1]] >= lcp[p]:
            dq.pop()
        dq.append(p)
        if dq[0] < p - w + 1:
            dq.popleft()
        if p >= w and lcp[dq[0]] > mejorL:
            mejorL = lcp[dq[0]]
            mejorA = p - w + 1
    occ = sorted(sa[mejorA-1:mejorA+w])
    return f"{mejorL} -> '{s[sa[mejorA]:sa[mejorA]+mejorL
    ]}' x{len(occ)} en {occ}"

```

9.5 Firma Canónica + Hash Map (palabras con letras revueltas)

Preguntas guía:

- ¿Debo agrupar o emparejar palabras que son "la misma" bajo alguna transformación permitida (anagramas, reordenamientos parciales, rotaciones, cambios de caso)?
- ¿Puedo definir una función que colapse todas las palabras equivalentes al MISMO valor, sin comparar cada par entre sí?
- ¿Me piden, entre todos los candidatos equivalentes, el menor lexicográficamente (o algún otro criterio de desempate)?
- ¿Comparar todas las palabras del texto contra todas las del diccionario sería cuadrático o más?

Utilidad: Convierte un problema de "¿son equivalentes estas dos palabras?" en un problema de búsqueda exacta en tabla hash. En lugar de comparar cada palabra del texto contra cada palabra del diccionario, se define una FIRMA CANÓNICA: una función que asigna a todas las palabras de una misma clase de equivalencia un valor idéntico, y a palabras de clases distintas valores distintos. El diccionario se indexa una sola vez por firma, y cada consulta se resuelve con un acceso directo. Aplica a anagramas, reordenamientos parciales, normalización de cadenas y, en general, a cualquier relación de equivalencia que se pueda describir por un conjunto de invariantes.

Complejidad: $O(D \log L + T)$ amortizado por caso de prueba, donde D es el número de palabras del diccionario, T el del texto y L la longitud máxima de palabra (el $\log L$ viene de ordenar las letras del medio para construir la firma). Sin la firma, la comparación directa sería $O(D \cdot T \cdot L \log L)$.

Fundamento teórico: La transformación permitida — reordenar todas las letras excepto la primera y la última — genera una relación de equivalencia sobre las palabras. Dos palabras son equivalentes si y solo si coinciden en CUATRO invariantes: la longitud, la primera letra, la última letra y el MULTICONJUNTO de letras intermedias. Estos invariantes son completos: si coinciden, existe una permutación de las letras del medio que transforma una en la otra (basta reordenar el multiconjunto), y si difieren en alguno, ninguna permutación del medio puede igualarlas (porque una permutación conserva el multiconjunto y no toca los extremos ni la longitud). Por lo tanto, la tupla $(|w|, w_0, w_{-1}, \text{sort}(w_{1..-2}))$ es una forma canónica: dos palabras tienen la misma firma exactamente cuando son intercambiables. La longitud debe incluirse explícitamente: sin ella, las palabras "a" y "aa" producirían la misma firma ('a', 'a', "") — en la primera la única letra hace de primera Y de última, y en ambas el medio es vacío — a pesar de tener longitudes distintas y no ser intercambiables. Para el desempate, como la respuesta pedida es el menor lexicográficamente, basta conservar en cada casilla de la tabla el mínimo visto; nótese que la palabra correcta puede NO ser la propia palabra del texto aunque esta esté en el diccionario, como muestra el segundo caso de ejemplo, donde **drinaemg** pertenece al diccionario pero se reemplaza por **dreaming** por ser

anterior en orden lexicográfico.

Ejercicio aplicado

Dado un diccionario y un texto cuyas palabras tienen las letras del medio revueltas (primera y última en su lugar original), reemplazar cada palabra del texto por la palabra del diccionario que se convierte en ella reordenando solo las letras intermedias, eligiendo la menor lexicográficamente si hay varias, o dejándola igual si no hay ninguna.

I/O: N casos, cada uno en EXACTAMENTE dos líneas (diccionario y texto) de a lo sumo 200 caracteres cada una $\Rightarrow$ la estructura por líneas es semánticamente relevante (hay que saber dónde termina el diccionario y empieza el texto), así que se usa `sys.stdin.buffer.readline` línea por línea (patrón 1) y se tokeniza cada línea con `.split()`. Se trabaja directamente sobre `bytes` sin decodificar, ya que la firma solo necesita comparar e indexar; solo la salida se decodifica. Los límites son diminutos (200 caracteres por lista), así que el rendimiento no es un problema, aunque N no está acotado superiormente en el enunciado. Verificado contra los dos casos de ejemplo y contrastado con una fuerza bruta (comparar cada palabra del texto contra todo el diccionario) en 600 casos aleatorios, sin discrepancias, incluyendo casos borde de palabras de 1, 2 y 3 letras.

Código solución

```
import sys
input = sys.stdin.buffer.readline

def firma(w):
    # dos palabras son intercambiables si coinciden
    # longitud, primera letra,
    # ultima letra y el MULTICONJUNTO de letras del medio
    # la longitud es imprescindible: sin ella "a" y "aa"
    # colisionarian
    return (len(w), w[0], w[-1], bytes(sorted(w[1:-1])))

def solve():
    n = int(input())
    out = []
    for _ in range(n):
        dic = input().split()
        txt = input().split()
        mejor = {}
        for d in dic:
            f = firma(d)
            # conserva la menor lexicograficamente (el
            # diccionario ya viene ordenado,
            # pero la comparacion explicita lo hace
            # independiente del orden de entrada)
            if f not in mejor or d < mejor[f]:
                mejor[f] = d
            # si ninguna palabra del diccionario aplica, se
            # deja la original
        res = [mejor.get(firma(w), w) for w in txt]
        out.append(b" ".join(res).decode())
    sys.stdout.write("\n".join(out) + "\n")

solve()
```

9.6 Regex: findall y lookahead (conteo de patrones)

Preguntas guía:

- ¿El problema se reduce a contar cuántas veces aparecen unos pocos patrones LITERALES dentro de un texto, sin estructura algorítmica adicional?
- ¿Los patrones son fijos y conocidos de antemano, de modo que puedo hacer una pasada independiente por cada uno?
- ¿Debo verificar si los patrones pueden SOLAPARSE consigo mismos, para decidir entre conteo simple o lookahead?
- ¿El texto es suficientemente corto como para que el costo de varias pasadas sea irrelevante?

Utilidad: Resolver directamente, con el módulo `re` de la librería estándar, problemas cuya esencia es localizar o contar ocurrencias de patrones en un texto. Para patrones literales y fijos evita escribir a mano un autómata o un KMP: el motor de expresiones regulares ya está optimizado en C y una sola línea reemplaza decenas. Además, `re` da acceso gratuito a posiciones de coincidencia, sustitución, partición del texto y conteo con solapamiento, que suelen ser justamente las variantes que pide un problema parecido.

Complejidad: $\mathcal{O}(kn)$ donde n es la longitud del texto y k la cantidad de patrones, ya que se hace una pasada lineal por patrón (el motor recorre el texto una vez por cada llamada a `findall`).

Fundamento teórico: Contar ocurrencias de un patrón literal admite dos semánticas distintas y el motor de `re` implementa solo una: `re.findall` avanza el cursor hasta el FINAL de cada coincidencia, de modo que nunca devuelve coincidencias solapadas. La pregunta relevante es entonces cuándo esa restricción cambia el resultado, y la respuesta es puramente estructural: un patrón P puede solaparse consigo mismo si y solo si tiene un BORDE PROPIO, es decir, un prefijo propio que también es sufijo. Si P tiene un borde de longitud $b > 0$, dos ocurrencias pueden empezar a distancia $|P| - b$ y compartir b caracteres; si no tiene ningún borde, dos ocurrencias distintas no pueden compartir ni un carácter, y el conteo sin solapamiento coincide exactamente con el conteo total. Los tres patrones de este problema (`bravo`, `ha`, `boooo`) no tienen borde propio — ninguno de sus prefijos propios coincide con un sufijo — así que `findall` es correcto sin más cuidados. En contraste, `aa` tiene borde `a` y en `aaaa` `findall` devuelve 2 mientras que las ocurrencias reales son 3. Cuando sí hace falta contar solapadas, se usa un LOOKAHEAD ($?=P$): al ser una aserción de ancho cero, no consume caracteres, así que el motor avanza una sola posición tras cada éxito y encuentra todas las ocurrencias; devuelve cadenas vacías, pero lo que interesa es cuántas son. El otro punto no evidente es que hacer una pasada separada por patrón NO equivale a una única pasada con alternancia $P1|P2|P3$: la alternancia consume el texto, así que los patrones compiten entre sí y una coincidencia larga puede ocultar a otra contenida en ella (en `bravo`, el patrón `bravo|ra` encuentra solo `bravo`, perdiendo el `ra` interior). Como el enunciado cuenta cada patrón de forma

independiente, las pasadas separadas son la traducción fiel.

Ejercicio aplicado

Dada una cadena de sonidos del público, calcular una puntuación sumando 3 por cada aparición de `bravo`, sumando 1 por cada `ha` y restando 1 por cada `boooo`.

I/O: Una única línea con la cadena, $|S| \leq 1000 \Rightarrow$ lectura de una sola línea con `sys.stdin.buffer.readline` (patrón 1), con `.strip()` para quitar el salto de línea y `.decode()` porque `re` trabaja sobre `str`. El tamaño es diminuto, así que las tres pasadas independientes no representan ningún costo. Verificado contra los cinco casos de ejemplo del enunciado, y comprobado aparte que ninguno de los tres patrones tiene borde propio, contrastando además `findall` contra el conteo con lookahead en 20 000 cadenas aleatorias sobre el alfabeto relevante sin una sola diferencia: por eso la ausencia de manejo de solapamiento es correcta y no una omisión.

Código solución

```
import re
import sys
input = sys.stdin.buffer.readline

data = input().strip().decode() # re trabaja sobre str,
                                # no sobre bytes

pat1 = r'bravo'
pat2 = r'ha'
pat3 = r'boooo'

score = 0
# findall NO cuenta solapamientos, pero ninguno de estos
# tres patrones
# tiene borde propio (prefijo == sufijo), así que no
# puede solaparse consigo mismo
score += 3*len(re.findall(pat1, data))
score += len(re.findall(pat2, data))
score -= len(re.findall(pat3, data))

sys.stdout.write(str(score)+"\n")
```

Extras: referencia rápida de re — no forman parte de la solución, son variantes útiles para problemas parecidos. Todos los resultados mostrados fueron ejecutados con `data = "boooohaboooo"`.

```
import re
data = "boooohaboooo"

# POSICIONES E INFORMACION DE CADA OCURRENCIA
for m in re.finditer(r'ha', data):
    m.group() # 'ha' el texto que coincidió
    m.start() # 5 indice inicial
    m.end() # 7 indice final (exclusivo)
    m.span() # (5, 7) la tupla (inicio, fin)

# SOLAPAMIENTO
# r'oo' CONSUME caracteres -> no solapa
re.findall(r'oo', data) # ['oo', 'oo', 'oo', 'oo']
-> 4
# r'(?=oo)' es un lookahead: ancho cero, NO consume -> si
# solapa
re.findall(r'(?=oo)', data) # ['', '', '', '', '', '']
-> 6
[m.start() for m in re.finditer(r'(?=oo)', data)] #
[1, 2, 3, 8, 9, 10]
```

```

# GRUPOS DE CAPTURA
m = re.search(r'(b)(o+)', data) # search: primera
                                # coincidencia o None
m.group(0) # 'boooo' la coincidencia completa
m.group(1) # 'b' primer parentesis
m.group(2) # 'oooo' segundo parentesis
m.groups() # ('b', 'oooo')
# CUIDADO: si el patron tiene grupos, findall devuelve
# TUPLAS, no la coincidencia
re.findall(r'(b)(o+)', data) # [('b', 'oooo'), ('b', '
                                oooo')]
re.findall(r'(?b)(?o+)', data) # ['boooo', 'boooo']
                                (? : ) agrupa sin capturar

# SUSTITUCION Y PARTICION
re.sub(r'o+', '0', data) # 'b0hab0'
re.subn(r'o+', '', data) # ('bhab', 8) devuelve (
                                texto, cuantas)
re.split(r'ha', data) # ['boooo', 'boooo']
re.split(r'(ha)', data) # ['boooo', 'ha', 'boooo']
                                con grupo conserva el separador

# COMPILAR (si el patron se reutiliza muchas veces)
rx = re.compile(r'boooo')
rx.findall(data) # ['boooo', 'boooo']

# UNA SOLA PASADA CON ALTERNANCIA
puntos = {'bravo':3, 'ha':1, 'boooo':-1}
rx2 = re.compile(r'bravo|boooo|ha')
sum(puntos[x] for x in rx2.findall(data)) # -1,
                                coincide aqui
# PERO la alternancia CONSUME, asi que los patrones
                                compiten entre si:
re.findall(r'bravo|ra', 'bravo') # ['bravo'] <- se
                                pierde el 'ra' interior
re.findall(r'ra', 'bravo') # ['ra']
# por eso, si el enunciado cuenta cada patron por
                                separado, van pasadas separadas

# PATRON LITERAL CON CARACTERES ESPECIALES
re.escape('a.b*c') # 'a\\.b\\|*c' neutraliza .
                                * + ? [ ] ( ) etc.

```

9.7 Subcadenas Distintas con Restricción (Suffix Array + LCP + Dos Punteros)

Preguntas guía:

- ¿Me piden contar subcadenas DISTINTAS (por contenido, no por posición) que cumplan una condición?
- ¿La condición es MONÓTONA respecto a la longitud, es decir, si una subcadena la cumple, también la cumple cualquier prefijo suyo (ej. "a lo sumo k caracteres malos")?
- ¿Puedo calcular, para cada posición de inicio, la longitud máxima que sigue cumpliendo la condición, por ejemplo con dos punteros?
- ¿Enumerar las $\mathcal{O}(n^2)$ subcadenas en un conjunto o en un trie sería lento o gastaría demasiada memoria?

Utilidad: Generaliza la fórmula clásica de cantidad de subcadenas distintas ($\frac{n(n+1)}{2} - \sum \text{lcp}$) al caso en que solo interesan las subcadenas que satisfacen una propiedad cerrada bajo prefijos. Cada sufijo aporta sus prefijos "válidos" que no hayan aparecido antes, y el suffix array con el LCP dice exactamente cuántos de ellos ya fueron contados. Sirve para cualquier conteo de subcadenas distintas

con restricción monótona: límite de caracteres prohibidos, suma acotada de pesos, a lo sumo k letras distintas, etc.

Complejidad: $\mathcal{O}(n \log^2 n)$ dominado por la construcción del suffix array por duplicación, más $\mathcal{O}(n)$ para Kasai, $\mathcal{O}(n)$ para los dos punteros y $\mathcal{O}(n)$ para el conteo final.

Fundamento teórico: Toda subcadena es un prefijo de algún sufijo, y cada subcadena distinta se cuenta UNA sola vez si se atribuye al primer sufijo, en orden lexicográfico, que la tiene como prefijo. Si los sufijos están ordenados en el suffix array, los prefijos del sufijo $\text{sa}[p]$ de longitud $\leq \text{lcp}[p]$ coinciden con prefijos de $\text{sa}[p-1]$, así que ya aparecieron antes; los de longitud $> \text{lcp}[p]$ son nuevos. Sin restricción, esto da $(n - \text{sa}[p]) - \text{lcp}[p]$ subcadenas nuevas por sufijo, y sumando se obtiene la fórmula clásica. Con la restricción, se usa que la propiedad es monótona: si $s[i..i+L)$ tiene a lo sumo k letras malas, cualquier prefijo suyo también, así que los prefijos válidos del sufijo que empieza en i forman un segmento inicial de longitudes $1.. \text{largo}[i]$. El aporte nuevo del sufijo $\text{sa}[p]$ es entonces $\max(0, \text{largo}[\text{sa}[p]] - \text{lcp}[p])$. La corrección de no volver a contar los prefijos cortos se apoya en que la validez depende SOLO del contenido: un prefijo de longitud $L \leq \text{lcp}[p]$ es la misma cadena en $\text{sa}[p]$ y en $\text{sa}[p-1]$, por lo que si es válido en uno lo es en el otro y ya fue contado. Los valores $\text{largo}[i]$ se obtienen con dos punteros: el extremo derecho j nunca retrocede al avanzar i , porque quitar un carácter del inicio no puede aumentar la cantidad de malos, lo que da un total lineal. Cuando el propio $s[i]$ es malo y $k = 0$, la ventana es vacía y hay que empujar j a $i+1$ sin descontar nada. [25], [26]

Ejercicio aplicado

(Codeforces 271D — Good Substrings) Dada una cadena s ($|s| \leq 1500$), una máscara que indica qué letras son buenas y un entero k , contar cuántas subcadenas DISTINTAS de s contienen a lo sumo k letras malas.

I/O: Tres líneas (cadena, máscara de 26 caracteres, entero k) sin espacios internos $\Rightarrow$ lectura total + tokenización (patrón 2), obteniendo exactamente 3 tokens. Se trabaja sobre **bytes**: la tabla **malo** se indexa directamente por el código ASCII de cada letra (con 97 posiciones de relleno al inicio) para evitar restar 97 en el bucle. Verificado contra los dos casos de ejemplo y contrastado con una fuerza bruta (conjunto de todas las subcadenas buenas) en 600 casos aleatorios con alfabetos de 2 a 26 letras y máscaras y k al azar, sin discrepancias, más casos borde (una sola letra mala con $k = 0$, todas las letras malas con $k = 0$ y $k = n$, todas buenas). En el peor caso ($|s| = 1500$) tarda $\sim 0.02\text{s}$; como referencia, el enfoque clásico de insertar en un trie todas las subcadenas buenas da la misma respuesta pero tarda $\sim 0.7\text{s}$, por recorrer las ~ 1.1 millones de subcadenas.

Código solución

```

import sys

def suffix_array(s):
    n = len(s)
    # ronda inicial: ordenar los sufijos por su primer
    # caracter

```

```

sa = sorted(range(n), key=lambda i: s[i])
rank = [0]*n
r = 0
prev = s[sa[0]]
for idx, i in enumerate(sa):
    if s[i] != prev:
        r += 1
        prev = s[i]
    rank[i] = r
k = 1
# duplicacion: si ya se conoce el rango por k
# caracteres,
# ordenar por el par (rank[i], rank[i+k]) da el rango
# por 2k
while r < n - 1:
    # corta apenas todos
    # los rangos son distintos
    key = [0]*n
    for i in range(n):
        # el par se empaqueta en un solo entero para
        # ordenar mas rapido
        key[i] = rank[i] * (n + 1) + (rank[i+k] + 1
    if i + k < n else 0)
    sa.sort(key=lambda i: key[i])
    new_rank = [0]*n
    r = 0
    prev = key[sa[0]]
    for i in sa:
        if key[i] != prev:
            r += 1
            prev = key[i]
        new_rank[i] = r
    rank = new_rank
    k <= 1
return sa, rank

def kasai(s, sa, rank):
    n = len(s)
    lcp = [0]*n
    h = 0
    # se recorre por POSICION en el texto: al avanzar de
    # i a i+1,
    # el lcp baja como mucho en 1, asi que h nunca se
    # reinicia a 0
    for i in range(n):
        if rank[i] > 0:
            j = sa[rank[i]-1]
            # sufijo anterior en
            # orden lexicografico
            while i + h < n and j + h < n and s[i+h] == s
            [j+h]:
                h += 1
            lcp[rank[i]] = h
            if h:
                h -= 1
        else:
            h = 0
    return lcp

def solve():
    data = sys.stdin.buffer.read().split()
    s = data[0]
    mascara = data[1]
    k = int(data[2])
    n = len(s)
    # malo[c] = 1 si la letra con codigo ASCII c es mala;
    # se rellena con 97
    # ceros al inicio para indexar directo por el byte de
    # la letra
    malo = [1 - (mascara[c - 97] - 48) for c in range(97,
    123)]
    malo = [0]*97 + malo

    # dos punteros: largo_bueno[i] = longitud maxima
    # desde i con <= k malas
    largo_bueno = [0]*n

```

```

j = 0
malos = 0
for i in range(n):
    while j < n and malos + malo[s[j]] <= k:
        malos += malo[s[j]]
        j += 1
    largo_bueno[i] = j - i
    if j > i:
        malos -= malo[s[i]]
        # s[i] sale de la
        # ventana
    else:
        j = i + 1
        # ventana vacia: s[i
        # es mala y k == 0

sa, rank = suffix_array(s)
lcp = kasai(s, sa, rank)
total = 0
for p in range(n):
    # los primeros lcp[p] prefijos ya se contaron en
    # el sufijo anterior;
    # solo aportan los prefijos buenos mas largos que
    # eso
    nuevas = largo_bueno[sa[p]] - lcp[p]
    if nuevas > 0:
        total += nuevas
sys.stdout.write(str(total)+"\n")

solve()

```

9.8 Z-Function (Bordes con Aparición Interior)

Preguntas guía:

- ¿Necesito saber, para cada posición del texto, cuánto coincide la cadena que empieza ahí con el PREFIJO de la cadena completa?
- ¿Me piden un prefijo que también sea sufijo (un borde), o que además aparezca en otra parte del texto?
- ¿Busco todas las apariciones de un patrón P en un texto T , y me sirve concatenar $P + \# + T$ para resolverlo con un solo arreglo?
- ¿El texto llega a 10^6 caracteres, de modo que necesito un algoritmo lineal y sin recursión?

Utilidad: La Z-function calcula, para cada posición i , la longitud $z[i]$ del mayor prefijo común entre la cadena completa y el sufijo que empieza en i . Con ese arreglo se responden en tiempo lineal preguntas sobre bordes (prefijos que son sufijos), apariciones de un patrón (concatenándolo con el texto), períodos de la cadena y compresión de cadenas repetidas. Es una alternativa a la función de prefijos de KMP con una interpretación más directa: $z[i]$ habla de la posición i en sí, mientras que la función de prefijos habla del borde del prefijo que termina en i .

Complejidad: $\mathcal{O}(n)$ en tiempo y memoria.

Fundamento teórico: El algoritmo mantiene la Z -box $[l, r)$: el segmento más a la derecha encontrado hasta ahora que coincide con un prefijo de la cadena, es decir, $s[l..r) = s[0..r-l)$. Al procesar una posición i dentro de la caja ($i < r$), la subcadena $s[i..r)$ es una copia de $s[i-l..r-l)$, así que $z[i]$ es al menos $\min(r-i, z[i-l])$: se

reutiliza un valor ya calculado en vez de comparar desde cero. Solo cuando esa cota llega al borde r hace falta seguir comparando carácter a carácter, y cada comparación exitosa extiende r . Como r nunca retrocede y a lo sumo vale n , el total de comparaciones exitosas es $\mathcal{O}(n)$, y cada posición hace a lo sumo una comparación fallida, lo que da el costo lineal. Para este problema, el sufijo de longitud L coincide con el prefijo exactamente cuando $z[n-L] = L$, es decir, cuando la coincidencia desde $i = n-L$ llega hasta el final. Falta que $t = s[0..L)$ aparezca además en el INTERIOR, empezando en alguna posición $j \geq 1$ y terminando antes del último carácter, lo que equivale a $z[j] \geq L$ con $j+L \leq n-1$. La observación clave es que esa condición obliga a $j \leq n-1-L = i-1$: toda aparición interior empieza estrictamente ANTES que el sufijo. Por eso basta llevar el máximo de z sobre las posiciones ya recorridas; y recíprocamente, si ese máximo es $\geq L$ en alguna $j < i$, la aparición en j termina en $j+L-1 \leq n-2$, así que no es el sufijo. Recorriendo i en orden creciente, $L = n-i$ decrece, y la primera posición que cumple ambas condiciones da el t más largo. [27]

Ejercicio aplicado

(Codeforces 126B — Password) Dada una cadena s ($|s| \leq 10^6$), hallar la subcadena t más larga que sea a la vez prefijo de s , sufijo de s , y que aparezca también en algún lugar que no sea ni el inicio ni el final de s ; si no existe, imprimir **Just a legend**.

I/O: Una única línea con la cadena, hasta 10^6 caracteres $\Rightarrow$ se lee con `sys.stdin.buffer.readline` (patrón 1) y se trabaja sobre `bytes`, donde indexar da enteros y las comparaciones son más baratas que sobre `str`; solo la respuesta se decodifica. La Z-function está escrita con un acumulador local `zi` en vez de leer y escribir `z[i]` en cada paso del bucle interno, para reducir accesos a lista. Verificado contra los dos casos de ejemplo y contrastado con una fuerza bruta (probar cada longitud con `endswith` y `find` restringido al interior) en 712 casos, incluyendo alfabetos de 1 a 3 letras y casos borde como **a**, **aa**, **aaa**, **ababa** y **abacaba**, sin discrepancias. Con $|s| = 10^6$ se midieron entre ~ 0.16 s y ~ 0.23 s según la topología (aleatoria, toda **a**, **ab** repetido, bloques con bordes largos), en CPython 3.12; en jueces con versiones anteriores de CPython conviene esperar varias veces más, pero sigue quedando con margen amplio.

Código solución

```
import sys

def z_function(s):
    n = len(s)
    z = [0]*n
    l = r = 0
    # Z-box: s[l..r) coincide con s[0..r-l)
    for i in range(1, n):
        zi = 0
        if i < r:
            # dentro de la caja, s[i..r) es copia de s[i-l..r-l):
            # se reutiliza z[i-l], sin pasarse del borde
```

```

        r
        zi = r - i
        if z[i - l] < zi:
            zi = z[i - l]
        # solo se compara caracter a caracter mas alla de lo ya conocido
        while i + zi < n and s[zi] == s[i + zi]:
            zi += 1
        z[i] = zi
        if i + zi > r:
            # la caja se corre a la derecha, nunca atras
            l = i
            r = i + zi
    return z

def solve():
    s = sys.stdin.buffer.readline().strip()
    n = len(s)
    z = z_function(s)
    maxx = 0
    # mayor z visto en posiciones anteriores a i
    for i in range(1, n):
        L = n - i
        # z[i] == L: el sufijo de largo L es igual al prefijo.
        # maxx >= L: hay una copia interior, y toda copia interior
        # empieza necesariamente antes de i
        if z[i] == L and maxx >= L:
            sys.stdout.write(s[:L].decode() + "\n")
            return
        if z[i] > maxx:
            maxx = z[i]
    sys.stdout.write("Just a legend\n")

solve()
```

9.9 Período Mínimo con la Función de Prefijos (KMP)

Preguntas guía:

- ¿Me piden si una cadena es la repetición exacta de un bloque más corto, o cuántas veces se repite el bloque más pequeño posible?
- ¿Necesito el período mínimo de una cadena, o su borde más largo (el mayor prefijo propio que también es sufijo)?
- ¿Tengo en cuenta que el período mínimo puede NO dividir a la longitud, y en ese caso la cadena no es una potencia de nada salvo de sí misma?
- ¿La cadena llega a 10^6 caracteres y necesito un algoritmo lineal?

Utilidad: La función de prefijos de KMP calcula, para cada prefijo de la cadena, la longitud de su borde más largo. El borde de la cadena completa, $\pi[n-1]$, determina directamente su período mínimo $p = n - \pi[n-1]$, que es el paso más pequeño tal que $s_i = s_{i+p}$ para todo i válido. Con eso se responde en tiempo lineal si una cadena es potencia de otra, cuál es su raíz primitiva, y cuántas veces se repite; es la misma herramienta que sirve para búsqueda de patrones, reutilizada sobre la propia cadena.

Complejidad: $\mathcal{O}(n)$ en tiempo y memoria por cadena.

Fundamento teórico: Una cadena tiene período p si y solo si tiene un borde de longitud $n-p$: decir que $s[0..n-$

$p) = s[p..n)$ es exactamente decir que $s_i = s_{i+p}$ para todo i . Por lo tanto, el período mínimo corresponde al borde MÁS LARGO, $p = n - \pi[n - 1]$. Pero tener período p no basta para ser una potencia: `abccab` tiene período 3 (es `abc` seguido del inicio de otro `abc`) y sin embargo no es a^k para ningún $k > 1$. Se necesita además que p divida a n , y entonces $s = (s[0..p))^{n/p}$. Si p no divide a n , la respuesta es 1, y no hay que buscar otro período: por el lema de periodicidad de Fine y Wilf, si s tuviera un período q que divide a n con $q > p$, entonces $p + q \leq n$ implicaría que $\gcd(p, q)$ también es período, y como $\gcd(p, q) \leq p$ y p es el mínimo, $\gcd(p, q) = p$, es decir, p dividiría a q y por lo tanto a n , contradicción. La función de prefijos en sí se calcula en tiempo lineal porque el borde actual k crece a lo sumo en 1 por carácter y cada retroceso $k \leftarrow \pi[k - 1]$ lo reduce estrictamente, de modo que el número total de retrocesos está acotado por el número total de incrementos, es decir, por n . [24]

Ejercicio aplicado

(UVA 10298 — Power Strings) Para cada cadena s de la entrada, hallar el mayor n tal que $s = a^n$ para alguna cadena a , es decir, cuántas veces se repite su bloque más corto.

I/O: Varias cadenas, una por línea, hasta una línea que contiene solo un punto (patrón 3 con centinela), con cadenas de hasta 10^6 caracteres IMPRIMIBLES: eso incluye espacios, así que no se puede tokenizar con `.split()` y se recorre línea por línea con `for line in sys.stdin.buffer`, quitando solo `\r\n` con `rstrip` para no borrar espacios que forman parte de la cadena. Verificado contra los tres casos de ejemplo y contrastado con una fuerza bruta (probar cada divisor de la longitud) en 1508 casos, incluyendo cadenas con espacios, con signos de puntuación, la cadena `..` (que no es el centinela) y potencias truncadas cuyo período no divide la longitud, sin discrepancias. Con $|s| = 10^6$ el peor caso medido fue $\sim 0.15s$ (`a...ab`, que fuerza la mayor cantidad de retrocesos), y nueve líneas de longitud máxima seguidas $\sim 0.9s$, en CPython 3.12.

Código solución

```
import sys

def prefix_function(s):
    n = len(s)
    pi = [0]*n          # pi[i] = borde mas largo
    de s[0..i]
    k = 0
    for i in range(1, n):
        c = s[i]
        # si no se puede extender el borde actual, se
        # prueba con el
        # siguiente borde mas corto: el borde del borde
        while k and s[k] != c:
            k = pi[k-1]
        if s[k] == c:
            k += 1
        pi[i] = k
    return pi

def solve():
```

```
    out = []
    for line in sys.stdin.buffer:
        # solo se quita el salto de línea: los espacios
        # son parte de s
        s = line.rstrip(b"\r\n")
        if s == b".":
            break
        n = len(s)
        pi = prefix_function(s)
        p = n - pi[n-1]      # periodo minimo = n -
        # borde mas largo
        # solo es potencia si el periodo divide a la
        # longitud (ej. "abccab" -> 1)
        out.append(str(n // p if n % p == 0 else 1))
    sys.stdout.write("\n".join(out) + "\n")

solve()
```

9.10 Palíndromo Más Largo (Algoritmo de Manacher)

Preguntas guía:

- ¿Me piden el palíndromo más largo contenido en una cadena, o el radio del palíndromo máximo centrado en cada posición?
- ¿Necesito contar TODAS las subcadenas palindrómicas (la suma de los radios da ese conteo)?
- ¿La cadena llega a 10^6 caracteres, de modo que expandir desde cada centro ($\mathcal{O}(n^2)$ en el peor caso) no es viable?
- ¿Debo manejar a la vez palíndromos de longitud par e impar?

Utilidad: Manacher calcula, para cada centro posible de la cadena, el radio del palíndromo más largo centrado ahí, en tiempo lineal. Con esos radios se obtiene el palíndromo más largo, la cantidad total de subcadenas palindrómicas (sumando radios), o se responde si una subcadena dada es palíndromo mirando el radio de su centro. Hay dos formulaciones equivalentes: intercalar un separador entre las letras para que todo palíndromo quede con centro en una posición, o mantener dos arreglos, uno para centros impares (en una letra) y otro para centros pares (entre dos letras).

Complejidad: $\mathcal{O}(n)$ en tiempo y memoria.

Fundamento teórico: La idea es la misma que en la Z-function: reutilizar información ya calculada gracias a una simetría. Se mantiene la ventana (l, r) del palíndromo encontrado hasta ahora cuyo extremo derecho r llega más lejos. Si la posición actual i cae dentro de esa ventana, su espejo respecto del centro es $j = l + r - i$ (o $l + r - i + 1$ para centros pares), y como la ventana es un palíndromo, alrededor de i se ve exactamente lo mismo que alrededor de j reflejado. Por lo tanto el radio en i es al menos $\min(\text{radio}[j], r - i)$: se toma el del espejo, pero sin pasar del borde de la ventana, porque más allá de r la simetría ya no está garantizada. Solo desde ese punto se sigue comparando carácter a carácter. Cada comparación exitosa en esa expansión hace crecer r , y como r nunca retrocede y está acotado por n , el total de comparaciones exitosas es lineal; cada posición hace además a lo sumo una

comparación fallida, que es la que detiene la expansión. Los centinelas `^` y `$` al inicio y al final son distintos entre sí y de cualquier letra, así que la expansión se detiene sola al llegar a un extremo sin necesidad de comprobar índices dentro del bucle, lo que en Python ahorra dos comparaciones por paso. Con los índices desplazados en 1 por el centinela, `d1[i] = k` representa el palíndromo impar `s[i - k + 1..i + k - 1]` de largo `2k - 1`, y `d2[i] = k` el palíndromo par `s[i - k..i + k - 1]` de largo `2k`, con centro entre `i - 1` e `i`. [28]

Ejercicio aplicado

(CSES 1111 — Longest Palindrome) Dada una cadena de hasta 10^6 letras minúsculas, imprimir su subcadena palindrómica más larga (cualquiera, si hay varias).

I/O: Una sola línea de hasta 10^6 caracteres $\Rightarrow$ `sys.stdin.buffer.readline` (patrón 1), trabajando sobre `bytes`, donde indexar da enteros y comparar es más barato que sobre `str`. El máximo y su posición se obtienen con `max` e `index`, que corren en C, en vez de un `max` con `key`, que haría dos millones de llamadas a función. Como se admite cualquier respuesta óptima, la verificación no compara texto sino que comprueba que la salida sea palíndromo, sea subcadena de la entrada y tenga la longitud óptima: se hizo contra una fuerza bruta de expansión desde cada centro en 807 casos, sin fallos. Con $n = 10^6$ se midieron entre $\sim 0.34s$ (aleatoria) y $\sim 0.58s$ (toda `a` y patrones periódicos) en CPython 3.12. Conviene ser honesto sobre el margen: jueces como CSES dan 1 segundo y suelen correr versiones de CPython más lentas que esta, así que el peor caso puede quedar justo; si el juez ofrece PyPy, es la opción segura. Se midió también la formulación con separadores `#` (en extras): da lo mismo pero es entre un 10% y un 25% más lenta, porque recorre $2n + 1$ posiciones y la mitad de sus comparaciones son `#` contra `#`, siempre verdaderas.

Código solución

```
import sys

def solve():
    s0 = sys.stdin.buffer.readline().strip()
    n = len(s0)
    s = b"^" + s0 + b"$"          # centinelas: índices
    # reales van de 1 a n
    # d1[i] = k -> palíndromo IMPAR s[i-k+1 .. i+k-1],
    # largo 2k-1
    d1 = [0]*(n + 2)
    l = r = 0                      # ventana (l, r)
    # exclusiva del palíndromo que llega mas lejos
    for i in range(1, n + 1):
        k = 1
        if i < r:
            # dentro de la ventana: se copia el radio del
            # espejo,
            # sin pasar del borde r (mas alla no hay
            # simetría garantizada)
            k = d1[l + r - i]
            if k > r - i:
                k = r - i
            while s[i - k] == s[i + k]:    # los centinelas
                # cortan solos
                k += 1
```

```
        d1[i] = k
        if i + k > r:
            l = i - k
            r = i + k
    # d2[i] = k -> palíndromo PAR s[i-k .. i+k-1], largo
    # 2k (centro entre i-1 e i)
    d2 = [0]*(n + 2)
    l = r = 0
    for i in range(1, n + 1):
        k = 0
        if i < r:
            k = d2[l + r - i + 1]        # el espejo de un
            # centro par se corre en 1
            if k > r - i:
                k = r - i
            while s[i - k - 1] == s[i + k]:
                k += 1
        d2[i] = k
        if i + k > r:
            l = i - k - 1
            r = i + k
    b1 = max(d1); c1 = d1.index(b1)      # max e index
    # corren en C
    b2 = max(d2); c2 = d2.index(b2)
    if 2*b1 - 1 >= 2*b2:
        ini, L = c1 - b1 + 1, 2*b1 - 1
    else:
        ini, L = c2 - b2, 2*b2
    sys.stdout.write(s[ini:ini+L].decode() + "\n")

solve()
```

Extras: formulación con separadores — equivalente y más fácil de recordar, a costa de ser algo más lenta. Intercalar `#` entre las letras hace que todo palíndromo tenga longitud impar en la cadena transformada, así que basta un solo arreglo; el radio `P[i]` coincide con la longitud del palíndromo en la cadena original, y empieza en `(i - P[i]) // 2`.

```
def manacher(s):
    n = len(s)
    T = bytearray(b"#") * (2*n + 1)
    T[1::2] = s                # T = # s0 # s1 # ... # s
    (n-1) #
    T = b"^" + T + b"$"
    m = len(T)
    P = [0]*m
    C2 = R = 0                  # C2 = 2*centro,
    # precalculado para el espejo
    for i in range(1, m - 1):
        if i < R:
            p = P[C2 - i]
            if p > R - i:
                p = R - i
        else:
            p = 0
            while T[i + 1 + p] == T[i - 1 - p]:
                p += 1
            P[i] = p
            if i + p > R:
                C2 = 2*i
                R = i + p
    return P

# uso: L = max(P); centro = P.index(L); ini = (centro - L
# ) // 2
# respuesta = s[ini:ini+L]
```

9.11 Aho-Corasick (Conteo de Múltiples Patrones)

Preguntas guía:

- ¿Tengo MUCHOS patrones y debo buscarlos todos en un mismo texto, de modo que correr KMP una vez por patrón sería $\mathcal{O}(k \cdot n)$?
- ¿Me piden cuántas veces aparece cada patrón (contando solapamientos), o si aparece alguno, o en qué posiciones terminan?
- ¿La suma de longitudes de los patrones está acotada, aunque la cantidad de patrones sea enorme?
- ¿Puedo tener patrones repetidos o que sean prefijo o sufijo unos de otros?

Utilidad: Aho-Corasick generaliza KMP a un conjunto de patrones: construye un trie con todos ellos y le agrega enlaces de fallo, obteniendo un autómata que recorre el texto UNA sola vez y en cada posición sabe qué patrones terminan ahí. Es la herramienta estándar para diccionarios de patrones: filtros de palabras, búsqueda de firmas, conteo de ocurrencias de muchas cadenas, y como base de DP sobre autómatas (contar textos que evitan un conjunto de patrones).

Complejidad: $\mathcal{O}(S + n)$ para construir y recorrer, donde S es la suma de longitudes de los patrones y n la del texto, más $\mathcal{O}(S)$ para propagar los conteos; con diccionarios como hijos, cada transición cuesta $\mathcal{O}(1)$ esperado.

Fundamento teórico: Cada nodo del trie representa un prefijo de algún patrón. El enlace de fallo de un nodo v apunta al nodo que representa el SUFIJO PROPIO más largo de la cadena de v que también es prefijo de algún patrón: es exactamente la función de prefijos de KMP, pero calculada sobre el trie en vez de sobre una sola cadena. Se construye en orden BFS (por profundidad), porque el fallo de un nodo depende del fallo de su padre, que está a menor profundidad: para el hijo v de u por la letra c , se sigue la cadena de fallos desde $\text{fail}(u)$ hasta encontrar un nodo con hijo por c , igual que el retroceso $k \leftarrow \pi[k-1]$ de KMP. Al recorrer el texto, el estado actual es siempre el nodo del sufijo más largo del texto leído que es prefijo de algún patrón; si no hay transición por la letra siguiente, se retrocede por los fallos. El detalle clave para CONTAR es que, cuando el recorrido está en el nodo v , terminan en esa posición no solo el patrón de v sino todos los patrones cuyos nodos están en la cadena de fallos de v (son sufijos de lo leído). Recorrer esa cadena en cada paso sería lento; en cambio, se suma 1 solo al nodo visitado y al final se PROPAGA: los enlaces de fallo forman un árbol con raíz en la raíz del trie, y el conteo real de un nodo es la suma de las visitas en su subárbol de fallos. Como el fallo de todo nodo está a menor profundidad, recorrer los nodos en orden BFS INVERSO y hacer $\text{cnt}[\text{fail}(v)] += \text{cnt}[v]$ acumula cada subárbol antes de pasarlo hacia arriba. Los patrones repetidos terminan en el mismo nodo, así que se resuelven solos guardando el nodo terminal de cada uno. [29]

Ejercicio aplicado

(CSES 2103 — Counting Patterns) Dado un texto de hasta 10^5 letras y hasta $5 \cdot 10^5$ patrones (con longitud total a lo sumo $5 \cdot 10^5$), contar para cada patrón en cuántas posiciones del texto aparece.

I/O: Texto, cantidad k y k patrones, todos sin espacios $\Rightarrow$ lectura total + tokenización (patrón 2), trabajando sobre **bytes** (cada letra se usa directamente como clave entera de diccionario). La salida de hasta $5 \cdot 10^5$ líneas se une en un solo **write**. Verificado contra el caso de ejemplo y contrastado con una fuerza bruta (**find** avanzando de a una posición para contar solapados) en 300 tandas aleatorias, incluyendo patrones repetidos, patrones que se solapan y patrones más largos que el texto, sin discrepancias. Tiempos medidos en CPython 3.12 con los límites máximos: $\sim 0.13\text{s}$ con $5 \cdot 10^5$ patrones de una letra, $\sim 0.14\text{s}$ con subcadenas de un texto binario, $\sim 0.25\text{s}$ con un único patrón de $5 \cdot 10^5$ caracteres, y $\sim 0.55\text{s}$ en el peor caso para esta estructura, $5 \cdot 10^4$ patrones aleatorios distintos de largo 10, que producen un trie de casi $5 \cdot 10^5$ nodos; memoria pico $\sim 112\text{ MB}$. Con un límite de 1 segundo y un juez con CPython más antiguo, ese último caso deja poco margen.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    text = data[0]
    k = int(data[1])
    pats = data[2:2+k]

    # ---- trie de patrones ----
    ch = [{}] # ch[v] = {letra: hijo}
    fin = [0]*k # nodo terminal de cada patron
    for idx in range(k):
        v = 0
        for c in pats[idx]:
            nv = ch[v].get(c)
            if nv is None:
                nv = len(ch)
                ch.append({})
                ch[v][c] = nv
            v = nv
        fin[idx] = v # patrones repetidos
                        # comparten nodo

    # ---- enlaces de fallo en orden BFS ----
    m = len(ch)
    fail = [0]*m
    orden = [0]
    for u in orden: # la lista crece
                        # mientras se recorre: es la cola BFS
        for c, v in ch[u].items():
            if u:
                # igual que el retroceso de KMP: se baja
                # por los fallos
                # hasta un nodo que tenga hijo por c
                f = fail[u]
                while f and c not in ch[f]:
                    f = fail[f]
                fail[v] = ch[f].get(c, 0)
            orden.append(v) # los hijos de la raiz
                        # quedan con fallo 0
```

```

# ---- recorrido del texto: solo se marca el nodo
# visitado ----
cnt = [0]*m
v = 0
for c in text:
    while v and c not in ch[v]:
        v = fail[v]
    v = ch[v].get(c, 0)
    cnt[v] += 1

# ---- propagacion por el arbol de fallos, de lo mas
# profundo a la raiz ----
# en v terminan tambien todos los patrones de su
# cadena de fallos
for v in reversed(orden):
    cnt[fail[v]] += cnt[v]
cnt[0] = 0 # la raiz no representa
# ningun patron

sys.stdout.write("\n".join(str(cnt[v]) for v in fin)
+ "\n")

solve()

```

10 Guía Rápida: Qué Usar y Cuándo

Resumen en lenguaje llano de cada técnica del libro: qué resuelve, cuándo conviene sobre las alternativas, y un ejemplo mínimo de entrada y salida. Sirve para el momento de leer un enunciado y decidir por dónde atacarlo; el desarrollo completo está en la página indicada. Los patrones de lectura rápida están en la sección 1 y las utilidades de Python en la sección 2.

Estructuras de Datos

Fenwick Tree (BIT) (p. 13) Un arreglo al que le vas a preguntar muchas veces "¿cuánto suman las posiciones de aquí a allá?" mientras además vas cambiando valores. Recalcular la suma cada vez sería demasiado lento; el BIT la responde en pasos logarítmicos. Es la estructura más corta de escribir de todo el capítulo, así que es la primera opción cuando la operación es una suma.

Ejemplo: con [5, 2, 9, 1], sumar el rango [2,4] da 12; si cambias la posición 3 a 0, la misma consulta pasa a dar 3.

Sumas de Prefijos + Mapa (p. 13) Para encontrar el tramo contiguo más largo que está "equilibrado": misma cantidad de dos cosas, o que suma exactamente un valor. El truco es convertir cada elemento en +1 o -1 y buscar dos puntos donde el acumulado valga lo mismo: todo lo que hay entre ellos está balanceado. Una sola pasada, sin revisar todos los tramos.

Ejemplo: en [+,+,-,-,+,-] el tramo más largo con igual cantidad de + y - es de longitud 6.

Fenwick con XOR (Nim) (p. 14) El mismo BIT pero combinando con XOR en vez de suma, que es lo que hace falta para juegos por turnos tipo Nim: hay varias pilas, cada jugador retira de una sola, y pierde quien no puede mover. El XOR de los tamaños decide el ganador, y el BIT permite responder eso para muchos rangos distintos mientras las pilas cambian.

Ejemplo: pilas [1, 2, 3]: el XOR da 0, así que pierde quien empieza.

Segment Tree (p. 15) Lo mismo que el BIT pero para operaciones que no se pueden "deshacer", como el mínimo o el máximo: ahí no sirve restar. Es la estructura por defecto para consultas de rango sobre un arreglo que cambia. Más largo de escribir que el BIT, así que si la operación es una suma, usa el BIT.

Ejemplo: con [3, 1, 4, 1, 5], el mínimo de [1,3] es 1; tras cambiar la posición 2 a 9, pasa a ser 3.

Dos Fenwick (rango + rango) (p. 16) Cuando además de consultar sumas de un rango necesitas SUMAR un valor a todo un rango de golpe. Se resuelve con dos BIT en vez de un segment tree con propagación perezosa: mucho menos código y varias veces más rápido en Python.

Ejemplo: sobre [0,0,0,0], sumar 3 a [1,3] y luego consultar [2,4] da 6.

Sparse Table (p. 17) Si el arreglo NUNCA cambia, esta tabla responde el mínimo (o máximo, o gcd) de cualquier rango en tiempo constante, más rápido que un segment tree. El precio es que no admite modificaciones. Regla simple: arreglo fijo, sparse table; arreglo que cambia, segment tree.

Ejemplo: con [7, 2, 8, 2, 4], el mínimo de [3,5] es 2, y la respuesta sale mirando solo dos casillas ya calculadas.

Pila Monótona (p. 18) Responde, para cada elemento, hasta dónde se extiende hacia los lados sin encontrar uno menor. Con eso se resuelve toda la familia de "el rectángulo más grande que cabe", "hasta cuándo este precio sigue siendo el más bajo" o "cuánto dura esta racha". Una sola pasada, aunque parezca que hay que probar todos los pares.

Ejemplo: alturas [2, 1, 5, 6, 2, 3]: el rectángulo más grande tiene área 10 (las alturas 5 y 6 con ancho 2).

Sumas de Prefijos 2D (p. 18) La versión en cuadrícula de las sumas de prefijos: precalculas una tabla y después cualquier pregunta del tipo "cuántas cosas hay dentro de este rectángulo" se responde sumando y restando cuatro valores. Su gemelo, el arreglo de diferencias, hace lo contrario: aplica miles de sumas sobre rectángulos y al final reconstruye la cuadrícula de una vez.

Ejemplo: en una grilla con árboles marcados, contar los de un rectángulo cuesta lo mismo sin importar si el rectángulo tiene 4 celdas o un millón.

Ordered Set con Fenwick (p. 20) Python no tiene el `ordered_set` de C++, y esta es la forma de suplirlo: mantener un conjunto donde puedes insertar, borrar, preguntar "¿cuántos elementos son menores que x ?" y "¿cuál es el k -ésimo más pequeño?". Requiere conocer de antemano todos los valores que aparecerán, para comprimirlos.

Ejemplo: tras insertar 10, 40, 20, el 2.º menor es 20 y hay 2 elementos menores que 40.

Balance de Paréntesis por Bloques (p. 21) Para secuencias de paréntesis balanceadas donde hay que razonar qué parejas se pueden formar sin romper el resto. La idea clave es que la secuencia se parte en bloques independientes, y solo se pueden emparejar elementos del mismo bloque.

Ejemplo: en `()()`, el primer `(` no puede emparejarse con el último `)`: están en bloques distintos.

Pila con Restricción (p. 21) Validar estructuras anidadas (cajas dentro de cajas, etiquetas, expresiones) donde además cada contenedor impone una condición sobre lo que lleva dentro. La pila lleva el control de qué está abierto, y cada cierre verifica la condición.

Ejemplo: -9 -2 2 -3 3 9 es válido (2 y 3 caben en el 9), pero -9 -9 9 9 no lo es.

Reducción con Pila (p. 22) Simplificar un recorrido eliminando los rodeos: si pasaste por un sitio y volviste, todo lo del medio sobra. Lo importante es que al quitar un rodeo puede aparecer otro más grande, así que hay que seguir revisando.

Ejemplo: la ruta $a!b!c!b!a!d$ se reduce a $a!d$: primero desaparece c , y eso deja b junto a b .

Grafos

Emparejamiento Bipartito (Kuhn) (p. 24) Dos grupos y una lista de parejas posibles entre ellos; se busca formar la mayor cantidad de parejas sin repetir a nadie. Asignar trabajadores a tareas, estudiantes a proyectos, turnos a personas. El algoritmo reacomoda parejas ya formadas cuando eso permite meter una más.

Ejemplo: con 3 chicos, 3 chicas y las parejas posibles 1-1, 1-2, 2-1, la respuesta es 2.

Kuhn sobre Segmentos (p. 24) Colocar piezas en un tablero con obstáculos sin que se ataquen. En lugar de pensar celda por celda, se numeran los tramos libres de cada fila y de cada columna, y el problema se vuelve el emparejamiento de arriba. Es el ejemplo típico de "modelar para poder aplicar un algoritmo conocido".

Ejemplo: en un tablero 3×3 con un peón en el centro caben 4 torres sin que se ataquen, en vez de las 3 del tablero vacío.

Teorema de König (p. 25) Cuántos elementos hay que quitar, como mínimo, para que no quede ningún conflicto. En general es un problema durísimo, pero si los conflictos forman un grafo bipartito la respuesta es exactamente el emparejamiento máximo: se calcula igual que la entrada anterior y ya está.

Ejemplo: con caballos de ajedrez que se atacan, basta correr Kuhn; el número que sale es la cantidad mínima a retirar.

Componentes Fuertemente Conexas (p. 26) En un grafo de calles de un solo sentido, agrupa los nodos entre los que se puede ir y volver. Sirve para simplificar el grafo (cada grupo pasa a ser un solo nodo, y lo que queda ya no tiene ciclos) y es el motor de 2-SAT.

Ejemplo: con $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 1$ y $3 \rightarrow 4$, los planetas 1, 2 y 3 son un mismo reino y el 4 es otro.

Circuito Euleriano (Hierholzer) (p. 27) Recorrer TODAS las calles exactamente una vez y volver al punto de partida. Ojo con la confusión frecuente: esto es sobre aristas, no sobre nodos (pasar por todos los nodos una vez es otro problema, mucho más difícil). Se comprueba antes si existe: todos los nodos deben tener grado par.

Ejemplo: un triángulo tiene circuito (1 2 3 1); una "Y" de tres brazos no tiene, porque sus cuatro nodos quedan con grado impar.

Puntos de Articulación (Tarjan) (p. 27) Qué nodos son críticos: si ese nodo falla, la red se parte en pedazos. Se usa para medir robustez, encontrar cuellos de botella o identificar el servidor que no puede caerse.

Ejemplo: en la cadena $1 - 2 - 3$, el nodo 2 es punto de articulación; en un triángulo no hay ninguno.

Bron-Kerbosch (clique máximo) (p. 28) El grupo más grande en el que todos son compatibles con todos. Es exponencial, así que solo sirve con pocos nodos (unas decenas), pero para ese tamaño es la herramienta correcta y no hay atajo mejor.

Ejemplo: si A, B y C son compatibles entre sí y D solo con A, el grupo máximo es {A, B, C}, de tamaño 3.

Orden Topológico + Conteos (p. 29) Cuando entran cantidades astronómicas de elementos por un grafo sin ciclos y simularlos uno por uno es imposible. En vez de mover elementos, se mueven CONTEOS: se procesa cada nodo cuando ya sabe cuántos le llegaron en total, y reparte de una sola vez.

Ejemplo: con 10^{18} bolas entrando por el nodo 1, no se simula ninguna bola: se calcula de una cuántas pasan por cada nodo.

Union-Find (DSU) (p. 30) Va uniendo elementos en grupos y responde al instante si dos están en el mismo grupo. Es la forma más rápida de contar cuántos grupos separados hay cuando las conexiones van llegando una a una, y no necesita construir el grafo.

Ejemplo: con las uniones A-B y C-D sobre cuatro elementos quedan 2 grupos.

Dijkstra (p. 31) La ruta más barata desde un punto a todos los demás, cuando ningún tramo tiene costo negativo. Es el algoritmo de caminos mínimos por defecto: mapas, costos, cualquier grafo donde avanzar cuesta. Si hay costos negativos, no sirve: ahí va Bellman-Ford.

Ejemplo: con $1 \rightarrow 2$ costo 5 y $1 \rightarrow 3 \rightarrow 2$ costos 1 y 1, la distancia de 1 a 2 es 2, no 5.

BFS en grilla (y 0-1 BFS) (p. 32) El camino más corto cuando todos los pasos cuestan igual: laberintos, mínimo número de movimientos, distancia en casillas. Recorre por capas, así que la primera vez que llega a un sitio ya es por el camino más corto. La variante 0-1 permite que algunos pasos sean gratis y otros cuesten uno, sin necesitar Dijkstra.

Ejemplo: en un laberinto, devuelve la longitud mínima y la secuencia de movimientos DRRU... que hay que seguir.

Bipartito por coloreo (p. 34) ¿Se puede partir el grupo en dos equipos de forma que ningún par conflictivo quede junto? Se pinta cada nodo del color contrario al de su vecino; si en algún momento choca, es imposible. Incluye la plantilla de recorrido sin recursión, necesaria en Python para grafos grandes.

Ejemplo: un ciclo de 4 nodos se puede partir; uno de 3 no, porque tiene longitud impar.

Árbol de Expansión Mínima (Kruskal) (p. 35) Conectar todo al menor costo posible: cables, carreteras, tuberías. Se ordenan las conexiones de más barata a más cara y se van tomando las que unen grupos todavía separados. También sirve para agrupar: cortar las conexiones más caras del árbol deja los grupos más separados entre sí.

Ejemplo: con tres ciudades y costos 1, 2 y 3, el mínimo para conectarlas es $1 + 2 = 3$.

Puentes y árbol de puentes (p. 36) Qué conexiones son críticas: si esa calle se corta, la red se parte. Contrayendo todo lo que sigue conectado sin ella, el grafo se convierte en un árbol, y sobre ese árbol se responden preguntas que en el grafo original serían difíciles.

Ejemplo: para maximizar cuántos tramos obligatorios hay entre dos puntos, se toma el camino más largo del árbol de puentes.

Orden Topológico + DP en DAG (p. 37) En un grafo sin ciclos, ordena las tareas de modo que nada venga antes que sus requisitos. Recorriendo en ese orden, cualquier cálculo acumulado (camino más largo, cuántos caminos hay, costo mínimo) sale en una sola pasada.

Ejemplo: la ruta que visita más ciudades de 1 a n , y la lista de ciudades por las que pasa.

Bellman-Ford (ciclos negativos) (p. 38) Para grafos donde avanzar puede DAR beneficio en vez de costar, que es justo donde Dijkstra se rompe. Además detecta el caso patológico: un ciclo que te deja mejor cada vez que lo recorres, o sea, ganancia infinita.

Ejemplo: detectar arbitraje en cambio de divisas: una secuencia de cambios que te devuelve más dinero del que pusiste.

LCA y distancias en árbol (p. 40) En un árbol genealógico o una jerarquía, el ancestro común más cercano de dos nodos. Con él se obtiene al instante la distancia entre dos nodos cualesquiera, sin recorrer el árbol en cada consulta.

Ejemplo: en un árbol de 200 000 nodos, cada pregunta de distancia se responde de inmediato tras un precálculo único.

Flujo Máximo y Corte Mínimo (Dinic) (p. 41) Cuánto se puede mandar de un punto a otro por una red con capacidades, y el resultado gemelo: cuál es el conjunto más barato de conexiones a cortar para separarlos del todo. Los dos números son el mismo, y ese es el teorema que hace útil el algoritmo.

Ejemplo: si hay 2 rutas independientes entre el banco y el puerto, hacen falta exactamente 2 cierres de calle para incomunicarlos.

2-SAT (p. 42) Cada elemento tiene dos opciones y hay restricciones de la forma "si esta, entonces aquella". Decidir

si existe una combinación que las cumpla todas es rapidísimo con dos opciones, aunque con tres el mismo problema sea intratable. Se resuelve reutilizando las componentes fuertemente conexas.

Ejemplo: con n personas que piden cada una dos deseos sobre los ingredientes, dice si hay una pizza que contente a todos.

Búsqueda, Greedy y Optimización

Búsqueda Binaria sobre la Respuesta (p. 44) El truco mental más rentable del repertorio: cuando calcular la respuesta es difícil pero VERIFICAR un candidato es fácil, se adivina el candidato a la mitad y se descarta media búsqueda cada vez. Funciona siempre que valga "si X sirve, todo lo mayor también".

Ejemplo: para la k -ésima suma más pequeña entre pares, se prueba un valor y se cuenta cuántas sumas quedan por debajo.

Enumeración por Bitmask (p. 44) Cuando hay pocas decisiones de sí/no (unas 20 como mucho) y se pueden probar todas. Cada número entero, leído en binario, representa una combinación distinta, así que un simple bucle las recorre todas sin necesidad de recursión.

Ejemplo: con 16 bits desconocidos se prueban las 65 536 combinaciones posibles y se devuelve la primera válida.

Maximizar el mínimo (p. 45) La variante más común de la búsqueda binaria: repartir algo para que el peor elemento quede lo mejor posible. "Maximizar el mínimo" y "minimizar el máximo" son la misma idea, y casi siempre se resuelven así.

Ejemplo: con una muralla de alturas desiguales y un solo refuerzo disponible, la mayor altura mínima que se puede garantizar.

Huffman (fusión greedy) (p. 46) Cuando hay que juntar (o partir) grupos repetidamente y cada operación cuesta el tamaño del grupo resultante, lo óptimo es unir siempre los dos más pequeños. La misma idea aparece disfrazada en "unir cuerdas", "mezclar archivos" y compresión de datos.

Ejemplo: pesos [1, 2, 3]: unir 1+2 (cuesta 3) y luego 3+3 (cuesta 6) da un total de 9, que es el mínimo.

Programación Dinámica

DP de Particionamiento sobre Residuos (p. 47) Partir una lista en grupos minimizando un costo, cuando el costo de cada grupo depende solo de su suma módulo un número pequeño. El estado deja de ser "dónde corté" (miles de opciones) y pasa a ser "qué resto llevaba" (diez), que es lo que vuelve viable el cálculo en Python.

Ejemplo: pagar una compra por partes cuando cada parte se redondea al múltiplo de 10 más cercano: conviene elegir dónde cortar.

Distancia de Edición y LCS (p. 48) Cuánto se parecen dos textos: el mínimo de inserciones, borrados y sustituciones para convertir uno en el otro. Es la base de los correctores ortográficos y de `diff`. La versión bit-paralela mete una columna entera de la tabla en un solo número entero, y por eso corre en Python donde la versión normal no llega.

Ejemplo: de "casa" a "calle" la distancia es 3.

Subsecuencia Creciente Más Larga (LIS) (p. 49) Cuántos elementos se pueden tomar, en orden, de forma que cada uno sea mayor que el anterior. La versión obvia compara todos los pares y es lenta; la buena guarda, para cada longitud, el valor más chico con que puede terminar, y ubica cada número nuevo con búsqueda binaria. Aparece disfrazada en apilar cajas o encajar sobres: se ordena por una dimensión y se busca la LIS en la otra.

Ejemplo: en [7, 3, 5, 3, 6, 2, 9, 8] la respuesta es 4 (3, 5, 6, 9).

Mochila 0/1 (p. 50) Elegir objetos, cada uno una sola vez, para sacar el mayor valor sin pasarse de una capacidad. El error típico es tomar primero lo más valioso: a veces conviene más una combinación de cosas medianas. La versión de esta entrada empaqueta una fila entera de la tabla en un solo entero, que es lo que la hace viable en Python.

Ejemplo: con presupuesto 10 y libros de precio 4, 8 y 5 que dan 5, 12 y 8 páginas, lo mejor no es el de 12 páginas sino el de 4 más el de 5: 13 páginas.

Caminos en Grilla (p. 51) Cuántas formas hay de cruzar una cuadrícula moviéndose solo a la derecha o hacia abajo, esquivando casillas prohibidas. Cada casilla suma lo que le llega de arriba y de la izquierda. Es el DP más simple en dos dimensiones y la base de sus variantes: costo mínimo, máximo de objetos recogidos, pasar por puntos obligados.

Ejemplo: en una grilla 3×3 vacía hay 6 caminos; si el centro es trampa, quedan 2 (los que van por el borde).

DP en Árboles (p. 52) Resolver un problema sobre un árbol yendo de las hojas hacia la raíz: cada nodo decide con lo que ya le informaron sus hijos, guardando solo unos pocos estados (por ejemplo, "estoy emparejado" o "estoy libre"). Con esa misma plantilla salen el emparejamiento máximo, el mayor grupo sin vecinos y la cobertura mínima. Se recorre sin recursión, porque en Python un árbol profundo la desbordaría.

Ejemplo: en el camino $1-2-3-4$ se pueden elegir 2 aristas sin compartir nodos; en una estrella de cuatro puntas, solo 1.

DP de Dígitos (p. 53) Contar cuántos números de un rango enorme cumplen una condición sobre sus dígitos (sin dígitos repetidos juntos, suma de dígitos múltiplo de algo, sin cierto dígito). Es imposible recorrer hasta 10^{18} números, así que se construyen dígito a dígito, recordando

solo lo necesario: si ya se quedó por debajo del límite y cuál fue el último dígito.

Ejemplo: entre 123 y 321 hay 171 números sin dos dígitos adyacentes iguales.

DP de Intervalos (p. 53) Problemas donde se decide en qué orden partir o juntar un segmento, y el orden cambia el costo. El estado es un tramo y se prueban todos sus puntos de corte. Así se resuelven cortar barras, multiplicar cadenas de matrices y fusionar montones. Una propiedad del costo (la optimización de Knuth) permite probar muchos menos puntos.

Ejemplo: en una barra de 100 con cortes en 25, 50 y 75, cortar primero en el centro cuesta 200; de izquierda a derecha, 225.

Matemáticas y Simulación

Reducción Binaria (tipo Josefo) (p. 56) Procesos donde en cada ronda se elimina uno de cada dos y la cantidad se parte por la mitad. Simular es imposible si hay miles de millones de elementos, pero como cada ronda reduce el problema a la mitad, bastan unas pocas decenas de pasos.

Ejemplo: con mil millones de personas en fila, la respuesta sale en unos 30 pasos en vez de mil millones.

Fuerza Bruta / Simulación Directa (p. 56) La entrada de reconocer cuándo NO hace falta ser listo: si los límites son pequeños, hacer exactamente lo que dice el enunciado es más rápido de escribir y mucho menos propenso a errores. Saber identificarlo ahorra minutos valiosos.

Ejemplo: cuántas personas hacen falta para que dos compartan cumpleaños: se van multiplicando probabilidades hasta pasar el 50%.

Simulación Lineal con Máximo de Prefijo (p. 56) Problemas que parecen complicados y se reducen a recorrer una vez y quedarse con el máximo (o el mínimo). La dificultad está en darse cuenta de cuál es el cuello de botella, no en programarlo.

Ejemplo: dos paredes que se acercan chocan en el tiempo que marca la fila más estrecha.

Simulación de Eventos con Colas (p. 57) Sistemas donde llegan cosas en momentos distintos y hay un recurso compartido con estado. En vez de avanzar segundo a segundo, se salta directamente al siguiente evento relevante usando una cola por tipo.

Ejemplo: una escalera que solo va en un sentido a la vez: se calcula cuándo termina de usarla la última persona.

Simulación XOR con Detección de Ciclos (p. 58) Procesos deterministas que pueden no terminar nunca. Si el estado se repite, es que está dando vueltas y hay que cortar. Guardar los estados vistos es lo que distingue "no hay solución" de un bucle infinito.

Ejemplo: interruptores que cambian grupos de bombillas: dice en cuántos pasos se apagan todas, o que nunca ocurrirá.

Invariante de Paridad (ciclos de permutación) (p. 58) Para preguntas de "¿se puede llegar de esta configuración a esta otra?" donde probar todas las combinaciones es imposible. Se busca una cantidad que NUNCA cambie por más movimientos que hagas; si vale distinto en las dos configuraciones, la respuesta es no y no hace falta buscar nada. En los rompecabezas deslizantes esa cantidad es la paridad: cada movimiento intercambia dos piezas, y eso la invierte siempre.

Ejemplo: el clásico 15-puzzle con las dos últimas piezas cambiadas es imposible de resolver, y se sabe sin mover ni una ficha.

Fracciones Unitarias (Erdős–Straus) (p. 60) Escribir una fracción como suma de fracciones de numerador 1. El espacio de búsqueda parece infinito, pero se puede acotar: la primera fracción no puede ser demasiado pequeña, y eso limita cuánto hay que probar. Es el ejemplo típico de "búsqueda que parece imposible hasta que demuestras una cota".

Ejemplo: $4/5 = 1/2 + 1/4 + 1/20$.

Construcción y Combinatoria

Constructivo por Invariante (p. 61) Problemas de "construye algo que cumpla X" donde no hay que buscar nada: se propone una fórmula y se argumenta que cierta propiedad (típicamente la paridad) nunca puede romperse, sin importar las decisiones que se tomen.

Ejemplo: construir una permutación donde el resultado nunca pueda dar cero, sin importar los signos que se elijan.

Constructivo Explícito (p. 61) El mismo tipo de problema resuelto dando directamente la respuesta a partir de una identidad numérica conocida, en vez de programar una búsqueda.

Ejemplo: un arreglo cuya suma sea divisible por cada uno de sus elementos sale de la identidad $1 = 1/2 + 1/3 + 1/6$.

Conteo por Simetría de Binomios (p. 62) Contar subconjuntos que cumplen una condición sin enumerarlos. La observación clave es que, dentro de un grupo de elementos iguales, elegir una cantidad par o impar es igual de probable, y eso colapsa el conteo a una potencia de dos.

Ejemplo: cuántas subsecuencias tienen suma alternada cero, con hasta 10^5 elementos.

Greedy por Alternancia (p. 62) Construir una secuencia larga respetando límites de repetición consecutiva. Resulta que alternar entre las dos opciones más holgadas siempre basta si el problema tiene solución, así que no hace falta buscar.

Ejemplo: repartir noches entre varios alojamientos cuando cada uno tolera un máximo de noches seguidas.

Stars and Bars (p. 63) Contar de cuántas formas se reparte una cantidad entre varios grupos, cuando los elementos son indistinguibles y un grupo puede quedarse sin nada. Es una fórmula cerrada, no un algoritmo.

Ejemplo: repartir 3 días de espera entre 2 paradas da 4 maneras posibles.

Cadenas

Rotación Mínima (Booth) (p. 64) Cuando dos palabras deben considerarse iguales si una es la otra girada ("abc" y "cab"). Da una forma canónica: la rotación alfabéticamente menor. Con eso, comparar o eliminar duplicados es comparar dos cadenas normales.

Ejemplo: "cab", "abc" y "bca" colapsan todas a "abc".

Palíndromos por normalización (p. 64) El caso simple: comprobar si un texto se lee igual al derecho y al revés ignorando espacios, mayúsculas y puntuación. Se limpia primero y se compara con su reverso; no hace falta nada más sofisticado.

Ejemplo: "Anita lava la tina" es palíndromo una vez quitados espacios y mayúsculas.

KMP sobre codificación (p. 65) Convierte un problema de árboles en uno de texto: si se escribe cada árbol como una cadena en el orden correcto, "¿este subárbol tiene la misma forma que aquel?" se vuelve "¿aparece esta cadena dentro de esta otra?", que se responde en una pasada.

Ejemplo: contar cuántos subárboles de una expresión tienen la misma forma que otra expresión dada.

Suffix Array + LCP (p. 66) La herramienta pesada de cadenas: ordena alfabéticamente todos los finales posibles del texto, y guarda cuánto comparte cada uno con el siguiente. Con eso salen la subcadena repetida más larga, cuántas subcadenas distintas hay y la parte común entre dos textos, sin revisar todas las subcadenas una por una.

Ejemplo: en "banana", la subcadena repetida más larga es "ana", de longitud 3.

Firma Canónica + Hash (p. 67) Agrupar cosas "equivalentes" sin compararlas todas contra todas. Se define una huella que sale igual para todos los equivalentes (por ejemplo, las letras ordenadas) y se usa como llave de un diccionario: cada consulta es un acceso directo.

Ejemplo: "roma", "amor" y "mora" comparten la huella "amor", así que caen en el mismo grupo.

Expresiones Regulares (p. 68) Cuando el problema se reduce a contar o localizar patrones fijos, el módulo **re** ya está escrito y optimizado. La única trampa: por defecto no cuenta apariciones que se solapan, y hay que usar una construcción especial si el patrón puede solaparse consigo mismo.

Ejemplo: en "aaaa", el conteo normal de "aa" da 2, pero las apariciones reales son 3.

Subcadenas distintas con restricción (p. 70) Contar cuántos fragmentos DISTINTOS del texto cumplen una condición, sin contar dos veces los repetidos. Combina el suffix array con una ventana que avanza, y es la generalización natural de "cuántas subcadenas distintas tiene este texto".

Ejemplo: cuántos fragmentos distintos contienen a lo sumo k letras "malas".

Z-Function (p. 71) Para cada posición dice cuánto coincide el texto consigo mismo empezando ahí. Es prima de KMP y resuelve lo mismo, pero responde una pregunta más directa: "¿cuánto del principio del texto vuelve a aparecer aquí?". Por eso es más cómoda en todo lo que trata de prefijos —bordes, repeticiones, comparar el texto con partes de sí mismo— mientras que KMP encaja mejor cuando buscas un patrón fijo dentro de un texto largo. Muchos la prefieren simplemente porque es más corta de escribir y más difícil de equivocar.

Ejemplo: en "aabaab" da $[-, 1, 0, 3, 1, 0]$: el 3 de la cuarta posición avisa que "aab" vuelve a empezar ahí. Con entrada "fixprefixsuffix" la salida es "fix": es principio, es final, y además aparece en medio.

Período Mínimo (función de prefijos) (p. 72) Detecta si una cadena es un bloque repetido y cuántas veces se repite. Sale de una sola resta sobre la tabla de KMP, y es de los trucos con mejor relación entre lo poco que cuesta y lo seguido que aparece.

Ejemplo: entrada "abcabcabc" → salida 3; entrada "abcd" → salida 1.

Manacher (p. 73) El palíndromo más largo dentro de un texto, en una sola pasada. Probar todos los centros y expandir sería demasiado lento con textos de un millón de caracteres; Manacher aprovecha lo ya calculado para no repetir comparaciones. También sirve para contar todos los fragmentos palindrómicos.

Ejemplo: entrada "aybabbu" → salida "bab".

Aho-Corasick (p. 75) Buscar MUCHOS patrones a la vez en un texto, recorriéndolo una sola vez. Correr KMP por separado para cada patrón sería lentísimo si hay miles; aquí todos los patrones se meten en una misma estructura y el texto se lee de corrido. Es lo que hay detrás de un filtro de palabras o un antivirus por firmas.

Ejemplo: con el texto "abcaab" y los patrones "ab", "bc", "xyz", la salida es 2, 1, 0.

Referencias

- [1] P. M. Fenwick, "A new data structure for cumulative frequency tables," *Software: Practice and Experience*, vol. 24, no. 3, pp. 327–336, 1994.
- [2] C. L. Bouton, "Nim, a game with a complete mathematical theory," *Annals of Mathematics*, vol. 3, no. 1/4, pp. 35–39, 1901.
- [3] R. P. Sprague, "Über mathematische kampfspiele," *Tohoku Mathematical Journal*, vol. 41, pp. 438–444, 1935.
- [4] M. A. Bender and M. Farach-Colton, "The LCA problem revisited," in *LATIN 2000: Theoretical Informatics*, ser. LNCS, vol. 1776, 2000, pp. 88–94.
- [5] H. W. Kuhn, "The hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [6] J. E. Hopcroft and R. M. Karp, "An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs," *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.
- [7] D. König, "Gráfok és mátrixok," *Matematikai és Fizikai Lapok*, vol. 38, pp. 116–119, 1931.
- [8] M. Sharir, "A strong-connectivity algorithm and its applications in data flow analysis," *Computers & Mathematics with Applications*, vol. 7, no. 1, pp. 67–72, 1981.
- [9] C. Hierholzer, "Über die möglichkeit, einen linienzug ohne wiederholung und ohne unterbrechung zu umfahren," *Mathematische Annalen*, vol. 6, no. 1, pp. 30–32, 1873.
- [10] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [11] C. Bron and J. Kerbosch, "Algorithm 457: Finding all cliques of an undirected graph," *Communications of the ACM*, vol. 16, no. 9, pp. 575–577, 1973.
- [12] A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962.
- [13] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [14] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," *Proceedings of the American Mathematical Society*, vol. 7, no. 1, pp. 48–50, 1956.
- [15] R. Bellman, "On a routing problem," *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.
- [16] E. A. Dinic, "Algorithm for solution of a problem of maximum flow in networks with power estimation," *Soviet Mathematics Doklady*, vol. 11, pp. 1277–1280, 1970.
- [17] B. Aspvall, M. F. Plass, and R. E. Tarjan, "A linear-time algorithm for testing the truth of certain quantified boolean formulas," *Information Processing Letters*, vol. 8, no. 3, pp. 121–123, 1979.
- [18] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.
- [19] G. Myers, "A fast bit-vector algorithm for approximate string matching based on dynamic programming," *Journal of the ACM*, vol. 46, no. 3, pp. 395–415, 1999.
- [20] M. L. Fredman, "On computing the length of longest increasing subsequences," *Discrete Mathematics*, vol. 11, no. 1, pp. 29–35, 1975.
- [21] R. Bellman, *Dynamic Programming*. Princeton University Press, 1957.
- [22] D. E. Knuth, "Optimum binary search trees," *Acta Informatica*, vol. 1, no. 1, pp. 14–25, 1971.
- [23] K. S. Booth, "Lexicographically least circular substrings," *Information Processing Letters*, vol. 10, no. 4-5, pp. 240–242, 1980.
- [24] D. E. Knuth, J. H. Morris, and V. R. Pratt, "Fast pattern matching in strings," *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [25] U. Manber and G. Myers, "Suffix arrays: A new method for on-line string searches," *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.
- [26] T. Kasai, G. Lee, H. Arimura, S. Arikawa, and K. Park, "Linear-time longest-common-prefix computation in suffix arrays and its applications," in *Combinatorial Pattern Matching (CPM 2001)*, ser. LNCS, vol. 2089, 2001, pp. 181–192.
- [27] D. Gusfield, *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge University Press, 1997.
- [28] G. Manacher, "A new linear-time on-line algorithm for finding the smallest initial palindrome of a string," *Journal of the ACM*, vol. 22, no. 3, pp. 346–351, 1975.
- [29] A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [30] S. Halim and F. Halim, *Competitive Programming 3: The New Lower Bound of Programming Contests*. Lulu, 2013.
- [31] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [32] cp-algorithms contributors, "cp-algorithms: Algorithms for competitive programming," <https://cp-algorithms.com>, consultado en 2026.